CENTRO ESTADUAL DE EDUCAÇÃO TECNOLÓGICA PAULA SOUZA
FACULDADE DE TECNOLOGIA DE INDAIATUBA
DR. ARCHIMEDES LAMOGLIA
CURSO DE TECNOLOGIA
EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

BRUNO ROCHA PEREIRA E GIOVANNI SILVA LOOSE

Desenvolvimento e comparação de modelos de negociação algorítmica: ARIMA-GARCH e médias móveis

Indaiatuba
2026

CENTRO ESTADUAL DE EDUCAÇÃO TECNOLÓGICA PAULA SOUZA

FACULDADE DE TECNOLOGIA DE INDAIATUBA

DR. ARCHIMEDES LAMOGLIA

CURSO DE TECNOLOGIA

EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

BRUNO ROCHA PEREIRA E GIOVANNI SILVA LOOSE

Desenvolvimento e comparação de modelos de negociação algorítmica: ARIMA-GARCH e médias móveis

Trabalho de graduação apresentado por Bruno Rocha Pereira e Giovanni Silva Loose como pré-requisito para a conclusão do Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas, da Faculdade de Tecnologia de Indaiatuba, elaborado sob a orientação da Profa. Simone Mendes da Silva.

Indaiatuba

2026

RESUMO

Este trabalho retoma um estudo desenvolvido na Faculdade de Tecnologia de Indaiatuba em 2016, no qual um robô de negociação para a plataforma MetaTrader 5, guiado pelo cruzamento de médias móveis, obteve bons resultados em períodos de mercado estável, mas acumulou prejuízo no período que inclui a crise financeira de 2008. Ao final daquele estudo, o autor sugeriu que modelos capazes de medir a volatilidade poderiam ajudar a contornar essa limitação. Partindo dessa sugestão, desenvolveu-se um novo robô que reúne dois modelos estatísticos com papéis distintos. O primeiro, chamado ARIMA, procura antecipar a direção dos preços. O segundo, chamado GARCH, estima o quanto o mercado deve oscilar e decide se vale a pena operar e com quanto. Os dois modelos foram programados na própria linguagem da plataforma e testados sobre o histórico oficial da bolsa brasileira, em períodos de crise e de calma, sempre em comparação com a estratégia original e com uma versão do robô sem o controle de volatilidade. Os resultados mostram que esse controle tornou o robô mais seguro. Em todos os períodos analisados, ele oscilou menos que a versão sem o controle e que a estratégia original, e as perdas nos piores momentos foram menores que as da versão sem o controle. Em alguns períodos, o ganho final também foi maior. A proporção de operações com lucro, porém, ficou perto da metade, com ou sem o controle, e as diferenças de ganho em relação à estratégia original variaram conforme o período, ora a favor de um, ora de outro. Conclui-se que o modelo de volatilidade funciona como um instrumento de proteção, especialmente útil em quedas prolongadas como a crise de 2008, e não como uma forma de prever melhor o mercado.

Palavras-chave: ARIMA-GARCH; negociação algorítmica; MetaTrader 5.

ABSTRACT

This work revisits a study carried out at the Faculdade de Tecnologia de Indaiatuba in 2016, in which a trading robot for the MetaTrader 5 platform, guided by moving average crossovers, performed well in stable market periods but accumulated losses over the period that includes the 2008 financial crisis. At the end of that study, the author suggested that models able to measure volatility could help overcome this limitation. Building on that suggestion, a new robot was developed that brings together two statistical models with distinct roles. The first, called ARIMA, tries to anticipate the direction of prices. The second, called GARCH, estimates how much the market is likely to fluctuate and decides whether it is worth trading and with how much. Both models were programmed in the platform's own language and tested on the official history of the Brazilian stock exchange, in periods of crisis and of calm, always in comparison with the original strategy and with a version of the robot without the volatility control. The results show that this control made the robot safer. In every period analysed, it fluctuated less than both the version without the control and the original strategy, and its losses in the worst moments were smaller than those of the version without the control. In some periods the final gain was also larger. The share of profitable trades, however, stayed close to half, with or without the control, and the differences in gain relative to the original strategy varied from period to period, sometimes favouring one and sometimes the other. The conclusion is that the volatility model works as a protective instrument, especially useful in prolonged downturns such as the 2008 crisis, and not as a way of predicting the market better.

Keywords: ARIMA-GARCH; algorithmic trading; MetaTrader 5.

LISTA DE ILUSTRAÇÕES

Figura 1 – Interface do MetaTrader 5 13
Figura 2 – Ciclo de funcionamento de um *Expert Advisor* 14
Figura 3 – Cruzamento de médias móveis 16
Figura 4 – Etapas de aplicação do modelo ARIMA 18
Figura 5 – Retroalimentação da variância no modelo GARCH 20
Gráfico 1 – Série homocedástica simulada e retorno diário da PETR4, 2019 a 2021 21
Quadro 1 – Métricas de avaliação usadas no trabalho 23
Quadro 2 – Características dos estudos relacionados 25
Quadro 3 – Configurações testadas na otimização 35
Figura 6 – Períodos de otimização e de teste 36
Quadro 4 – Forma de uso do GARCH em cada robô escolhido 40
Gráfico 2 – Retorno por ano, 2007 a 2011 41
Gráfico 3 – Patrimônio e queda acumulada, 2007 a 2011 42
Gráfico 4 – Saldo registrado pelo MetaTrader 5, 2007 a 2011 42
Gráfico 5 – Patrimônio e queda acumulada, 2008 44
Gráfico 6 – Preço da PETR4 e patrimônio aplicado na ação, 2008 44
Gráfico 7 – Retorno por ano, 2013 a 2016 45
Gráfico 8 – Patrimônio e queda acumulada, 2013 a 2016 46
Gráfico 9 – Saldo registrado pelo MetaTrader 5, 2013 a 2016 46
Gráfico 10 – Patrimônio e queda acumulada, 2017 a 2024 47
Gráfico 11 – Retorno por ano, 2017 a 2024 48
Gráfico 12 – Saldo registrado pelo MetaTrader 5, 2017 a 2024 48
Gráfico 13 – Patrimônio e queda acumulada, 2020 49
Gráfico 14 – Preço da PETR4 e patrimônio aplicado na ação, 2020 50
Gráfico 15 – Patrimônio e queda acumulada, 2025 a 2026 51
Gráfico 16 – Saldo registrado pelo MetaTrader 5, 2025 a 2026 51

LISTA DE TABELAS

Tabela 1 – Reprodução dos testes de Almeida (2016) 38
Tabela 2 – Recuperação de parâmetros em séries artificiais 39
Tabela 3 – Resultado de 2007 a 2011 41
Tabela 4 – Resultado em 2008, ano da crise financeira 43
Tabela 5 – Resultado de 2013 a 2016 45
Tabela 6 – Resultado de 2017 a 2024 47
Tabela 7 – Resultado em 2020, ano da pandemia 49
Tabela 8 – Resultado de 2025 a 2026 50
Tabela 9 – Cada versão com e sem o GARCH 52
Tabela 10 – Testes adicionais, com e sem o GARCH 52
Tabela 11 – Retorno por etapa, 2007 a 2011 54
Tabela 12 – Retorno por etapa, 2013 a 2016 54
Tabela 13 – Retorno por etapa, 2017 a 2024 54
Tabela 14 – Retorno por etapa, 2025 a 2026 55
Tabela 15 – Retorno e queda máxima em outras ações 56

LISTA DE SIGLAS

ADF	*Augmented Dickey-Fuller*
AIC	*Akaike Information Criterion*
AR	Autorregressivo
ARCH	*Autoregressive Conditional Heteroskedasticity*
ARIMA	*Autoregressive Integrated Moving Average*
ARMA	*Autoregressive Moving Average*
B3	Brasil, Bolsa, Balcão
COTAHIST	Arquivo de Cotações Históricas da B3
EA	*Expert Advisor*
EGARCH	*Exponential* GARCH
FATEC	Faculdade de Tecnologia
GARCH	*Generalized Autoregressive Conditional Heteroskedasticity*
LSTM	*Long Short-Term Memory*
MQL5	*MetaQuotes Language* 5
PETR4	Código de negociação das ações preferenciais da Petrobras
SARIMA	*Seasonal Autoregressive Integrated Moving Average*
Selic	Sistema Especial de Liquidação e de Custódia
TGARCH	*Threshold* GARCH

SUMÁRIO

INTRODUÇÃO 9

CAPÍTULO I 12
1 Fundamentação teórica 12
1.1 Mercado de ações 12
1.2 MetaTrader 5 13
1.3 Séries temporais financeiras 15
1.4 Análise técnica 15
1.4.1 Médias móveis 16
1.5 ARIMA 17
1.6 GARCH 19
1.7 Integração entre ARIMA e GARCH 21
1.7.1 Uso da volatilidade prevista na operação 22
1.8 Métricas de avaliação 22
1.9 Trabalhos relacionados 24

CAPÍTULO II 26
2 Percurso metodológico 26
2.1 Caracterização da pesquisa 26
2.1.1 Quanto aos objetivos 26
2.1.2 Quanto ao delineamento 26
2.2 Caracterização do lugar e da amostra de pesquisa 26
2.3 Procedimentos para coleta e análise de dados 27
2.3.1 Técnicas para coleta de dados 27
2.3.2 Natureza da análise de dados 28
2.4 Implementação dos modelos 28
2.4.1 Preparação dos preços 29
2.4.2 Estimação do ARIMA 29
2.4.3 Estimação do GARCH 30
2.4.4 Janela de estimação 31
2.4.5 Funcionamento do robô 31
2.4.6 Uso do GARCH na decisão 32
2.4.7 Parâmetros do robô 32
2.5 Plano experimental 33
2.5.1 Condições das simulações 33
2.5.2 Medição dos resultados 33
2.5.3 Reprodução do estudo de referência 34

2.5.4 Validação dos estimadores 34
2.5.5 Configurações testadas 35
2.5.6 Escolha da configuração 36
2.5.7 Robôs comparados 36
2.5.8 Teste complementar 37
2.6 Obstáculos encontrados 37

CAPÍTULO III 38
3 Análise e discussão dos dados 38
3.1 Reprodução do estudo de referência 38
3.2 Validação dos estimadores 39
3.3 Como os resultados são apresentados 39
3.4 Crise financeira, 2007 a 2011 41
3.4.1 O período completo 41
3.4.2 O ano de 2008 43
3.5 Queda prolongada e recuperação, 2013 a 2016 44
3.6 Teste principal, 2017 a 2024 46
3.6.1 O período completo 46
3.6.2 O ano de 2020 48
3.7 Período recente, 2025 a 2026 50
3.8 O efeito vem do GARCH? 51
3.9 Como o GARCH protege 53
3.10 Efeito dos custos, da Selic e dos proventos 54
3.11 Outras ações 55

CONSIDERAÇÕES FINAIS 57

REFERÊNCIAS 60

APÊNDICE A – Código do *Expert Advisor* 64
APÊNDICE B – Código dos modelos 68
APÊNDICE C – Código do robô de médias móveis 83

INTRODUÇÃO

Este trabalho dá continuidade ao estudo de Almeida (2016), desenvolvido nesta mesma instituição. O autor programou em MQL5, para a plataforma MetaTrader 5, um robô de negociação para a PETR4, chamado na plataforma de *Expert Advisor*, isto é, um programa que compra e vende o ativo automaticamente segundo regras definidas previamente. A regra escolhida foi o cruzamento de duas médias móveis do preço: quando a média de curto prazo cruza a de longo prazo, o robô interpreta o movimento como início de uma tendência e abre a posição. Avaliado sobre dados históricos, o robô obteve lucro entre 2013 e 2016, mas prejuízo entre 2007 e 2011, período que inclui a crise financeira de 2008. Ao discutir esse resultado, o próprio autor alertou para as limitações da técnica em mercados de alta volatilidade e indicou a investigação de modelos de volatilidade, entre eles o GARCH, como continuidade da pesquisa.

A dificuldade apontada por ele continua atual, porque as crises voltam a acontecer, e é nelas que as maiores perdas se concentram. Em março de 2020, no início da pandemia de covid-19, o Ibovespa, principal índice de ações da bolsa brasileira, caiu 29,9% em um único mês (Máximo, 2020). A própria PETR4 chegou a perder 64,2% do seu valor ao longo de 2020, do ponto mais alto ao mais baixo do ano, segundo as cotações oficiais da bolsa (B3, 2026b). Uma estratégia baseada apenas no cruzamento de médias responde a uma única pergunta, a de para que lado o preço parece estar indo, e nada diz sobre o quanto ele pode oscilar no caminho. Por isso, ela entra no mercado com a mesma convicção em um dia calmo e em um dia de forte turbulência.

Os modelos estatísticos de séries temporais permitem separar essas duas perguntas. O modelo ARIMA estima a direção esperada do preço a partir do comportamento passado da própria série, de forma parecida com uma média móvel, mas com pesos ajustados aos dados. O modelo GARCH estima a volatilidade esperada para o período seguinte. Reunidos em um mesmo algoritmo, o primeiro indicaria o lado da operação e o segundo poderia atuar como um filtro de risco, evitando operar quando a oscilação esperada fosse alta. É essa a proposta que este trabalho investiga, retomando a sugestão de Almeida (2016) dez anos depois, sobre o mesmo ativo e na mesma plataforma.

Resta saber, porém, o que efetivamente se ganha ao acrescentar essa camada de modelagem. Acrescentar um modelo de volatilidade torna o algoritmo mais complexo, exige a estimação de parâmetros adicionais a cada operação e pode reduzir o número de negócios realizados, e nada garante, de antemão, que o resultado compense esse custo. Coloca-se, assim, a questão norteadora deste trabalho: em que medida a integração dos modelos ARIMA e GARCH em um único algoritmo de negociação altera o desempenho e o risco das operações, em comparação com o uso do ARIMA isolado e com uma estratégia de cruzamento de médias móveis?

Duas hipóteses foram formuladas antes dos experimentos. A primeira é a de que a integração entre a previsão de séries temporais pelo ARIMA e a modelagem de volatilidade condicional pelo GARCH gera sinais de negociação mais eficazes, tanto em acerto direcional

quanto em adequação ao risco, do que o emprego isolado do ARIMA. A ideia de reunir, em um mesmo robô, técnicas com funções distintas aparece também em trabalhos como o de Batista (2024), que combinou indicadores técnicos. A segunda é a de que essa combinação representa um avanço em relação às análises baseadas em médias móveis. Ambas são confrontadas com os resultados nas considerações finais.

O objetivo geral foi desenvolver, em MQL5, um *Expert Advisor* que integra os modelos ARIMA e GARCH e avaliar o efeito do GARCH sobre o desempenho e o risco das operações, em comparação com o ARIMA isolado e com a estratégia de Almeida (2016). Para isso, foram definidos quatro objetivos específicos. O primeiro foi aprofundar os conceitos de tendência e de volatilidade no mercado de ações. O segundo, implementar os modelos ARIMA e GARCH em MQL5 e integrá-los em um robô de negociação no MetaTrader 5. O terceiro, reproduzir a estratégia de médias móveis de Almeida (2016). O quarto, avaliar e comparar os três algoritmos por simulação com dados históricos, em diferentes períodos do mercado, incluindo períodos de crise.

A pesquisa se justifica, em primeiro lugar, por retomar uma recomendação deixada em aberto em um estudo da própria instituição. Justifica-se, também, por levar à bolsa brasileira uma combinação de modelos que os estudos relacionados aplicaram a ações negociadas nos Estados Unidos (Xing *et al.*, 2024; Araya; Aduda; Berhane, 2024). Os testes usaram o histórico oficial da B3 e atravessaram crises que atingiram a bolsa do país, como a de 2008 e a de 2020, de modo que os resultados podem servir de referência para outras pesquisas sobre esse mercado. Há, ainda, uma contribuição técnica. O código dos modelos ARIMA e GARCH foi escrito em MQL5 para este trabalho, está reproduzido nos apêndices e pode ser reaproveitado em outros robôs e pesquisas feitos na mesma plataforma. Além disso, o controle de risco feito pelo GARCH é separado da previsão de direção, o que permite, em princípio, acoplá-lo a outras estratégias, possibilidade que este trabalho testou apenas com o ARIMA. A pesquisa é também viável com recursos gratuitos. O histórico de cotações da B3 e a taxa Selic do Banco Central são públicos, e a plataforma MetaTrader 5, o seu testador de estratégias e a biblioteca numérica que a acompanha podem ser usados sem custo.

Quanto à natureza, a pesquisa é primária, por produzir conhecimento novo a partir da realização de experimentos, e se apoia no método empírico, que fundamenta as conclusões em observações (Wazlawick, 2021). É também aplicada, por voltar esse conhecimento a um problema concreto de decisão. Quanto aos objetivos, é explicativa, quanto ao delineamento, é quase experimental, e a análise dos dados é quantitativa. Os algoritmos foram testados por simulação no MetaTrader 5, sobre o histórico oficial de preços da B3, conforme detalhado no Capítulo II.

O trabalho está organizado em três capítulos, seguidos das considerações finais. O Capítulo I apresenta a fundamentação teórica, que parte do mercado de ações e da plataforma MetaTrader 5, passa pelas séries de preços, pelas médias móveis e pelos modelos ARIMA e GARCH, e encerra-se com as métricas de avaliação e os trabalhos relacionados. O Capítulo

II descreve o percurso metodológico, isto é, a caracterização da pesquisa, a amostra, a coleta dos dados, a implementação dos modelos, o plano experimental e os obstáculos encontrados. O Capítulo III apresenta a análise e a discussão dos dados. As considerações finais respondem à questão norteadora, retomam as hipóteses e avaliam o cumprimento dos objetivos.

CAPÍTULO I

1 Fundamentação teórica

Este capítulo apresenta os conceitos sobre os quais o trabalho se apoia, na ordem em que um depende do outro. Começa pelo mercado de ações, onde as operações acontecem, e pela plataforma MetaTrader 5, onde os robôs são programados e testados. Em seguida, trata do dado que os robôs leem, a série de preços, e das regras que eles usam para decidir, primeiro as médias móveis de Almeida (2016) e depois os modelos ARIMA e GARCH. Termina com as métricas usadas para comparar os robôs e com os trabalhos relacionados.

1.1 Mercado de ações

De acordo com o Banco Central do Brasil (2025a), as bolsas de valores são sociedades anônimas ou associações civis que mantêm local ou sistema adequado à realização de operações de compra e venda de títulos e valores mobiliários, em mercado livre e aberto, organizado e fiscalizado por seus membros e pela Comissão de Valores Mobiliários. No Brasil, esse papel é exercido pela B3.

Quando uma empresa abre o capital, ela o divide em ações e as vende a investidores. Cada ação é uma pequena parte da empresa, e quem a compra se torna sócio do negócio, com direito a uma parte dos lucros distribuídos, os dividendos. As ações ordinárias dão direito a voto nas decisões da empresa. As preferenciais não dão esse direito, mas têm prioridade no recebimento dos dividendos. Depois dessa primeira venda, as ações passam a ser negociadas entre os próprios investidores na bolsa, e o preço de cada uma muda a cada pregão, isto é, a cada dia de negociação, conforme a oferta e a procura. É dessa variação que as estratégias de negociação procuram tirar proveito.

O investidor não negocia diretamente na bolsa. O acesso se dá por meio de corretoras, instituições que intermediam as operações e oferecem serviços como plataformas de negociação pela internet, custódia dos títulos e financiamento para a compra de ações, cobrando por isso comissões e taxas (Banco Central do Brasil, 2025b). É pela corretora que uma plataforma como o MetaTrader 5 se conecta ao mercado.

A taxa Selic é a taxa básica de juros da economia brasileira. Ela serve de base para as demais taxas praticadas no mercado e influencia tanto o custo do crédito quanto o retorno das aplicações financeiras (Banco Central do Brasil, 2025c). Por esse motivo, o capital que uma estratégia mantém fora do mercado costuma ser comparado ao que renderia se estivesse aplicado a essa taxa.

1.2 MetaTrader 5

O MetaTrader 5 é uma plataforma de negociação multimercado desenvolvida pela MetaQuotes, que reúne em um mesmo ambiente a execução de ordens, a análise gráfica e o desenvolvimento de sistemas automatizados para ações, futuros, moedas e outros instrumentos (MetaQuotes Ltd., 2024). Sua interface, mostrada na Figura 1, organiza em janelas as cotações em tempo real, os gráficos de preços e o navegador, que dá acesso às contas, aos indicadores e aos programas instalados.

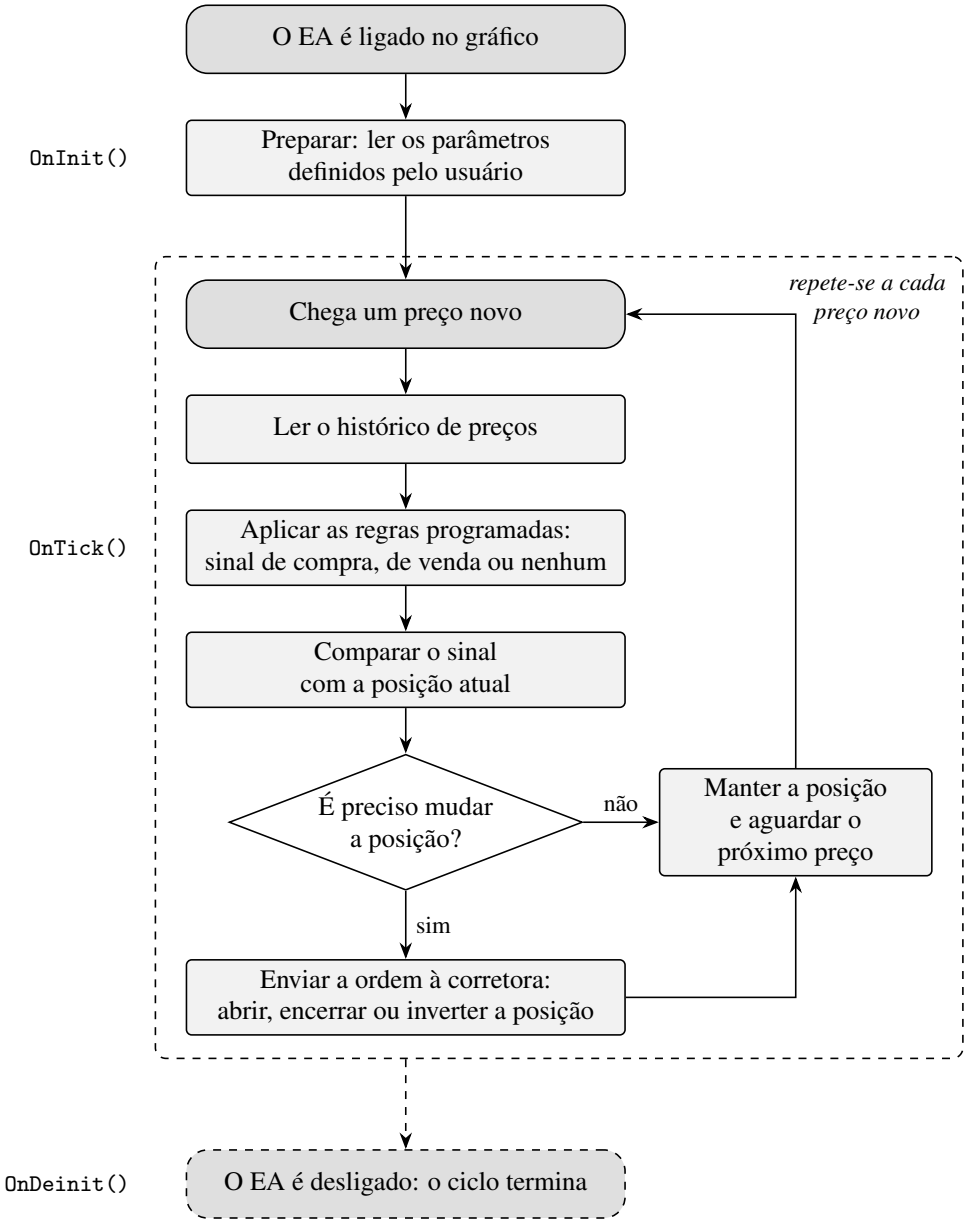
Figura 1 – Interface do MetaTrader 5



Fonte: MetaQuotes Ltd. (2024).

Os programas que negociam de forma automática na plataforma são chamados de *Expert Advisors* (EAs), também conhecidos como robôs de negociação. Um EA abre, modifica e encerra posições segundo regras programadas, sem intervenção manual (Almeida, 2016). Os EAs são escritos em MQL5, linguagem própria da plataforma, de sintaxe próxima à do C++, no ambiente de desenvolvimento chamado MetaEditor, que oferece editor, compilador e depurador integrados. O programa funciona por eventos, cada um tratado por uma função própria. A função `OnInit` é chamada uma vez, quando o EA é carregado no gráfico, a função `OnTick`, a cada novo preço recebido, e a função `OnDeinit`, antes de o EA ser retirado (MetaQuotes Ltd., 2026b). A Figura 2 apresenta o ciclo de funcionamento de um EA, com a função responsável por cada etapa.

Figura 2 – Ciclo de funcionamento de um *Expert Advisor*



Fonte: Elaboração própria.

Para avaliar uma estratégia antes de usá-la, a plataforma dispõe do testador de estratégias, o *Strategy Tester*, que executa o EA sobre dados históricos, simulando as ordens como se o mercado estivesse aberto, e produz relatórios de desempenho (MetaQuotes Ltd., 2024). Esse procedimento é conhecido como *backtesting*, e foi por meio dele que todos os experimentos deste trabalho foram conduzidos.

O testador também faz muitos testes de uma só vez, recurso chamado de otimização. Nele, o robô é executado várias vezes sobre o mesmo período, cada vez com uma combinação diferente de parâmetros, e os resultados ficam lado a lado para que se escolha a combinação mais adequada (MetaQuotes Ltd., 2026a). A otimização pode percorrer todas as combinações possíveis, no modo de busca completa, que é o mais lento, ou apenas parte delas, no modo rápido,

guiado por um algoritmo (MetaQuotes Ltd., 2026d).

Escolher a melhor combinação no mesmo período em que ela é avaliada, porém, favorece a que apenas se ajustou bem àquele trecho do passado. Para evitar isso, o testador oferece o teste avançado, também chamado de teste *forward*, que divide o período em duas partes. Os parâmetros são escolhidos na primeira e verificados na segunda, que não participou da escolha (MetaQuotes Ltd., 2026a). Neste trabalho, as duas etapas foram feitas no testador do MetaTrader 5, com a otimização em um período e o teste da combinação escolhida no período seguinte, como se descreve no Capítulo II.

1.3 Séries temporais financeiras

Uma série temporal é uma sequência de observações ordenadas no tempo. Neste trabalho, é a sequência dos preços de fechamento de uma ação, um por pregão, e é esse o dado que os robôs leem para decidir. Como o que interessa a quem investe são os ganhos e as perdas, mais do que o preço em si, as séries financeiras costumam ser estudadas pela variação do preço de um dia para o outro, chamada de retorno. O retorno tem ainda a vantagem de não depender do nível do preço, o que permite comparar ativos e períodos diferentes (Thomaz, 2019).

As séries de preços e de retornos de ações têm comportamentos bem conhecidos (Thomaz, 2019). O preço anda sem rumo definido, como um passeio aleatório, e não tende a voltar a uma média. O retorno, ao contrário, oscila em torno de zero. Além disso, o retorno de um dia diz pouco sobre o do dia seguinte, mas o tamanho das oscilações se repete. Dias agitados costumam vir seguidos de dias agitados, e dias calmos de dias calmos, fenômeno conhecido como agrupamento de volatilidade.

Essas duas características explicam a divisão de tarefas entre os modelos deste trabalho. Como a direção do preço é difícil de prever, ela fica a cargo de um modelo que procura aproveitar a pouca dependência que existe entre os retornos, o ARIMA. Como o tamanho das oscilações é mais previsível, ele fica a cargo de um modelo feito para isso, o GARCH. Antes deles, apresenta-se a regra mais simples aplicada à série de preços, a média móvel usada por Almeida (2016).

1.4 Análise técnica

A análise técnica busca antecipar o comportamento dos preços a partir do estudo do seu próprio histórico, procurando padrões e tendências que possam indicar o movimento seguinte. Sua origem remonta ao comércio de arroz, e hoje ela é aplicada a ações, moedas e outros ativos. Para essa abordagem, o que importa é o comportamento do preço, sem considerar notícias ou outros fatores que possam influenciar o mercado (Fernandes, 2014).

Fernandes (2014) distingue duas vertentes. A empírica, ou clássica, é a mais antiga: o analista observa o gráfico e identifica tendências, suportes, resistências e padrões que, segundo essa abordagem, tendem a se repetir. Por depender da interpretação de quem observa, ela é

sujeita a conclusões diferentes sobre o mesmo ativo. A estatística, ou moderna, substitui a interpretação visual por cálculos aplicados à série de preços, o que torna a análise reproduzível e permite que ela seja executada por um programa de computador.

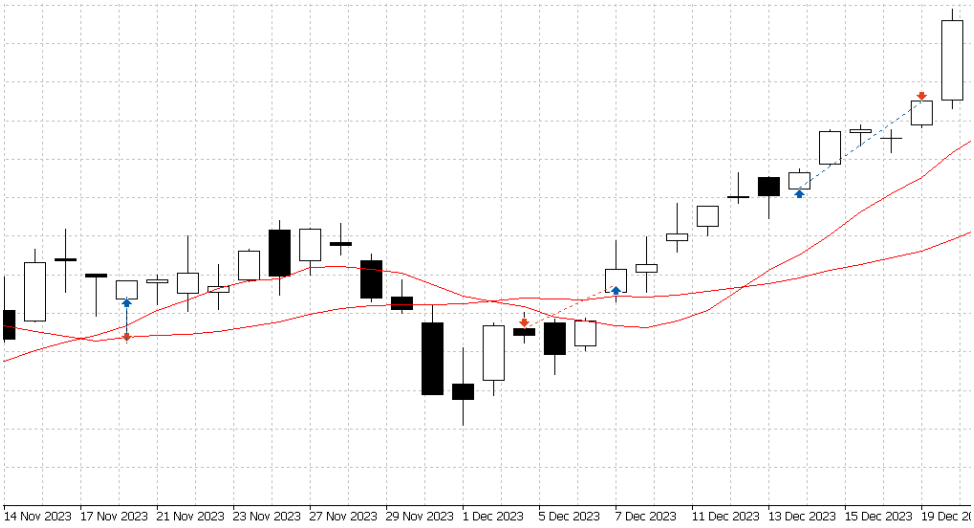
É na segunda vertente que este trabalho se insere. Tanto a estratégia de médias móveis de Almeida (2016) quanto os modelos ARIMA e GARCH são regras matemáticas aplicadas ao histórico de preços, e por isso podem ser programados, testados sobre dados passados e comparados entre si sob as mesmas condições.

1.4.1 Médias móveis

A média móvel é um dos instrumentos mais utilizados da análise técnica. Ela calcula o preço médio de um ativo nos últimos n períodos e é recalculada a cada novo período, descartando o valor mais antigo e incorporando o mais recente. Com isso, suaviza as oscilações de curto prazo e evidencia a direção predominante dos preços (Fernandes, 2014). Na média móvel simples, todos os preços da janela recebem o mesmo peso, de modo que a média de dez períodos atribui peso de um décimo a cada um dos dez últimos fechamentos.

O tamanho da janela determina o comportamento da média. Janelas curtas reagem rapidamente a mudanças de preço, mas produzem mais sinais falsos, enquanto janelas longas filtram melhor o ruído, à custa de reconhecer uma nova tendência com atraso (Fernandes, 2014). A estratégia de cruzamento combina as duas: utiliza uma média rápida, de janela curta, e uma lenta, de janela longa, e interpreta o cruzamento entre elas como sinal de compra ou de venda, como ilustra a Figura 3.

Figura 3 – Cruzamento de médias móveis



Fonte: Elaboração própria com dados e interface do MetaTrader 5.

Foi essa a estratégia adotada por Almeida (2016). O autor utilizou duas médias móveis simples, uma de dez e outra de vinte períodos, calculadas sobre o preço de fechamento, e operou nos dois sentidos, comprando quando a média rápida cruzava a lenta para cima e vendendo no cruzamento inverso. Seu algoritmo previa ainda parâmetros de limite de perda e de realização

de lucro, embora os testes relatados tenham sido conduzidos com ambos nulos. O próprio autor alertou para as limitações de se usar apenas essa técnica em mercados de alta volatilidade (Almeida, 2016).

1.5 ARIMA

O modelo autorregressivo integrado de médias móveis, conhecido pela sigla ARIMA, do inglês *Autoregressive Integrated Moving Average*, é um dos modelos mais empregados na previsão de séries temporais, inclusive em séries financeiras (Xing *et al.*, 2024). Ele é composto por três partes, cada uma associada a uma ordem.

A parte autorregressiva, de ordem p, expressa o valor atual da série como combinação dos seus p valores anteriores. A parte de médias móveis, de ordem q, acrescenta a influência dos q erros de previsão anteriores, isto é, das diferenças entre o que o modelo previu e o que de fato ocorreu. A parte integrada, de ordem d, indica quantas vezes a série precisa ser diferenciada, ou seja, substituída pela variação entre uma observação e a anterior, para se tornar estacionária (Xing *et al.*, 2024). De forma simplificada, uma série é estacionária quando as suas propriedades estatísticas, como a média e a variância, não se alteram ao longo do tempo, condição exigida pelas partes autorregressiva e de médias móveis.

Convém registrar que a média móvel do ARIMA não é a mesma média móvel da análise técnica. Na análise técnica, a média é calculada sobre os preços. No ARIMA, o termo de médias móveis é uma combinação dos erros de previsão passados. Os dois conceitos têm o mesmo nome, mas descrevem coisas diferentes.

Representando por y_t a série já diferenciada d vezes, o modelo pode ser escrito como

$$y_t = c + \phi_1 y_{t-1} + \cdots + \phi_p y_{t-p} + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \cdots + \theta_q \varepsilon_{t-q},$$

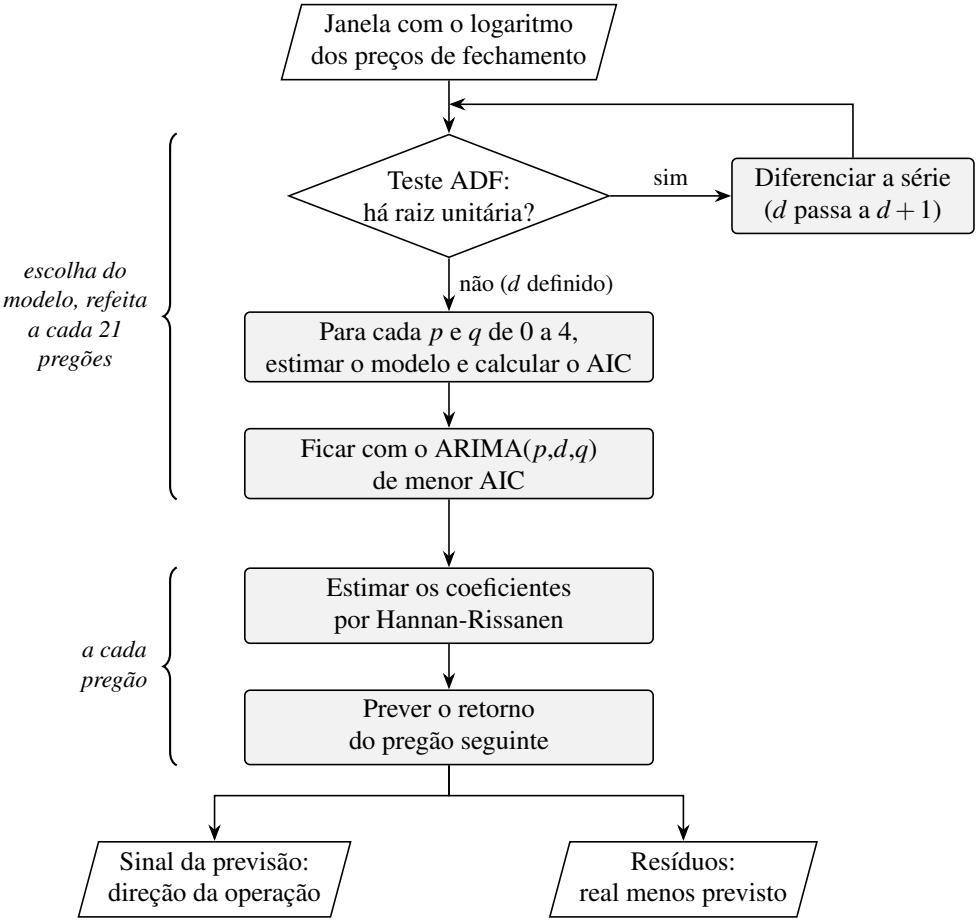
em que c é uma constante, os coeficientes ϕ ponderam os valores passados da série, os coeficientes θ ponderam os erros passados e ε_t é o erro do período atual, que o modelo não consegue prever.

O ARIMA supõe, portanto, que a série diferenciada segue um modelo ARMA(p,q), que tem apenas as partes autorregressiva e de médias móveis (Thomaz, 2019). Quando a série não precisa ser diferenciada ($d = 0$), o próprio ARIMA é um ARMA, e quando também não há termos de médias móveis ($q = 0$), o modelo é chamado de AR(p), ou autorregressivo.

Há uma relação direta entre essa equação e a estratégia de Almeida (2016). A média móvel simples de dez períodos é um filtro linear que atribui pesos fixos, iguais a um décimo, aos últimos preços. A parte autorregressiva do ARIMA também é um filtro linear sobre os valores passados, mas seus pesos ϕ são estimados a partir dos dados, em vez de fixados por convenção. A diferença é que, com $d = 1$, esse filtro é aplicado aos retornos, e a média móvel, aos preços. Nesse sentido, o ARIMA pode ser visto como uma generalização da média móvel: mantém a mesma família de operação e ajusta os coeficientes ao ativo e ao período analisados.

A aplicação do modelo segue as etapas representadas na Figura 4.

Figura 4 – Etapas de aplicação do modelo ARIMA



Fonte: Elaboração própria.

A primeira etapa é verificar se a série é estacionária. Uma série estacionária oscila em torno de uma média estável, e um choque que a afasta dessa média tende a se desfazer com o tempo. Os preços de ações, ao contrário, costumam se comportar como um passeio aleatório, em que cada choque se incorpora ao preço e não se desfaz, de modo que a variância da série cresce com o tempo (Thomaz, 2019). Esse comportamento é chamado de raiz unitária. Para identificá-lo, usa-se o teste de Dickey-Fuller aumentado (ADF), que estima, por regressão, o quanto a variação de cada dia depende do nível da série no dia anterior. Se a série tende a voltar à sua média, essa dependência é negativa e forte. Se não tende, ela é próxima de zero, e a série tem raiz unitária (Thomaz, 2019).

A hipótese nula do teste é a existência de raiz unitária. O valor-p indica a probabilidade de se obter um resultado tão favorável à estacionariedade quanto o observado se a série de fato tivesse raiz unitária. Como a estatística do teste não segue as distribuições usuais, os seus valores críticos foram tabelados especialmente para ele, e a hipótese nula é rejeitada quando a estatística fica abaixo do valor crítico, o que corresponde a um valor-p pequeno, em geral menor que 0,05 (Thomaz, 2019). Quando a hipótese não é rejeitada, a série é diferenciada e testada de novo. No estudo de Xing *et al.* (2024) sobre as ações da NVIDIA, o teste aplicado aos preços resultou em valor-p de 0,998773, isto é, sem nenhum indício contra a raiz unitária. Após uma diferenciação,

o valor-p ficou próximo de zero, e a ordem de integração foi definida como $d = 1$.

Definido d, resta escolher p e q. Como várias combinações são possíveis, Xing *et al.* (2024) testaram valores de zero a quatro para cada ordem e escolheram a combinação de menor critério de informação de Akaike (AIC), dado por

$$\mathrm{AIC} = -2\ln\hat{L} + 2k,$$

em que $\hat{L}$ é a verossimilhança do modelo ajustado, que mede o quão provável é a série observada segundo o modelo, e k é o número de parâmetros estimados (Thomaz, 2019). A primeira parcela diminui quanto melhor o ajuste, e a segunda aumenta a cada parâmetro acrescentado. Um modelo com mais parâmetros sempre se ajusta um pouco melhor aos dados passados, mas parte desse ganho é apenas ruído reproduzido, que não ajuda a prever. A penalidade faz com que um parâmetro a mais só compense se melhorar o ajuste mais do que custa. Ajusta-se, então, cada combinação candidata, e escolhe-se a de menor AIC, que é a que melhor equilibra ajuste e simplicidade. Com o modelo escolhido, obtêm-se a previsão para o período seguinte e os resíduos, que são os erros de previsão observados na amostra.

Segundo Xing *et al.* (2024) e Araya, Aduda e Berhane (2024), a principal limitação do ARIMA é tratar apenas relações lineares e supor que a variância dos erros é constante. Em séries financeiras essa segunda suposição raramente se verifica, e é dela que trata o modelo GARCH.

1.6 GARCH

O modelo autorregressivo generalizado de heterocedasticidade condicional, conhecido pela sigla GARCH, do inglês *Generalized Autoregressive Conditional Heteroskedasticity*, é usado para estimar e prever a volatilidade de séries financeiras, isto é, o grau de variação dos preços. O ARIMA procura prever para onde o preço vai, e o GARCH, o quanto ele deve oscilar. Conhecer a volatilidade é fundamental para a gestão de risco (Araya; Aduda; Berhane, 2024).

O modelo tem origem no ARCH, proposto por Engle (1982), em que a variância do erro de cada período depende dos erros dos períodos anteriores. Para acompanhar a volatilidade real, porém, o ARCH costumava precisar de muitos erros passados na equação (Bollerslev, 1986, p. 308). Bollerslev (1986) resolveu isso acrescentando à equação a própria variância estimada nos períodos anteriores. Como essa variância já resume os erros que vieram antes dela, poucos termos passam a bastar para representar uma memória longa. Daí vem o G da sigla, de generalizado. O GARCH está para o ARCH assim como o ARMA está para o AR (Bollerslev, 1986, p. 308).

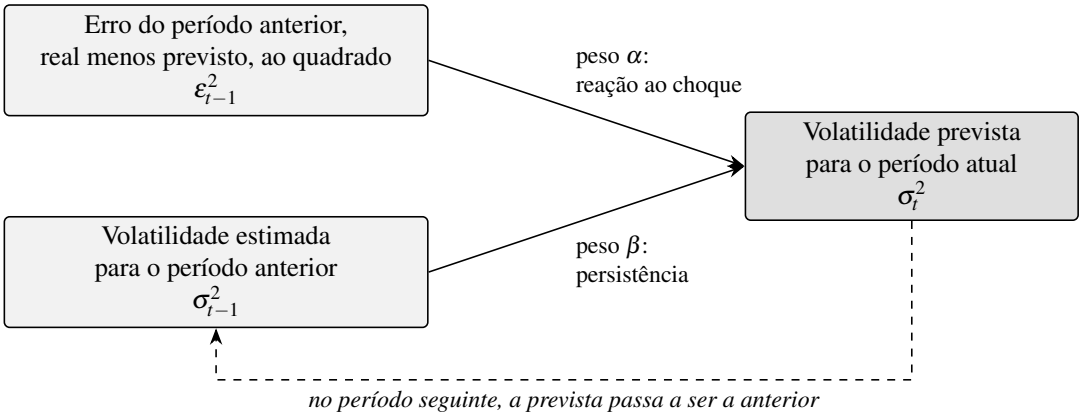
Na especificação GARCH(1,1), adotada neste trabalho, a variância condicional σ_t^2 do erro no período t é dada por

$$\sigma_t^2 = \omega + \alpha\,\varepsilon_{t-1}^2 + \beta\,\sigma_{t-1}^2,$$

em que ε_{t-1} é o erro do período anterior, σ_{t-1}^2 é a variância estimada para o período anterior e ω, α e β são os parâmetros do modelo. O erro é a diferença entre o valor que de fato ocorreu e o valor que se esperava, ou seja, a parte da variação que foi surpresa.

A Figura 5 mostra como a equação funciona a cada período. A volatilidade prevista para o período atual combina duas informações do período anterior. A primeira é o erro elevado ao quadrado, que mede o tamanho da surpresa. Por estar ao quadrado, uma alta e uma queda inesperadas de mesmo tamanho pesam igualmente, porque o que interessa aqui é o tamanho da variação. Essa parcela é multiplicada por α, que representa a reação da volatilidade a um choque recente. A segunda é a própria volatilidade estimada para o período anterior, multiplicada por β, que representa a persistência da volatilidade passada (Bollerslev, 1986). A essas duas parcelas soma-se a constante ω.

Figura 5 – Retroalimentação da variância no modelo GARCH



Fonte: Elaboração própria.

No período seguinte, a volatilidade que acabou de ser prevista passa a ser a anterior, e o cálculo se repete, como indica a seta tracejada da figura. É essa retroalimentação que faz a volatilidade estimada vir em ondas. Um dia de surpresa grande eleva a previsão do dia seguinte por meio de α. Essa previsão mais alta continua pesando nos dias seguintes por meio de β, de modo que o efeito do choque se dissipa aos poucos. Da mesma forma, uma sequência de dias calmos, com erros pequenos, faz a volatilidade prevista cair gradualmente.

Os parâmetros precisam respeitar algumas restrições (Bollerslev, 1986). O parâmetro ω deve ser positivo, e α e β não podem ser negativos, porque uma variância nunca é negativa. Além disso, a soma $\alpha+\beta$ deve ser menor que um. Se fosse igual ou maior, um choque nunca se dissiparia e a volatilidade prevista poderia crescer sem limite. Com a soma menor que um, a variância tem um valor de longo prazo, igual a $\omega/(1-\alpha-\beta)$, para o qual a volatilidade volta depois de um choque (Bollerslev, 1986). Quanto mais próxima de um for essa soma, mais devagar essa volta acontece.

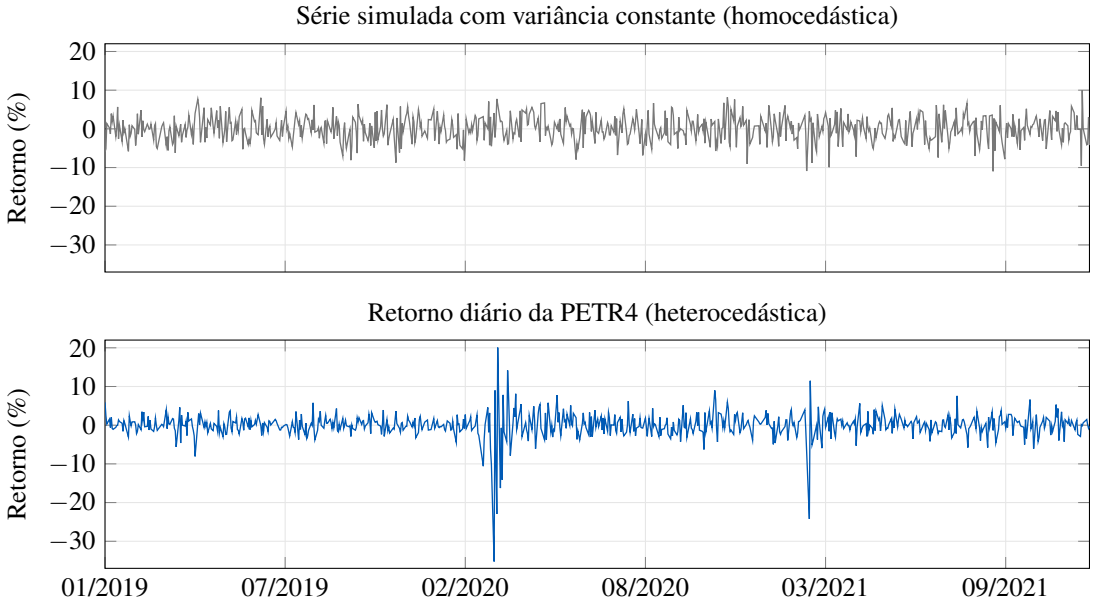
O GARCH padrão trata de forma simétrica os choques positivos e os negativos de mesma magnitude. Nos mercados de ações, porém, as quedas costumam elevar a volatilidade mais do que as altas. Para representar essa assimetria foram propostas variações do modelo, como o EGARCH e o TGARCH, que acrescentam à equação termos capazes de fazer a volatilidade reagir de forma diferente a choques positivos e negativos (Araya; Aduda; Berhane, 2024). Este trabalho emprega a especificação simétrica, e as variações assimétricas ficam como possibilidade

para trabalhos futuros.

Por fim, o nome do modelo vem do comportamento que ele representa. Uma série é homocedástica quando a sua variância, que mede o quanto os valores se espalham em torno da média, é constante ao longo do tempo (Thomaz, 2019, p. 47). Em outras palavras, ela oscila sempre com a mesma intensidade. Quando a variância muda com o tempo, a série é heterocedástica. Os modelos econométricos tradicionais supõem variância constante (Engle, 1982, p. 987), e o ARIMA também. Quando a variância de cada período depende do que aconteceu nos períodos anteriores, fala-se em heterocedasticidade condicional, que é justamente o que o GARCH descreve.

O Gráfico 1 ilustra a diferença. Na série simulada, de variância constante, as oscilações ficam sempre dentro da mesma faixa. No retorno diário da PETR4, longos trechos de oscilações pequenas se alternam com períodos de oscilações grandes, como no início de 2020.

Gráfico 1 – Série homocedástica simulada e retorno diário da PETR4, 2019 a 2021



Fonte: Elaboração própria com dados de B3 (2026b).

Nota: Retorno logarítmico do preço de fechamento, em porcentagem. A série simulada tem distribuição normal, média zero e o mesmo desvio-padrão da PETR4 no período.

1.7 Integração entre ARIMA e GARCH

Os dois modelos se complementam. O ARIMA estima a média condicional da série, ou seja, o valor esperado para o período seguinte. O GARCH estima a variância condicional dos erros desse valor esperado. Na forma combinada, os resíduos gerados pelo ARIMA servem de entrada para o GARCH, que modela a variância desses resíduos ao longo do tempo (Araya; Aduda; Berhane, 2024).

Em uma estratégia de negociação, essa combinação permite separar duas perguntas. A direção da operação é dada pelo sinal da previsão do ARIMA, e o risco associado a ela é dado

pela volatilidade prevista pelo GARCH. Como o GARCH não fornece nenhuma informação sobre a direção dos preços, ele não pode, sozinho, gerar uma ordem de compra ou de venda. O que ele pode fazer é condicionar a operação indicada pelo ARIMA ao nível de risco esperado, impedindo a operação ou reduzindo o seu tamanho quando a volatilidade prevista é alta.

1.7.1 Uso da volatilidade prevista na operação

A volatilidade prevista pelo GARCH pode entrar na decisão de duas maneiras básicas, que respondem a perguntas diferentes.

A primeira é o filtro de volatilidade, que decide se a operação é feita. Define-se um limiar, e a operação indicada pelo ARIMA só é realizada quando a volatilidade prevista está abaixo dele. Nos dias em que se espera uma oscilação acima do limiar, a operação é vetada, e o investidor deixa de se expor justamente quando o risco esperado é maior.

A segunda é o ajuste do tamanho da posição, técnica conhecida como *volatility targeting*, ou volatilidade-alvo, que decide quanto aplicar. Em vez de manter o valor aplicado constante, procura-se manter constante o nível de oscilação da carteira, aumentando a posição quando a volatilidade está baixa e reduzindo-a quando está alta (Harvey *et al.*, 2018). Define-se um nível de oscilação considerado aceitável, o alvo, e o tamanho da posição é multiplicado pela razão entre esse alvo e a volatilidade prevista. Se a volatilidade prevista dobra, a posição cai pela metade. A técnica parte do fato de que a volatilidade é persistente, isto é, períodos de oscilação alta tendem a ser seguidos por oscilação alta, a mesma característica que o GARCH modela. Em um estudo com mais de 60 ativos, Harvey *et al.* (2018) observaram que esse ajuste reduziu a ocorrência de retornos extremos em todos eles e, nas ações, também melhorou a relação entre retorno e risco.

As duas maneiras podem ser combinadas. Nesse caso, o filtro veta a operação nos dias de volatilidade prevista mais alta e, nos dias liberados, a posição é menor quanto maior for a volatilidade prevista.

1.8 Métricas de avaliação

Avaliar uma estratégia de negociação apenas pelo lucro final esconde o caminho percorrido até ele. Duas estratégias com o mesmo retorno podem ter exposto o investidor a riscos muito diferentes. Por isso, o relatório do testador de estratégias do MetaTrader 5 apresenta, ao lado do lucro, medidas de risco e índices que relacionam as duas coisas (MetaQuotes Ltd., 2026c). O Quadro 1 resume as métricas usadas neste trabalho para avaliar e comparar os robôs.

Risco é a possibilidade de que um evento inesperado e desfavorável produza um retorno menor que o esperado, ou mesmo uma perda. Em finanças, ele costuma ser medido pela variância ou pelo desvio-padrão dos retornos, isto é, pela volatilidade. Volatilidade e risco, porém, não são a mesma coisa, porque a volatilidade mede a dispersão para os dois lados, enquanto o risco está ligado às perdas (Thomaz, 2019, p. 22-23).

Quadro 1 – Métricas de avaliação usadas no trabalho

Métrica	O que indica	Como é calculada
Retorno acumulado	quanto o patrimônio cresceu ou diminuiu no período	patrimônio final dividido pelo inicial, menos um
Queda máxima	o pior tombo que um investidor teria sofrido seguindo a estratégia	maior queda percentual do patrimônio, ao fim de cada pregão, entre um ponto alto e o ponto mais baixo seguinte
Volatilidade	o quanto o patrimônio sobe e desce de um dia para o outro	desvio-padrão dos retornos diários do patrimônio, em base anual
Negociações	quantas operações o robô realizou	contagem do relatório do testador
Taxa de acerto	a proporção de negociações que terminaram com lucro	negociações com lucro, descontados os custos de negociação, divididas pelo total
Sinais vetados	quantas vezes o GARCH impediu uma operação indicada pelo ARIMA	contagem registrada pelo robô
Índice de Sharpe	o ganho além de uma referência por unidade de oscilação	média do retorno diferencial diário dividida pelo seu desvio-padrão, em base anual
Índice de Calmar	o ganho por unidade de queda máxima	retorno anualizado dividido pela queda máxima

Fonte: Elaboração própria, com base em MetaQuotes Ltd. (2026c) e Sharpe (1994).

Por isso, o risco é medido aqui de duas formas complementares. A queda máxima, que o relatório da plataforma chama de rebaixamento relativo e que também é conhecida pelo termo em inglês *drawdown*, é a maior queda percentual entre um máximo local do patrimônio e o mínimo local seguinte (MetaQuotes Ltd., 2026c). Neste trabalho, ela foi calculada pelo próprio robô sobre o patrimônio ao fim de cada pregão. Ela olha apenas para as perdas e responde à pergunta de qual foi o pior tombo do caminho. A volatilidade, por sua vez, é o desvio-padrão dos retornos e mede o quanto o patrimônio oscila no dia a dia, para cima e para baixo.

Como os retornos são diários, a volatilidade é convertida para base anual, o que facilita a leitura e a comparação. Se os retornos de dias diferentes forem independentes entre si, a variância de um período de T dias é T vezes a variância diária, e o desvio-padrão, a raiz de T vezes o diário (Sharpe, 1994). Usou-se $T = 252$, valor próximo da média de 248 pregões por ano observada na série da PETR4 usada neste trabalho. A volatilidade anual é, portanto, o desvio-padrão diário multiplicado pela raiz de 252, e uma oscilação típica de 2% ao dia corresponde a cerca de 32% ao ano.

O índice de Sharpe relaciona o retorno de uma estratégia à sua volatilidade. Ele divide o retorno diferencial, isto é, a diferença entre o retorno da estratégia e o de uma referência, pelo desvio-padrão dessa diferença (Sharpe, 1994),

$$S = \frac{\bar{d}}{\sigma_d},$$

em que $\bar{d}$ é a média do retorno diferencial diário e σ_d o seu desvio-padrão. Como na volatilidade, o índice foi convertido para base anual, multiplicando-se o valor diário pela raiz de 252. O índice indica quanto se ganhou além da referência por unidade de oscilação, e quanto maior, melhor a relação. Um índice negativo indica que a estratégia rendeu menos que a referência. O testador

do MetaTrader 5 usa como referência um rendimento livre de risco igual a zero (MetaQuotes Ltd., 2026c), e neste trabalho a referência foi a taxa Selic.

O índice de Calmar relaciona o retorno à queda máxima. Ele divide o retorno anualizado pela queda máxima do período,

$$C = \frac{\text{retorno anualizado}}{\text{queda máxima}},$$

e segue a mesma ideia do fator de recuperação do relatório da plataforma, que divide o lucro obtido pelo rebaixamento máximo (MetaQuotes Ltd., 2026c), com a diferença de usar o retorno anualizado, o que permite comparar períodos de durações diferentes. O índice indica quanto se ganhou por unidade do pior tombo e, por isso, favorece estratégias que evitam perdas grandes. Os dois índices medem coisas diferentes. O Sharpe penaliza toda oscilação, para cima ou para baixo, e o Calmar considera apenas a pior queda.

1.9 Trabalhos relacionados

Os trabalhos relacionados foram selecionados entre pesquisas sobre o desenvolvimento de robôs de negociação na plataforma MetaTrader 5 e sobre a aplicação dos modelos ARIMA e GARCH à previsão de séries de preços de ações.

O estudo de Almeida (2016), desenvolvido na FATEC Indaiatuba, é o ponto de partida deste trabalho. O autor desenvolveu em MQL5 um robô de negociação para a PETR4 baseado no cruzamento de médias móveis simples de dez e vinte períodos e o avaliou por *backtesting* em períodos de crise, entre 2007 e 2011, e fora de crise, entre 2013 e 2016. A estratégia apresentou lucro no período fora de crise e prejuízo no período de crise. Ao discutir os limites do trabalho, o autor indicou a investigação de modelos de volatilidade, entre eles o GARCH, como continuidade da pesquisa. É essa indicação que o presente trabalho retoma, mantendo o ativo, a plataforma e a linguagem.

Batista (2024) também desenvolveu um *Expert Advisor* em MQL5 para a B3, combinando três indicadores técnicos, a média móvel exponencial, a convergência e divergência de médias móveis e o índice de força relativa. Os testes, realizados sobre o ativo BBAS3F em 2023, compararam os indicadores isoladamente, em pares e em conjunto, e a combinação dos três apresentou o melhor resultado. O estudo mostra que a integração de técnicas com funções distintas em um mesmo algoritmo pode melhorar a decisão, ideia que este trabalho aplica aos modelos estatísticos.

Xing *et al.* (2024) compararam o ARIMA, o ARIMA-GARCH e dois modelos de redes neurais na previsão do preço de fechamento das ações da NVIDIA, com dados diários de abril de 2019 a abril de 2024. As ordens dos modelos ARIMA foram definidas pelo teste ADF e pelo menor AIC, com p e q variando de zero a quatro, procedimento semelhante ao adotado neste trabalho. O ARIMA-GARCH apresentou a menor raiz do erro quadrático médio, e o ARIMA com janela expansiva teve o melhor desempenho geral, superando as redes neurais. Os autores atribuem esse resultado, possivelmente, ao comportamento predominantemente linear da série.

Araya, Aduda e Berhane (2024) estudaram a previsão da volatilidade das ações da Apple. No modelo proposto, os resíduos de um modelo SARIMA, versão do ARIMA com componentes sazonais, alimentam um modelo GARCH, e a volatilidade estimada por essa combinação é depois utilizada como entrada de redes neurais. A etapa que interessa a este trabalho é a primeira, em que os resíduos do modelo de média servem de entrada para o modelo de variância, mesma arquitetura adotada no algoritmo aqui desenvolvido.

O Quadro 2 resume as características desses estudos e as deste trabalho.

Quadro 2 – Características dos estudos relacionados

Estudo	Ativo	Fonte de dados	Periodicidade	Método
Almeida (2016)	PETR4	MetaTrader 5	diário	Cruzamento de médias móveis simples (10 e 20)
Batista (2024)	BBAS3F	MetaTrader 5	30 minutos	Combinação de três indicadores técnicos
Xing *et al.* (2024)	NVIDIA	Yahoo Finance	diário	ARIMA e ARIMA-GARCH, comparados a redes neurais
Araya, Aduda e Berhane (2024)	Apple	Yahoo Finance	diário	Resíduos do SARIMA como entrada do GARCH, seguidos de redes neurais
Este trabalho	PETR4 e outras três ações	B3 (COTAHIST)	diário	ARIMA e ARIMA-GARCH, comparados às médias móveis de Almeida (2016)

Fonte: Elaboração própria.

CAPÍTULO II

2 Percurso metodológico

Este capítulo descreve como a pesquisa foi conduzida. Apresenta a caracterização do estudo, a amostra, a coleta e a análise dos dados, a implementação dos modelos, o plano experimental e os obstáculos encontrados.

2.1 Caracterização da pesquisa

Quanto à natureza, a pesquisa é primária, por produzir conhecimento novo a partir da realização de experimentos, e aplicada, por voltar esse conhecimento a um problema concreto de decisão. Ela se apoia no método empírico, que fundamenta as conclusões em observações (Wazlawick, 2021).

2.1.1 Quanto aos objetivos

Quanto aos objetivos, a pesquisa é explicativa, com uma parte descritiva. Segundo Wazlawick (2021), a pesquisa explicativa busca identificar as causas de determinados fenômenos. Neste trabalho, ela investiga por que a inclusão de um modelo de volatilidade altera o desempenho de uma estratégia cuja previsão de direção permanece a mesma. A parte descritiva está no registro de como cada robô se comportou em diferentes períodos do mercado.

2.1.2 Quanto ao delineamento

Quanto ao delineamento, trata-se de uma pesquisa quase experimental. A pesquisa experimental envolve a manipulação controlada de variáveis e a observação dos efeitos resultantes. Quando os objetos estudados não podem ser selecionados de forma aleatória, trata-se de um quase experimento (Wazlawick, 2021). É o caso deste trabalho, em que a série histórica de um ativo é única e não pode ser sorteada.

O problema e as hipóteses foram formulados na introdução. As variáveis são os parâmetros configuráveis do robô, e o plano experimental foi definido antes da observação dos resultados. Os sujeitos são as séries históricas dos ativos, e o ambiente é a plataforma de simulação.

2.2 Caracterização do lugar e da amostra de pesquisa

O lugar da pesquisa é computacional. Os experimentos foram conduzidos no testador de estratégias do MetaTrader 5, que simula a execução de ordens sobre séries históricas com as mesmas regras de negociação que usaria em tempo real. A plataforma foi escolhida, em primeiro lugar, por ser a mesma utilizada no estudo que este trabalho continua (Almeida, 2016), o que permitiu reproduzir a estratégia de referência no mesmo ambiente. Pesou também o fato de ela

reunir em um só lugar a programação dos robôs, o histórico de preços e o teste sobre dados passados, e de contar com uma grande comunidade de usuários e desenvolvedores, motivos que levaram Batista (2024) à mesma escolha. Ela é nativa do sistema Windows, e os experimentos apresentados aqui foram executados em ambiente Linux, por meio da camada de compatibilidade Wine, pelos motivos expostos ao final deste capítulo.

A amostra é composta por séries históricas diárias de ações negociadas na B3. O ativo principal é a PETR4, ação preferencial da Petrobras, escolhida por dois motivos. É o ativo estudado por Almeida (2016), o que permite a comparação direta, e possui série longa e contínua no arquivo histórico da bolsa, com 5.881 pregões, de 2 de janeiro de 2003 a 21 de setembro de 2026.

Para verificar se os resultados se repetem fora do ativo principal, foram incluídas outras três ações, VALE3, da mineração, USIM5, da siderurgia, e POMO4, de bens de capital. O critério de escolha, definido antes da observação de qualquer resultado, considerou a qualidade dos dados e a diversidade de setores. Cada ação precisava ter série completa de 2003 a 2026 no arquivo da bolsa, sem mudanças na forma de cotação que impedissem a reconstrução dos preços, e preço sempre acima de R$ 3, para que o arredondamento em centavos não distorcesse os retornos. As ações também deveriam pertencer a setores diferentes entre si.

O recorte temporal foi dividido em períodos, em vez de tratado como um período único, porque o comportamento de uma estratégia depende do momento do mercado. Foram examinados um período de crise, de 2007 a 2011, um de queda prolongada seguida de recuperação, de 2013 a 2016, o intervalo de 2017 a 2024, que inclui a crise da pandemia, e o período mais recente, de 2025 a 2026.

2.3 Procedimentos para coleta e análise de dados

2.3.1 Técnicas para coleta de dados

Os dados foram obtidos de fontes públicas oficiais. O servidor da corretora não foi usado, porque disponibiliza cotações apenas a partir de setembro de 2021. As cotações vieram do arquivo COTAHIST, série histórica publicada pela própria B3, que registra, para cada pregão e cada ação, os preços de abertura, máximo, mínimo e fechamento e o volume negociado (B3, 2026b). A taxa Selic diária foi obtida da série 11 do Sistema Gerenciador de Séries Temporais do Banco Central (Banco Central do Brasil, 2026).

Sobre as cotações foi aplicada uma única transformação, a correção por desdobramentos. No desdobramento, cada ação é dividida em várias, e o preço cai na mesma proporção, sem que o investidor perca nada, já que passa a ter mais ações. Na PETR4 houve dois desdobramentos no período. Em 1º de setembro de 2005, cada ação virou quatro, e em 28 de abril de 2008, cada ação virou duas. No arquivo da bolsa, esses dias aparecem como quedas de 75,2% e de 49,5%, que não aconteceram de fato e que o modelo interpretaria como dois tombos do mercado. Para corrigir, os preços anteriores a 28 de abril de 2008 foram divididos por dois, e os anteriores a

1º de setembro de 2005, por oito, isto é, por quatro e depois por dois. Com isso, a série passa a registrar apenas as variações reais de preço. Os dois eventos foram confirmados nos eventos corporativos que a B3 divulga para a Petrobras (B3, 2026a) e, o de 2005, que esse registro não alcança, no comunicado em que a própria companhia anunciou o desdobramento (Petróleo Brasileiro S.A., 2005).

Os proventos, isto é, os dividendos e os juros sobre capital próprio pagos aos acionistas, não foram incorporados à série, para que os modelos trabalhassem apenas com as variações de preço negociadas em bolsa. Eles foram obtidos à parte, para a análise complementar do Capítulo III, no histórico de proventos em dinheiro que a B3 divulga para a Petrobras (B3, 2026a). Para cada provento, esse histórico informa a última data com direito a recebê-lo e o seu valor como percentual do fechamento daquele dia. O valor por ação foi obtido aplicando esse percentual ao fechamento da série corrigida, o que o deixa na mesma escala dos preços corrigidos pelos desdobramentos.

As séries preparadas foram importadas para a plataforma como símbolos personalizados, recurso que permite ao testador operar sobre dados próprios.

2.3.2 Natureza da análise de dados

A análise é quantitativa. Os robôs foram comparados pelas métricas apresentadas no Capítulo I. Como o GARCH decide se o robô opera e com quanto, enquanto a direção da operação continua a cargo do ARIMA, as métricas principais são as de risco, a queda máxima e a volatilidade. O retorno acumulado acompanha essas métricas como resultado complementar. O acerto direcional, que a primeira hipótese previa melhorar, foi medido pela taxa de acerto das negociações, e a quantidade de sinais vetados ajuda a explicar o funcionamento do filtro.

A consistência das diferenças entre os robôs foi avaliada pela repetição delas entre os períodos, entre as configurações e entre os ativos. Não foi aplicado teste estatístico formal.

2.4 Implementação dos modelos

O MetaTrader 5 oferece, na biblioteca padrão do MQL5, indicadores de análise técnica como médias móveis e osciladores. Os modelos ARIMA e GARCH foram implementados para este trabalho. A implementação se apoiou na documentação oficial da linguagem (MetaQuotes Ltd., 2026b) e em artigos publicados pela comunidade de desenvolvedores da plataforma, que serviram de aprendizado para a implementação: a estimação de modelos ARIMA em MQL5 (Dube, 2023), o teste ADF (Dube, 2024) e o GARCH estimado por máxima verossimilhança (Chernish, 2024). Uma ferramenta de inteligência artificial também foi usada, de forma pontual, como apoio para juntar os trechos de código e para fazer verificações. Para os cálculos numéricos mais pesados, a resolução de sistemas lineares e a otimização com restrições, foram usadas rotinas da ALGLIB, biblioteca numérica que acompanha a instalação da plataforma.

O robô de médias móveis de Almeida (2016) seguiu o mesmo caminho. O algoritmo original, publicado no Anexo 1 daquele trabalho (Almeida, 2016, p. 69-77), depende de seis bibliotecas externas (Almeida, 2016, p. 35). Por isso, ele foi reescrito com o indicador de média móvel da própria plataforma e compartilha com o ARIMA-GARCH a rotina de envio de ordens e de medição dos resultados, para que os dois sejam avaliados nas mesmas condições. Foram mantidas as médias simples de 10 e 20 períodos sobre o preço de fechamento, a compra quando a média rápida fica acima da lenta, a venda no caso inverso, com a inversão da posição a cada sinal, e a decisão tomada uma vez por pregão, com as médias do último pregão encerrado.

Há três diferenças em relação ao original. A entrada é feita no pregão em que a média rápida cruza a lenta, e não sempre que ela está acima, diferença tratada na reprodução do estudo de referência. O volume calculado por porcentagem de risco não foi reproduzido, porque a rotina que o calcula não consta do anexo, e ficou o volume fixo, que o original também oferece. Por fim, foram acrescentadas duas opções usadas nas comparações com o ARIMA-GARCH, a aplicação de uma fração do patrimônio em cada operação e a operação somente na compra. O código desse robô está no Apêndice C.

As subseções a seguir descrevem a implementação do ARIMA-GARCH parte por parte, da preparação dos preços aos parâmetros do robô.

2.4.1 Preparação dos preços

Os modelos foram aplicados ao logaritmo do preço, e não ao preço em reais. A diferença entre os logaritmos de dois preços seguidos é o retorno logarítmico, que mede a variação percentual do dia (Thomaz, 2019). Trabalhar com retornos tem duas vantagens. A primeira é que eles são livres de escala, ou seja, uma alta de 1% significa o mesmo para uma ação cotada a R$ 5 ou a R$ 40, de modo que os parâmetros do robô valem para qualquer ativo e período. A segunda é que os retornos costumam ser estacionários, ao contrário dos preços, condição exigida pelas partes autorregressiva e de médias móveis do ARIMA (Thomaz, 2019).

Como consequência, a volatilidade estimada pelo GARCH também sai em porcentagem. Uma volatilidade de 0,02 indica uma oscilação típica de 2% ao dia, enquanto, com o preço em reais, o mesmo número significaria dois centavos e mudaria de sentido conforme o preço da ação.

2.4.2 Estimação do ARIMA

O ARIMA foi estimado pelo método de Hannan e Rissanen (1982), que divide a estimação em duas regressões de mínimos quadrados ordinários, o método que escolhe os coeficientes que tornam mínima a soma dos quadrados dos erros de ajuste. A dificuldade de um modelo com termos de médias móveis é que os erros passados não podem ser observados, pois dependem dos próprios coeficientes que se quer estimar. A primeira regressão ajusta um modelo autorregressivo longo, cujos resíduos servem de estimativa desses erros, e a segunda usa essas estimativas para

obter os coeficientes.

A escolha do método foi uma decisão de projeto. Ele exige apenas regressões lineares, sem processo iterativo, o que convém a um modelo reestimado todos os dias e com muitas combinações de ordens a comparar. O método também é usado na identificação automática de modelos ARIMA, em que muitas combinações de ordens precisam ser estimadas, por exigir apenas regressões lineares e fornecer estimativas próximas às de máxima verossimilhança (Gómez; Maravall, 1998).

A inclusão da constante c do modelo foi tratada como parâmetro da configuração. Com $d = 1$, ela representa a tendência média dos retornos da janela. No caso mais simples, sem termos autorregressivos nem de médias móveis, a previsão é a própria média dos retornos da janela, e o robô passa a seguir a direção da tendência recente. Sem a constante, a previsão depende apenas dos retornos e dos erros passados ponderados pelos coeficientes, e o modelo sem nenhum termo foi descartado da busca, porque a sua previsão seria sempre zero. Com $d = 0$, a constante foi sempre incluída, porque o logaritmo do preço não oscila em torno de zero.

Os coeficientes saem das regressões, mas a comparação entre as ordens, feita pelo AIC, exige a verossimilhança de cada modelo. Ela foi calculada na forma condicional, supondo que os erros sigam a distribuição normal, caso em que depende da soma dos quadrados dos resíduos do modelo (Thomaz, 2019, p. 42). Quanto menores os erros que o modelo comete sobre a janela, maior a sua verossimilhança. Todas as combinações candidatas foram avaliadas sobre os mesmos pregões da janela, para que nenhuma levasse vantagem por somar mais ou menos dias.

As ordens do modelo foram escolhidas pelo próprio robô. A ordem de integração d foi determinada pela aplicação sequencial do teste ADF (Thomaz, 2019), ao nível de significância de 5%. Na regressão do teste foi incluída uma constante, e o número de defasagens da variação foi escolhido pelo AIC. Testa-se a série e, enquanto a hipótese de raiz unitária não for rejeitada, testa-se a série diferenciada mais uma vez. Com $d = 1$, o ARIMA modela os retornos. Com $d = 0$, escolhido quando o próprio logaritmo do preço se mostra estacionário dentro da janela, ele modela o logaritmo do preço diretamente. As ordens p e q foram escolhidas por busca em grade, de zero a quatro, pelo menor AIC (Thomaz, 2019), procedimento semelhante ao de Xing *et al.* (2024). As ordens são reavaliadas a cada 21 pregões, cerca de um mês, de modo que podem mudar ao longo de um teste.

2.4.3 Estimação do GARCH

O GARCH foi estimado sobre os resíduos do ARIMA, isto é, sobre a parte do retorno que o ARIMA não conseguiu prever. Foi adotada a especificação GARCH(1,1), que exige poucos parâmetros e é a escolha predominante nos estudos de previsão de ações (Xing *et al.*, 2024). Estimar o modelo significa encontrar os valores de ω, α e β que melhor descrevem como a agitação desses resíduos variou ao longo da janela.

Para isso foi usada a máxima verossimilhança, que escolhe os valores dos parâmetros que tornam os dados observados os mais prováveis segundo o modelo (Thomaz, 2019, p. 41).

Na prática, cada combinação candidata é posta à prova sobre a janela. Com ela, a equação do GARCH apresentada no Capítulo I indica, para cada dia, a variância que o modelo esperava, e essa expectativa é comparada com o erro que de fato ocorreu. Supondo que os erros sigam a distribuição normal, um erro grande em um dia para o qual o modelo esperava calma é pouco provável e reduz a verossimilhança, enquanto o mesmo erro em um dia para o qual o modelo já esperava agitação é plausível (Thomaz, 2019, p. 43). A melhor combinação é a que antecipa melhor, ao longo de toda a janela, quais dias seriam agitados e quais seriam calmos.

Ao contrário do que ocorre no ARIMA, essa busca não se resolve com uma regressão. Foi usado um otimizador numérico, que experimenta valores dos parâmetros, verifica se a verossimilhança aumentou e repete o processo até não conseguir mais melhorá-la. As restrições apresentadas no Capítulo I foram impostas dentro do próprio otimizador, de modo que ele só experimenta valores válidos.

Um cuidado foi necessário com ω. Ele é a parte fixa da variância de cada dia, que não depende dos choques nem da variância anterior, e por isso fica na escala da variância diária dos retornos, um número muito pequeno, enquanto α e β ficam entre zero e um. Para que o otimizador não precisasse lidar com números de tamanhos tão diferentes, ω foi expresso como uma fração da variância dos resíduos da janela e convertido de volta ao final. Concluída a estimação, o valor usado na decisão é a volatilidade prevista para o pregão seguinte.

2.4.4 Janela de estimação

Os dois modelos são reestimados a cada pregão sobre uma janela deslizante, isto é, sobre os últimos 120 ou 240 pregões já encerrados, conforme a configuração. A cada dia, o pregão mais antigo sai da janela e o mais recente entra, o que permite ao modelo acompanhar as mudanças de comportamento do mercado. A janela nunca inclui o pregão em andamento, para que a decisão do dia não use uma informação que só estaria disponível no seu encerramento. O GARCH não tem janela própria e é estimado sobre os resíduos do ARIMA nessa mesma janela.

Xing *et al.* (2024) compararam esse tipo de janela com a janela expansiva, que acumula todo o histórico desde o início, e no seu estudo o ARIMA teve melhor desempenho com a janela expansiva. A janela expansiva não foi testada neste trabalho. O tamanho máximo da janela, de 240 pregões, decorreu da forma como os testes foram montados. Cada teste foi iniciado no testador na data inicial do período, e observou-se que, nessa condição, o histórico disponível antes do início começa em 1º de janeiro do ano anterior e soma cerca de 250 pregões. Janelas maiores exigiriam iniciar a simulação antes do período e só começar a operar na data inicial, alternativa que não foi adotada neste trabalho.

2.4.5 Funcionamento do robô

Os dois modelos foram reunidos em um único *Expert Advisor*, com um parâmetro que liga e desliga o GARCH. Com ele desligado, o programa percorre o mesmo caminho de previsão,

a mesma janela de estimação e a mesma rotina de envio de ordens, de modo que qualquer diferença entre as duas execuções só pode vir do GARCH.

A divisão de papéis é estrita. O ARIMA é o único responsável por indicar o lado da operação, e o GARCH nunca gera sinal de direção. A cada novo pregão, o robô carrega a janela, ajusta o ARIMA, obtém a previsão do retorno do dia, ajusta o GARCH sobre os resíduos e obtém a volatilidade prevista. Se o GARCH não vetar a operação, o sinal da previsão do ARIMA determina a direção, e a ordem é executada na abertura do pregão.

2.4.6 Uso do GARCH na decisão

As duas maneiras de usar a volatilidade prevista apresentadas no Capítulo I, o filtro e o ajuste da posição, foram implementadas no robô, cada uma sozinha e as duas combinadas.

No filtro, o limiar pode ser definido de duas formas. Na primeira, ele é um valor fixo de volatilidade diária, como 2% ao dia. Na segunda, ele é um percentil da volatilidade que o próprio GARCH estimou dentro da janela. Com o percentil 70, por exemplo, a operação é vetada quando a volatilidade prevista para o dia supera 70% dos valores estimados na janela. O limiar por percentil se adapta ao ativo e ao período, porque o que é um dia agitado muda com o tempo.

No ajuste da posição, o tamanho da posição é multiplicado pela razão entre o alvo e a volatilidade prevista, dentro de limites de um quarto e do dobro do tamanho normal. Com um alvo de 1% ao dia e uma volatilidade prevista de 2%, o robô compra metade do tamanho normal. Como o robô não opera alavancado, a posição nunca passa de 98% do patrimônio. Partindo de um tamanho normal de 95%, o ajuste, na prática, reduz a posição, mas quase não a aumenta.

O ajuste da posição sozinho foi testado, mas não foi escolhido em nenhum período nem em nenhuma ação. Todos os robôs escolhidos usam o filtro, sozinho ou combinado com o ajuste da posição.

Nas formas com filtro, resta decidir o que fazer com a posição que já está aberta. Por padrão, o veto só impede novas compras, e o robô continua com o que já tinha comprado. Uma opção do robô permite que o veto também encerre a posição, retirando o capital do mercado enquanto a volatilidade prevista estiver alta. O ajuste do tamanho, por sua vez, vale apenas para as novas compras.

2.4.7 Parâmetros do robô

O comportamento do robô é definido por parâmetros de entrada, que o MetaTrader 5 permite alterar sem modificar o código. Eles se dividem em três grupos.

Os parâmetros do ARIMA são o tamanho da janela de estimação, a inclusão ou não de uma constante no modelo e uma previsão mínima, abaixo da qual o robô não opera mesmo que a previsão indique alta. Os parâmetros do GARCH são a chave que liga e desliga o modelo, a forma de uso, o tipo e o valor do limiar, o alvo de volatilidade e a opção de encerrar a posição no veto. Os parâmetros de execução são a fração do patrimônio aplicada em cada operação e a

restrição de operar apenas na ponta compradora.

Alguns parâmetros ficaram fixos em todos os testes, como a ordem máxima de zero a quatro do ARIMA e a reavaliação das ordens a cada 21 pregões. Os demais foram variados na otimização, com os valores apresentados no plano experimental.

2.5 Plano experimental

O plano experimental foi definido antes da observação dos resultados. Esta seção descreve, em partes, as condições das simulações, a forma de medir o resultado, a reprodução do estudo de referência, a validação dos estimadores, as configurações testadas, a forma de escolher entre elas, os robôs comparados e o teste complementar.

2.5.1 Condições das simulações

Todas as simulações usaram os preços de abertura de cada pregão, já que o robô decide uma vez por dia e executa na abertura. No cenário principal, partiram de um capital de R$ 100 mil, aplicaram a cada operação 95% do patrimônio, valor que o ajuste da posição, quando usado, pode reduzir, e operaram apenas na ponta compradora. A venda a descoberto, em que o robô vende uma ação que não tem para recomprá-la mais barato, ficou fora do cenário principal. Nela, o robô passa a apostar também na queda, e o resultado depende ainda mais da direção prevista pelo ARIMA, enquanto o foco deste trabalho é a proteção dada pelo GARCH. O efeito dela foi medido à parte, no teste complementar. A reprodução do estudo de referência usou as condições do autor, descritas mais adiante.

2.5.2 Medição dos resultados

O resultado de cada simulação foi medido pelo patrimônio da conta simulada pelo testador, sem nenhum ajuste, chamado aqui de resultado bruto. A comparação entre os modelos é feita sobre ele, porque depende apenas das decisões de cada robô. A compra e manutenção da ação foi medida pela variação do preço.

Na escolha das configurações, descrita adiante, o desempenho foi medido de forma mais próxima da operação real, com dois ajustes. O primeiro desconta, a cada ordem, 0,03% do valor negociado, que é a soma das tarifas de negociação, de contraparte central e de transferência que a B3 cobra de cada investidor, na compra e na venda, em operações com ações à vista que não são *day trade*, na faixa de quem negocia em média até R$ 3 milhões por dia (B3, 2026c). A corretagem foi considerada zero, e a tarifa, que é a vigente em 2026, foi aplicada a todos os períodos. O segundo acrescenta a remuneração pela taxa Selic diária do capital que fica fora do mercado, para que configurações que passam tempos diferentes fora dele sejam comparadas de forma justa. O desconto dos custos também foi aplicado na taxa de acerto, em que uma negociação só conta como lucrativa se o lucro superar o custo.

Como análise complementar, o resultado foi medido também com os dois ajustes e com os proventos. Cada provento foi creditado no primeiro pregão em que a ação passa a ser negociada sem direito a ele, na proporção do patrimônio que o robô mantinha aplicado na ação naquele dia. Na compra e manutenção, o provento foi somado ao preço do mesmo dia. O resultado é uma estimativa do que o investidor teria recebido, porque não considera impostos nem o prazo entre essa data e o pagamento.

Além das medidas de desempenho, o robô pode registrar em arquivo, a cada pregão, as ordens do ARIMA em uso, os parâmetros estimados, a volatilidade prevista, o limiar e a decisão tomada. Esse registro não interfere nas decisões e foi usado para acompanhar o funcionamento dos modelos, como no episódio de 2020 descrito no Capítulo III.

2.5.3 Reprodução do estudo de referência

Antes das comparações, a estratégia de médias móveis reproduzida foi submetida aos testes de Almeida (2016), para garantir que a comparação fosse feita com a estratégia dele. Os testes foram repetidos com as mesmas datas, o mesmo capital de R$ 50 mil, as mesmas médias de 10 e 20 períodos, sem custos e com operações de compra e venda. O autor declara mil ações para o teste de 2013 a 2016, mas não informa o volume dos testes de crise, e, como ele menciona cem ações como volume mínimo, esse foi o volume adotado nesses testes.

A reprodução foi feita de duas formas. A primeira usou a série de preços deste trabalho, a mesma dos demais experimentos. A segunda testou uma hipótese sobre o ambiente do autor, que usou o histórico da corretora sem descrever como ele era. A hipótese é que o histórico da PETR4 no servidor da corretora começasse em 2008. O testador precisa de pelo menos cem pregões de histórico antes da data inicial e, quando não os encontra, adia o início do teste por conta própria (MetaQuotes Ltd., 2026a). Assim, os testes de crise do autor teriam começado, na prática, em junho de 2008. Para testar essa hipótese, a série deste trabalho foi cortada para começar em 2 de janeiro de 2008, sem nenhuma outra alteração nos preços. Trata-se de uma hipótese, e não de um fato, porque o servidor usado pelo autor não existe mais e não há como conferir o histórico que ele baixou. Nessa segunda forma, a entrada nas operações também seguiu exatamente a regra do anexo do autor, que compra sempre que a média rápida está acima da lenta, em vez de esperar o cruzamento. Essa diferença altera apenas a primeira operação de cada teste.

2.5.4 Validação dos estimadores

Como o ARIMA e o GARCH foram programados neste trabalho, os dois métodos de estimação descritos na implementação dos modelos foram verificados antes do uso com dados reais, por um teste de recuperação de parâmetros. Gera-se uma série artificial a partir de coeficientes escolhidos, o estimador recebe apenas a série, e os valores que ele encontra são comparados com os usados na geração. Como a resposta certa é conhecida, o teste mostra se o método chega a ela.

Foram usados três casos. O primeiro é um AR(2), que tem apenas a parte autorregressiva, caso em que o método de Hannan e Rissanen (1982) se reduz a uma única regressão. O segundo é um ARMA(1,1), que inclui a parte de médias móveis e passa pelas duas regressões do método. Como o ARIMA aplica um ARMA à série já diferenciada, esses dois casos cobrem o estimador do ARIMA. O terceiro é um GARCH(1,1), que verifica a máxima verossimilhança com o otimizador numérico. As séries do AR(2) e do ARMA(1,1) têm 4.000 observações, e a do GARCH, 6.000. Elas não correspondem a nenhum período do mercado e foram geradas com uma semente fixa de números aleatórios, para que o teste possa ser repetido com o mesmo resultado.

2.5.5 Configurações testadas

As configurações testadas foram geradas a partir dos parâmetros descritos na Seção 2.4.7, pelo otimizador do testador de estratégias do MetaTrader 5, no modo de busca completa, em que a própria plataforma executa uma simulação para cada combinação dos valores informados.

Três parâmetros variaram em todas as formas de uso do GARCH. O primeiro foi a inclusão ou não de uma constante no ARIMA. O segundo, a janela de estimação, de 120 ou de 240 pregões. O terceiro, uma previsão mínima, de zero ou de 0,1%, abaixo da qual o robô não opera. Essas três escolhas formam oito combinações, e cada forma de uso do GARCH acrescentou a elas os seus próprios parâmetros e, quando havia veto, a opção de encerrar ou não a posição, como mostra o Quadro 3. Ao todo, foram 304 configurações com o GARCH e oito sem ele, 312 no total.

Quadro 3 – Configurações testadas na otimização

Forma de uso do GARCH	Parâmetros próprios e valores testados	Configurações
Sem GARCH (ARIMA)	nenhum	8
Filtro, limiar fixo	limiar de 2%, 3% ou 4% ao dia, encerrar ou não a posição no veto	$8 \times 3 \times 2 = 48$
Filtro, limiar por percentil	percentil 50, 70 ou 90, encerrar ou não a posição no veto	$8 \times 3 \times 2 = 48$
Filtro e ajuste da posição, limiar fixo	limiar de 2%, 3% ou 4% ao dia, alvo de 1% ou 2% ao dia, encerrar ou não a posição no veto	$8 \times 3 \times 2 \times 2 = 96$
Filtro e ajuste da posição, limiar por percentil	percentil 50, 70 ou 90, alvo de 1% ou 2% ao dia, encerrar ou não a posição no veto	$8 \times 3 \times 2 \times 2 = 96$
Ajuste da posição	alvo de 1% ou 2% ao dia	$8 \times 2 = 16$
Total		312

Fonte: Elaboração própria.

Nota: O número 8 corresponde às combinações comuns a todas as formas: constante no ARIMA (sim ou não), janela de estimação (120 ou 240 pregões) e previsão mínima para operar (zero ou 0,1%).

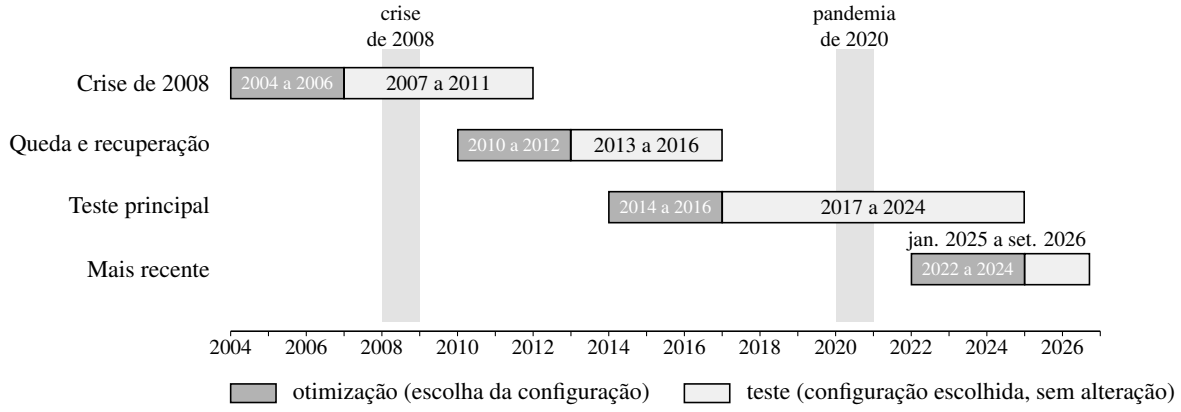
Cada forma de uso foi otimizada separadamente, para que parâmetros que não se aplicam a ela não gerassem simulações repetidas. Em cada simulação, o próprio robô gravou em arquivo as medidas de desempenho, e o critério de escolha foi aplicado sobre esses arquivos.

2.5.6 Escolha da configuração

Escolher a melhor das 312 configurações no mesmo período em que o desempenho é medido daria uma vantagem falsa, porque, olhando para trás, sempre existe alguma combinação que teria dado certo. Por isso, a escolha seguiu o princípio do teste avançado, ou *forward*, do testador de estratégias, em três passos. No primeiro, as 312 configurações foram simuladas nos três anos anteriores ao período de teste. No segundo, a lista de resultados foi ordenada por um critério definido antes, e a primeira configuração da lista foi a escolhida. No terceiro, só a configuração escolhida foi executada no período de teste, que não tinha sido usado na escolha, sem nenhuma alteração. Durante o teste, os coeficientes do ARIMA e do GARCH continuam a ser reestimados a cada pregão, como descrito na seção anterior. O que fica fixo é a configuração.

Para cada período de teste, os três passos foram refeitos do zero, com os três anos anteriores a ele. Por isso, a configuração escolhida, inclusive a forma de uso do GARCH, pode mudar de um período para outro. A Figura 6 mostra os períodos usados.

Figura 6 – Períodos de otimização e de teste



Fonte: Elaboração própria.

Foram usados dois critérios para ordenar a lista, porque não existe uma única forma de definir a melhor configuração. O ARIMA-GARCH Sharpe é a primeira configuração da lista ordenada pelo índice de Sharpe, ou seja, a de maior ganho por unidade de oscilação na otimização. O ARIMA-GARCH Calmar é a primeira da lista ordenada pelo índice de Calmar, a de maior ganho por unidade de queda máxima. O nome indica apenas o critério usado para ordenar a lista, sem relação com o resultado obtido no teste. Na otimização, os dois índices foram calculados com os dois ajustes descritos na medição dos resultados, e o de Sharpe, sobre o ganho acima da Selic.

2.5.7 Robôs comparados

Em cada período, as versões com GARCH foram comparadas a três referências. A primeira é o melhor robô sem o GARCH, escolhido entre as oito configurações sem o modelo pelo mesmo critério do ARIMA-GARCH Sharpe, e que aparece nas tabelas simplesmente como

ARIMA. A segunda é a estratégia de médias móveis de Almeida (2016), reproduzida neste trabalho e executada nos mesmos períodos e nas mesmas condições, inclusive apenas na ponta compradora, ao contrário do teste original, que também vendia a descoberto. A terceira é a compra e manutenção da própria PETR4.

Essas comparações colocam lado a lado robôs com configurações diferentes. Para separar o efeito do GARCH do efeito dos demais parâmetros, cada versão com o GARCH também foi executada com o modelo desligado e todo o resto igual. Como o robô tem um único parâmetro que liga e desliga o GARCH, essa comparação isola o efeito dele.

2.5.8 Teste complementar

Um teste complementar ampliou a verificação. O mesmo procedimento foi repetido na PETR4, na VALE3, na USIM5 e na POMO4, nos períodos de 2007 a 2011, 2013 a 2016 e 2025 a 2026, e cada configuração com GARCH foi comparada à mesma configuração sem ele. Nesse teste, a grade incluiu também a opção de vender a descoberto, o que elevou o número de configurações para 624 e o total para 14.976 simulações. Como no cenário principal, a escolha considerou apenas as configurações que operam na ponta compradora, e as demais serviram para medir o efeito da venda a descoberto.

2.6 Obstáculos encontrados

O desenvolvimento enfrentou dois obstáculos que alteraram o cronograma original e influenciaram decisões metodológicas.

O primeiro foi a perda do ambiente Windows. O desenvolvimento começou em uma máquina com Windows e MetaTrader 5 instalado nativamente, e a perda desse equipamento obrigou a migração para Linux, onde a plataforma não tem suporte oficial e precisa ser executada por camada de compatibilidade. A solução funcionou, mas trouxe instabilidades ocasionais e exigiu que a automação dos experimentos fosse refeita em linha de comando.

O segundo foi o acesso aos dados históricos pela corretora. A corretora utilizada inicialmente, a Rico, deixou de oferecer suporte ao MetaTrader 5, o que exigiu a abertura de conta em outra instituição. A segunda corretora, a Clear, oferece a plataforma, mas disponibiliza cotações apenas a partir de setembro de 2021, o que não atendia aos períodos de teste, que retrocedem a 2003. Esse obstáculo levou à adoção do arquivo oficial da bolsa como fonte, o que tornou a série pública, verificável e livre de alterações posteriores.

CAPÍTULO III

3 Análise e discussão dos dados

Este capítulo apresenta os resultados dos experimentos descritos no Capítulo II. Começa pela reprodução do estudo de Almeida (2016) e pela validação dos estimadores e apresenta, em seguida, cada período de teste separadamente, com o ano de cada crise em detalhe. Depois verifica se o efeito observado vem mesmo do GARCH, mostra de que forma ele protege o capital e como os custos, a Selic e os proventos mudam o resultado, e fecha com outras três ações.

3.1 Reprodução do estudo de referência

A Tabela 1 compara os resultados relatados por Almeida (2016) com os das duas reproduções descritas no Capítulo II. Ela serve apenas para conferir se a estratégia de médias móveis foi reproduzida corretamente. O ARIMA e o ARIMA-GARCH não aparecem nela, e a comparação entre os três robôs começa depois da validação dos estimadores.

Tabela 1 – Reprodução dos testes de Almeida (2016)

	Almeida (2016)		Série deste trabalho		Histórico desde 2008	
Período	Operações	Resultado	Operações	Resultado	Operações	Resultado
2013 a 2016	38	+R$ 10.430	37	+R$ 10.050	37	+R$ 10.050
2008 a 2010	37	−R$ 1.595	38	−R$ 575	35	−R$ 1.482
2007 a 2011	52	−R$ 2.542	63	−R$ 1.274	50	−R$ 2.530

Fonte: Elaboração própria e Almeida (2016).

Nota: Médias móveis simples de 10 e 20 períodos, capital de R$ 50 mil, sem custos, compra e venda. Volume de mil ações em 2013 a 2016 e de cem ações nos testes de crise. A última coluna usa a série deste trabalho a partir de 2008, segundo a hipótese descrita no Capítulo II.

Na série deste trabalho, o teste de 2013 a 2016 difere do original em uma operação e em 3,6% do lucro, o que indica que a lógica da estratégia foi reproduzida. Nos testes de crise as diferenças são maiores, sobretudo em 2007 a 2011, com 63 operações contra 52 do autor.

Com o histórico começando em 2008, hipótese descrita no Capítulo II, os testes de crise se aproximaram muito dos originais. O resultado ficou a R$ 113 do relatado em 2008 a 2010 e a R$ 12 em 2007 a 2011, com duas operações a menos em cada teste. Em 2013 a 2016 o início do histórico não interfere, e o resultado é o mesmo da série deste trabalho.

A proximidade, porém, vale para o resultado final, mas não para as suas partes. Nos dois testes de crise, a soma das negociações com lucro e a soma das negociações com prejuízo ficaram, cada uma, entre R$ 731 e R$ 1.003 maiores que as do autor, e as duas diferenças se compensaram. Também foram testados o ajuste da série pelos proventos e o arredondamento das médias feito pela biblioteca de código usada pelo autor, sem que o conjunto dos resultados se aproximasse mais. Não há como confirmar que essas eram as condições do autor, mas a validade da reprodução não depende dessa hipótese. O código do robô segue o algoritmo que

Almeida (2016) publicou em anexo ao seu trabalho, com as mesmas médias móveis simples de 10 e 20 pregões sobre o preço de fechamento e a mesma regra de operação, que compra quando a média curta cruza a longa para cima e vende quando a cruza para baixo. Com a regra igual, as diferenças que restam vêm principalmente do histórico de preços que o autor usou, e não da lógica do robô. É o que indica o teste de 2013 a 2016, em que o início do histórico não interfere e a reprodução ficou a uma operação do original. Por isso, considera-se que a estratégia reproduzida pode ser usada como termo de comparação. Nas seções seguintes, ela é executada na série deste trabalho e nas mesmas condições dos demais robôs.

3.2 Validação dos estimadores

A Tabela 2 mostra os coeficientes estimados nas séries artificiais descritas no Capítulo II, ao lado dos valores usados para gerá-las.

Tabela 2 – Recuperação de parâmetros em séries artificiais

Parâmetro	Valor verdadeiro	Valor estimado
AR(2), ϕ_1	0,500	0,4815
AR(2), ϕ_2	$-0{,}300$	$-0{,}2891$
ARMA(1,1), ϕ_1	0,600	0,5799
ARMA(1,1), θ_1	0,400	0,4105
GARCH(1,1), ω	$1{,}00 \times 10^{-5}$	$1{,}17 \times 10^{-5}$
GARCH(1,1), α	0,090	0,0825
GARCH(1,1), β	0,890	0,8941

Fonte: Elaboração própria.

Os coeficientes do AR(2) e do ARMA(1,1) ficaram a menos de 4% dos valores verdadeiros. No GARCH, o parâmetro β, que responde pela persistência da volatilidade, ficou a menos de 1% do valor verdadeiro, enquanto ω e α se afastaram cerca de 17% e 8%. Ainda assim, a soma $\alpha + \beta$, que determina quanto tempo um choque de volatilidade leva para se dissipar, foi estimada em 0,9765, contra 0,98 do valor verdadeiro. Os dois métodos de estimação chegaram, portanto, perto dos valores que geraram as séries, o que indica que foram implementados corretamente.

3.3 Como os resultados são apresentados

Cada período de teste é apresentado em uma seção própria, sempre na mesma ordem. Primeiro vem uma tabela com o resultado de cada robô, depois gráficos com a evolução do patrimônio e, nos períodos mais longos, com o retorno de cada ano e, por fim, o saldo registrado pelo próprio MetaTrader 5 para o ARIMA e para o ARIMA-GARCH. Os números dessas seções são o resultado bruto, isto é, o patrimônio da conta simulada, sem custos, sem remuneração do caixa e sem proventos. Assim, a diferença entre os robôs vem apenas das decisões de cada modelo. O efeito dos custos, da Selic e dos proventos é mostrado à parte, na Seção 3.10.

Os dois períodos que contêm uma crise, de 2007 a 2011 e de 2017 a 2024, são divididos em duas partes. A primeira traz a visão geral do período completo, com o resultado acumulado ao longo de todos os anos do teste. A segunda olha de perto o ano da crise, 2008 e 2020, para mostrar como cada robô se comportou durante o acontecimento em si. Esses anos não são testes à parte. São recortes do mesmo teste, em que o robô chega à crise com a configuração escolhida antes dela e com o patrimônio e a posição que tinha no fim do ano anterior. Por isso, o retorno e a queda máxima do ano são medidos a partir do fechamento do ano anterior.

Nas tabelas, as médias móveis reproduzidas são a estratégia de Almeida (2016) executada por este trabalho, nas mesmas condições dos demais robôs. O ARIMA é o melhor robô sem o GARCH escolhido na otimização, que não é necessariamente o ARIMA-GARCH Sharpe com o GARCH desligado. A comparação de cada versão com a mesma configuração sem o GARCH, que isola o efeito do modelo, está na Seção 3.8. A PETR4 representa a compra e manutenção da ação, medida pela variação do preço.

Como a escolha foi refeita em cada período, o ARIMA-GARCH Sharpe e o ARIMA-GARCH Calmar não são sempre a mesma configuração. O Quadro 4 mostra o que cada um faz em cada período e o valor do índice que levou à sua escolha. Todos usam o filtro, e em alguns ele vem combinado com o ajuste da posição. Em quase todos, o veto também encerra a posição aberta. A exceção é o ARIMA-GARCH Calmar de 2017 a 2024, e essa diferença pode ajudar a explicar o resultado dele naquele período. Na otimização de 2010 a 2012, nenhuma configuração teve índice de Sharpe positivo, isto é, nenhuma teve ganho médio acima da Selic, e o ARIMA-GARCH Sharpe desse período foi a configuração com o índice menos negativo.

Quadro 4 – Forma de uso do GARCH em cada robô escolhido

Período de teste	ARIMA-GARCH Sharpe	ARIMA-GARCH Calmar
2007 a 2011	filtro e ajuste da posição, encerra a posição no veto (Sharpe 1,54)	a mesma configuração do Sharpe (Calmar 8,01)
2013 a 2016	filtro e ajuste da posição, encerra a posição no veto (Sharpe −0,12)	filtro e ajuste da posição, encerra a posição no veto (Calmar 1,04)
2017 a 2024	filtro, encerra a posição no veto (Sharpe 0,75)	filtro e ajuste da posição, mantém a posição no veto (Calmar 2,83)
2025 a 2026	filtro, encerra a posição no veto (Sharpe 1,15)	filtro e ajuste da posição, encerra a posição no veto (Calmar 4,23)

Fonte: Elaboração própria.

Nota: Entre parênteses, o índice que definiu a escolha, calculado no período de otimização.

3.4 Crise financeira, 2007 a 2011

3.4.1 O período completo

O primeiro período de teste inclui a crise financeira de 2008, em que o robô de Almeida (2016) acumulou prejuízo. A Tabela 3 apresenta o resultado.

Tabela 3 – Resultado de 2007 a 2011

Robô	Retorno	Queda máx.	Volatilidade	Negociações
Médias móveis reproduzidas	−31,6%	64,6%	28,7%	31
ARIMA	−12,7%	45,1%	27,5%	320
ARIMA-GARCH	−8,4%	17,5%	10,5%	263
PETR4	−14,0%	67,2%	43,2%	–

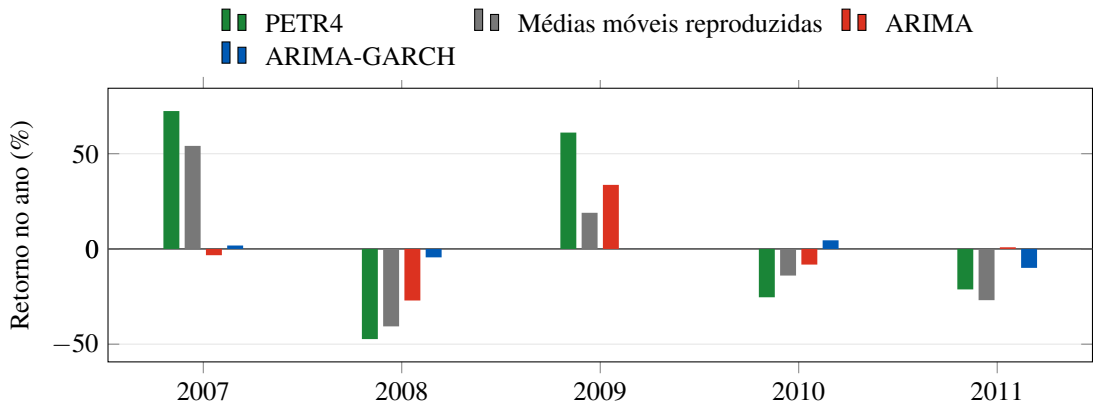
Fonte: Elaboração própria.

Nota: Resultado bruto. Neste período, os dois critérios de escolha resultaram na mesma configuração do ARIMA-GARCH.

Nenhum robô terminou o período com ganho. O ARIMA-GARCH foi o que menos perdeu e, principalmente, o que menos caiu. A sua queda máxima, de 17,5%, foi pouco mais de um quarto da das médias móveis e menos da metade da do ARIMA, e a sua volatilidade, de 10,5% ao ano, ficou abaixo de 40% da dos outros dois. O ARIMA e as médias móveis oscilaram de forma muito parecida, o que mostra que trocar o cruzamento de médias pela previsão do ARIMA, por si só, não reduziu o risco.

O Gráfico 2 mostra o resultado de cada ano. O ARIMA-GARCH foi o robô que menos perdeu em 2008, ano detalhado na subseção seguinte, e o único com ganho em 2010, quando a ação caiu 25,4%. A proteção teve um custo nos anos de alta. Em 2007, a ação subiu 72,4% e as médias móveis ganharam 54,1%, enquanto o ARIMA-GARCH ganhou 1,8%. Em 2009, a ação subiu 61,1% e o ARIMA-GARCH terminou o ano praticamente no zero, enquanto o ARIMA ganhou 33,6%.

Gráfico 2 – Retorno por ano, 2007 a 2011

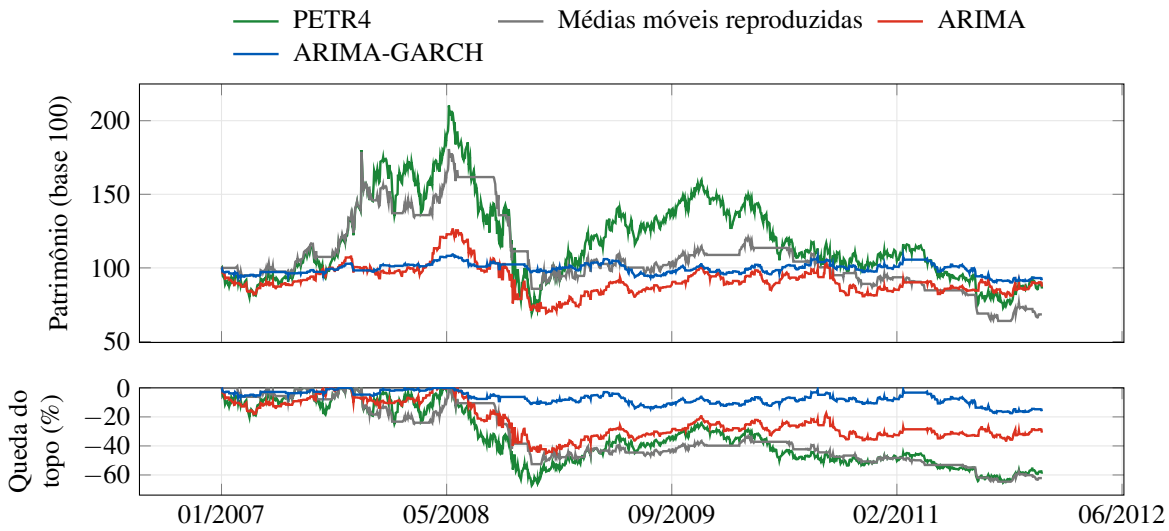


Fonte: Elaboração própria.

Nota: Resultado bruto.

O Gráfico 3 mostra a evolução do patrimônio e a queda acumulada de cada robô a partir do seu ponto mais alto, ao longo dos cinco anos. A linha do ARIMA-GARCH oscila pouco em todo o período, enquanto as demais acompanham os movimentos da ação.

Gráfico 3 – Patrimônio e queda acumulada, 2007 a 2011

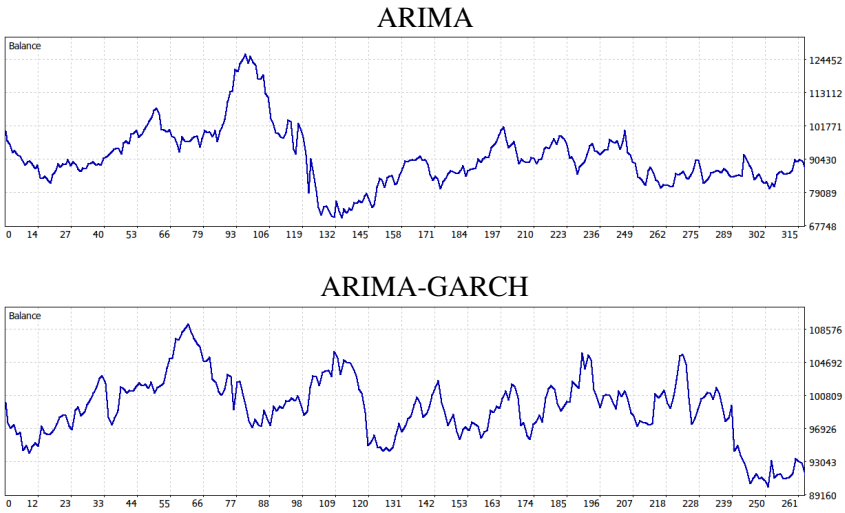


Fonte: Elaboração própria.

Nota: Resultado bruto, com o capital inicial igual a 100.

O Gráfico 4 reproduz o saldo registrado pelo testador do MetaTrader 5 ao fim de cada negociação, em uma conta que começou com R$ 100 mil. O saldo do ARIMA subiu até R$ 126 mil em maio de 2008 e desceu a R$ 71 mil em dezembro do mesmo ano, enquanto o do ARIMA-GARCH ficou sempre entre R$ 90 mil e R$ 109 mil.

Gráfico 4 – Saldo registrado pelo MetaTrader 5, 2007 a 2011



Fonte: Relatório do testador de estratégias do MetaTrader 5.

Nota: Saldo em reais após cada negociação encerrada (*Balance*, no original). O eixo horizontal é o número da negociação, e as escalas dos dois painéis são diferentes.

3.4.2 O ano de 2008

O ano de 2008 concentrou a crise financeira. Os números da Tabela 4 são um recorte do teste de 2007 a 2011, com a configuração escolhida em 2004 a 2006, antes da crise.

Tabela 4 – Resultado em 2008, ano da crise financeira

Robô	Retorno no ano	Queda máx. no ano
Médias móveis reproduzidas	−40,7%	52,5%
ARIMA	−27,1%	45,2%
ARIMA-GARCH	−4,4%	11,3%
PETR4	−47,3%	67,2%

Fonte: Elaboração própria.

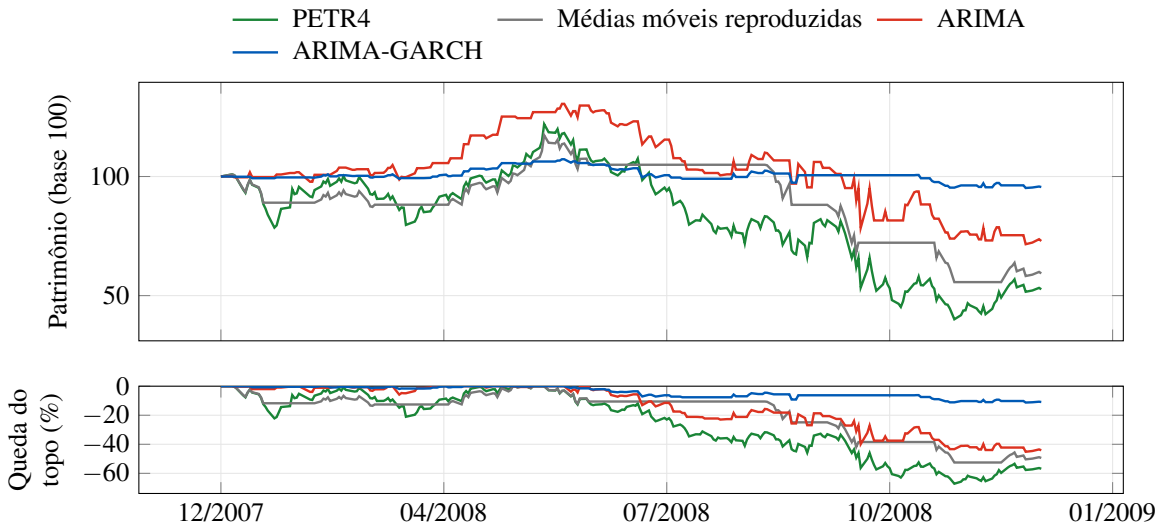
Nota: Resultado bruto. A queda máxima é medida dentro do ano, a partir do fechamento de 2007.

Enquanto a PETR4 perdia 47,3%, o ARIMA-GARCH encerrou o ano com perda de 4,4%, contra 27,1% do ARIMA e 40,7% das médias móveis. A sua maior queda dentro do ano foi de 11,3%, contra 45,2% do ARIMA, 52,5% das médias móveis e 67,2% da PETR4. A mesma configuração com o GARCH desligado perdeu 9,9% no ano e chegou a cair 35,1%, o que mostra que a proteção veio do GARCH. No período completo de 2007 a 2011, porém, essa configuração sem o GARCH terminou com ganho de 7,2%, contra a perda de 8,4% do ARIMA-GARCH, como mostra a Tabela 9. A proteção de 2008 não se converteu em ganho nos cinco anos. Foi justamente nesse tipo de mercado que Almeida (2016) apontou a limitação das médias móveis e sugeriu os modelos de volatilidade.

O Gráfico 5 mostra a evolução do patrimônio ao longo do ano. No segundo semestre, quando a ação despencou, a linha do ARIMA-GARCH ficou praticamente parada, e as dos demais robôs acompanharam a queda.

O Gráfico 6 mostra quanto do patrimônio cada robô manteve aplicado. Nesse período, o ARIMA-GARCH combina o filtro com o ajuste da posição, e o veto encerra a posição aberta. Em todos os meses de 2008, ele manteve aplicada na ação uma parte menor do patrimônio que o ARIMA, entre 0% e 22% em média, contra 23% a 62% do ARIMA. Em outubro, mês em que a ação caiu 32,8%, o ARIMA-GARCH ficou inteiramente fora do mercado, enquanto o ARIMA manteve, em média, 56,4% do patrimônio aplicado.

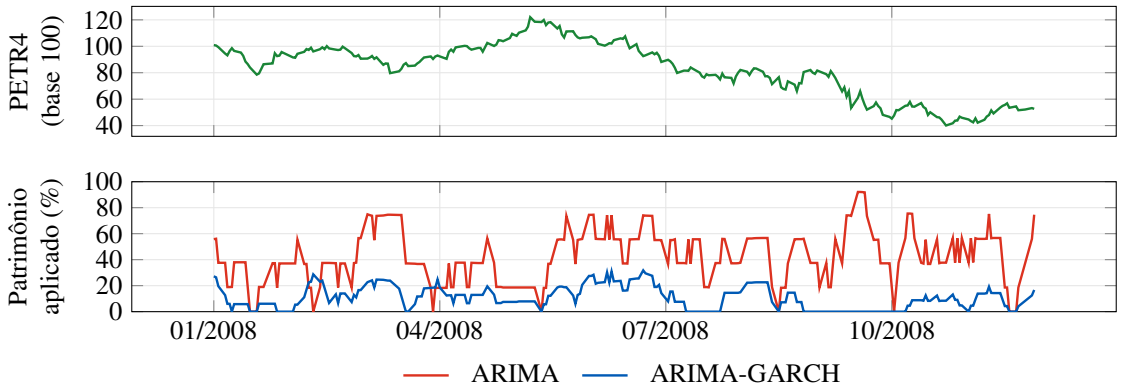
Gráfico 5 – Patrimônio e queda acumulada, 2008



Fonte: Elaboração própria.

Nota: Resultado bruto, com o patrimônio no fim de 2007 igual a 100. A queda é medida dentro do ano.

Gráfico 6 – Preço da PETR4 e patrimônio aplicado na ação, 2008



Fonte: Elaboração própria.

Nota: O patrimônio aplicado é a média dos cinco pregões anteriores, cerca de uma semana. O ARIMA é o robô sem GARCH da Tabela 3, que tem configuração própria. O preço da PETR4 tem base 100 no fim de 2007.

3.5 Queda prolongada e recuperação, 2013 a 2016

O segundo período combina a queda prolongada da PETR4 até o início de 2016 com a forte recuperação que veio depois. É também o período em que o robô de Almeida (2016) obteve lucro. A Tabela 5 apresenta o resultado.

As médias móveis tiveram o melhor resultado do período, com ganho de 100,8%. As duas versões com GARCH perderam pouco e foram as que menos caíram e menos oscilaram. O ARIMA-GARCH Calmar teve queda máxima de 15,5%, enquanto a PETR4 chegou a perder 83,1% do seu valor no pior momento. O ARIMA, sem o GARCH, teve o pior resultado, com perda de 35,8%.

O Gráfico 7 mostra de onde veio a vantagem das médias móveis. Elas perderam dinheiro

Tabela 5 – Resultado de 2013 a 2016

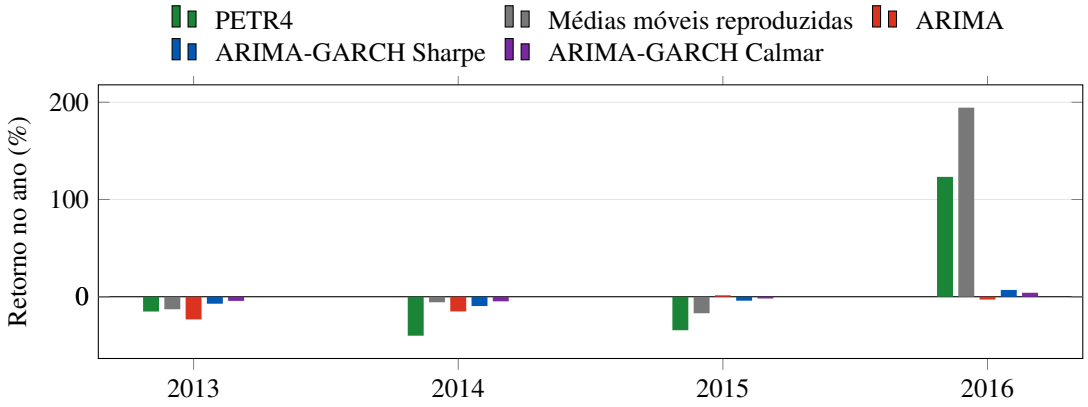
Robô	Retorno	Queda máx.	Volatilidade	Negociações
Médias móveis reproduzidas	+100,8%	42,0%	40,4%	24
ARIMA	−35,8%	54,5%	37,8%	266
ARIMA-GARCH Sharpe	−13,8%	28,6%	14,1%	140
ARIMA-GARCH Calmar	−6,8%	15,5%	7,1%	140
PETR4	−25,6%	83,1%	59,2%	–

Fonte: Elaboração própria.

Nota: Resultado bruto.

em 2013, 2014 e 2015 e ganharam 194,5% em 2016, quando a ação subiu 123,3%. Elas aproveitaram a recuperação com poucas negociações, e o ARIMA-GARCH, que ficou fora do mercado em boa parte do período, ganhou naquele ano 7,0% na configuração Sharpe e 4,1% na configuração Calmar. No trabalho de Almeida (2016), o lucro desse período também se concentrou em poucas operações, com três negociações lucrativas seguidas somando R$ 7.530 dos R$ 10.430 obtidos.

Gráfico 7 – Retorno por ano, 2013 a 2016



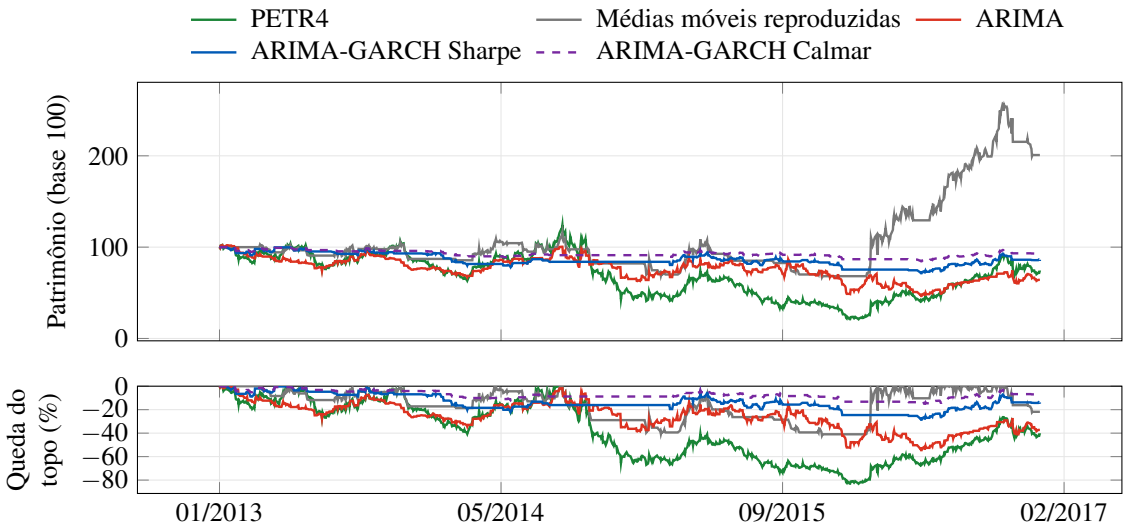
Fonte: Elaboração própria.

Nota: Resultado bruto.

O Gráfico 8 mostra que, até o fim de 2015, todos os robôs e a própria ação estavam abaixo do capital inicial, e que a separação entre as médias móveis e os demais aconteceu só em 2016.

No saldo registrado pelo MetaTrader 5, no Gráfico 9, o ARIMA desceu ao longo de quase todo o período e chegou a R$ 47 mil em junho de 2016, enquanto o menor saldo do ARIMA-GARCH Sharpe, no mesmo mês, foi de R$ 71 mil.

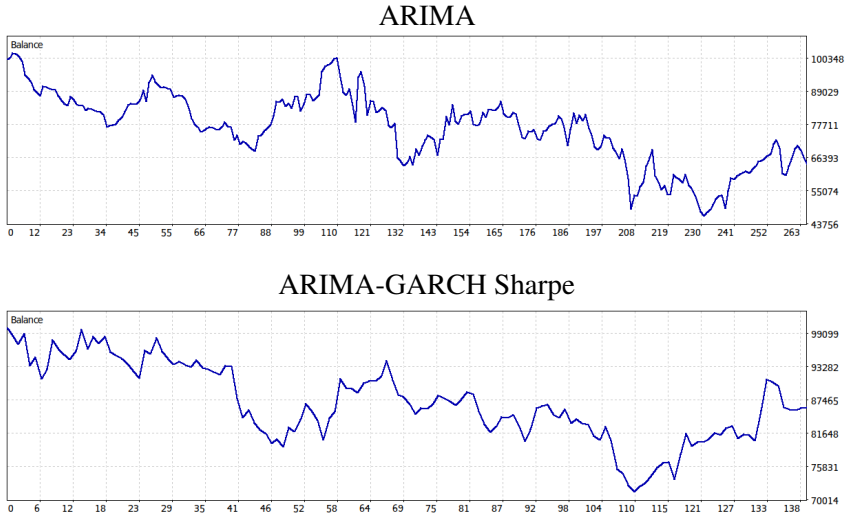
Gráfico 8 – Patrimônio e queda acumulada, 2013 a 2016



Fonte: Elaboração própria.

Nota: Resultado bruto, com o capital inicial igual a 100.

Gráfico 9 – Saldo registrado pelo MetaTrader 5, 2013 a 2016



Fonte: Relatório do testador de estratégias do MetaTrader 5.

Nota: Saldo em reais após cada negociação encerrada (*Balance*, no original). O eixo horizontal é o número da negociação, e as escalas dos dois painéis são diferentes.

3.6 Teste principal, 2017 a 2024

3.6.1 O período completo

O período de 2017 a 2024 é o teste principal, por ser o mais longo com escolha própria, e inclui a queda provocada pela pandemia em 2020. A Tabela 6 apresenta o resultado.

O ARIMA-GARCH Sharpe teve o maior retorno do período, de 199,5%, e a menor queda máxima, de 27,2%. Foi o único robô a render mais que a própria ação, que subiu 143,6% e chegou a perder 64,2% no pior momento. O ARIMA-GARCH Calmar foi o que menos oscilou,

Tabela 6 – Resultado de 2017 a 2024

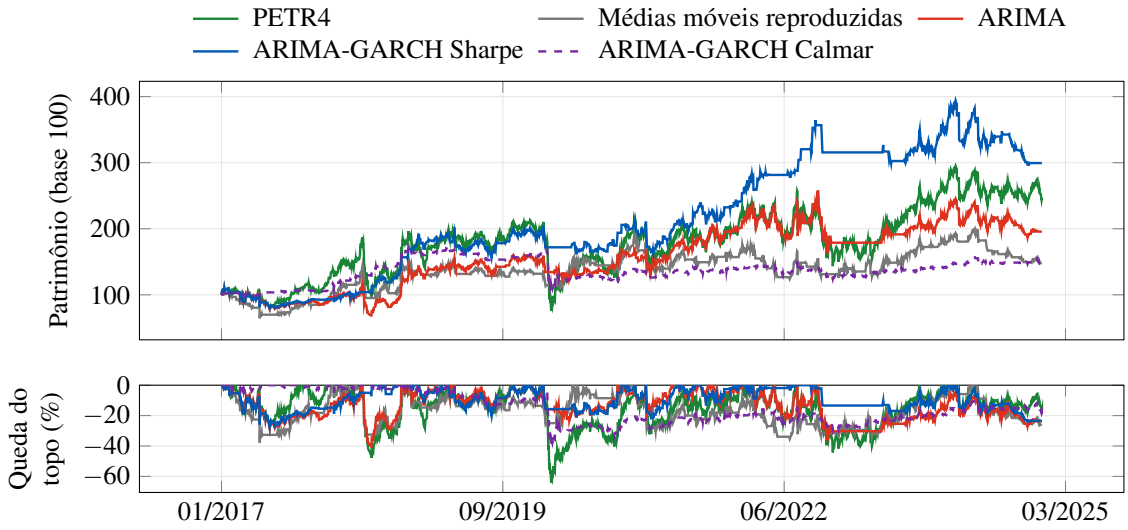
Robô	Retorno	Queda máx.	Volatilidade	Negociações
Médias móveis reproduzidas	+47,8%	37,9%	30,9%	56
ARIMA	+95,7%	40,5%	32,0%	244
ARIMA-GARCH Sharpe	+199,5%	27,2%	24,2%	238
ARIMA-GARCH Calmar	+42,9%	40,2%	16,3%	223
PETR4	+143,6%	64,2%	44,8%	–

Fonte: Elaboração própria.

Nota: Resultado bruto.

mas a sua queda máxima, de 40,2%, ficou no mesmo nível da do ARIMA, de 40,5%, embora ele tivesse sido escolhido justamente por cair pouco na otimização. Ele é o único robô escolhido em que o veto não encerra a posição aberta. Nos dias de volatilidade alta, ele deixa de comprar, mas continua com o que já tinha comprado, e o ajuste do tamanho só vale para as novas compras. Esse é também um exemplo de por que a escolha precisa ser testada em um período novo. O que funciona no período da otimização nem sempre se repete no seguinte.

Gráfico 10 – Patrimônio e queda acumulada, 2017 a 2024



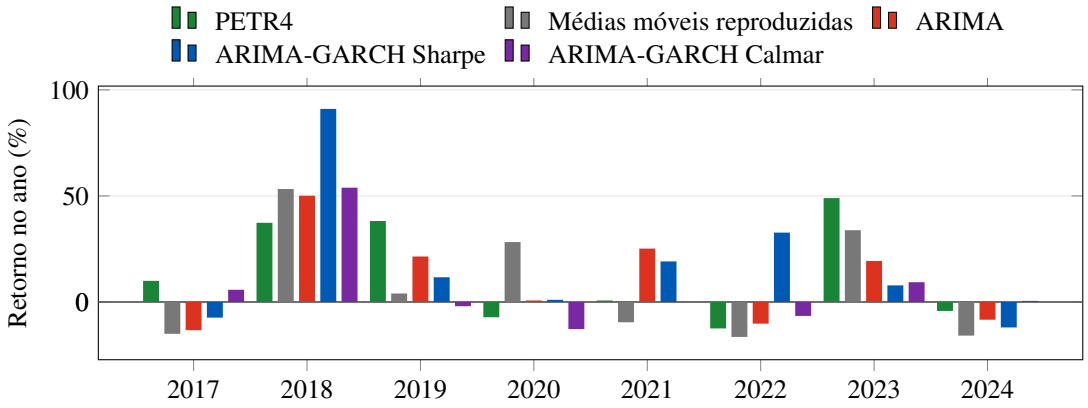
Fonte: Elaboração própria.

Nota: Resultado bruto, com o capital inicial igual a 100.

O Gráfico 10 mostra que a distância entre o ARIMA-GARCH Sharpe e os demais robôs se formou principalmente em dois anos. Pelo Gráfico 11, ele rendeu 91,0% em 2018, contra 50,1% a 53,9% dos outros três robôs, e 32,7% em 2022, ano em que os outros três perderam dinheiro. Nos demais anos, nenhum robô se destacou de forma consistente. As médias móveis, por exemplo, foram as melhores em 2020 e 2023 e as piores em 2017, 2021, 2022 e 2024.

No saldo registrado pelo MetaTrader 5, no Gráfico 12, a queda mais forte do ARIMA aparece por volta da negociação 67. Ela aconteceu em junho de 2018, quando a PETR4 perdeu 47,9% do valor em cerca de um mês, e levou o saldo do ARIMA a R$ 69 mil, o menor do teste. O ARIMA-GARCH atravessou esse trecho sem a mesma queda, já que o seu menor saldo, de

Gráfico 11 – Retorno por ano, 2017 a 2024

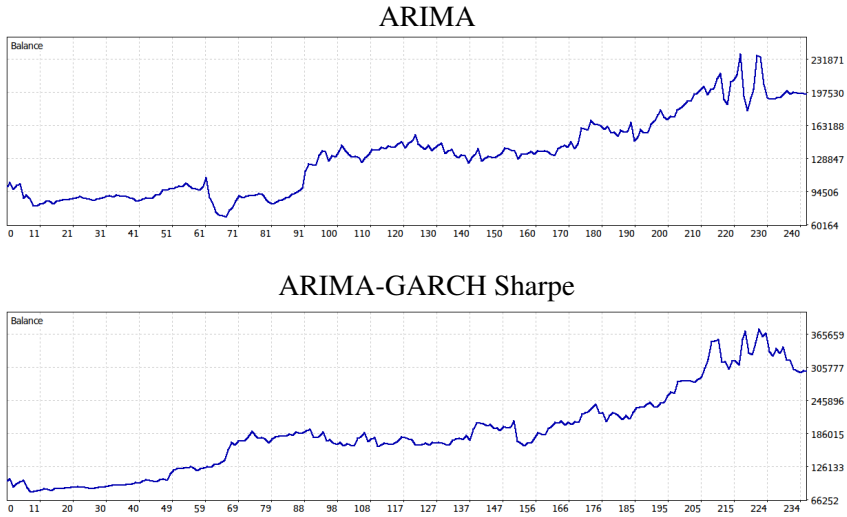


Fonte: Elaboração própria.

Nota: Resultado bruto.

R$ 81 mil, foi registrado logo no início do teste, em julho de 2017. Ao fim do período, o seu saldo era de R$ 299 mil, contra R$ 196 mil do ARIMA.

Gráfico 12 – Saldo registrado pelo MetaTrader 5, 2017 a 2024



Fonte: Relatório do testador de estratégias do MetaTrader 5.

Nota: Saldo em reais após cada negociação encerrada (*Balance*, no original). O eixo horizontal é o número da negociação, e as escalas dos dois painéis são diferentes.

3.6.2 O ano de 2020

O ano de 2020 concentrou a queda provocada pela pandemia, uma crise mais recente que a de 2008. Os números da Tabela 7 são um recorte do teste de 2017 a 2024, com as configurações escolhidas em 2014 a 2016, antes da pandemia.

A PETR4 chegou a perder 64,2% do seu valor no pior momento do ano e terminou 2020 com queda de 7,2%, isto é, a recuperação foi quase tão rápida quanto a queda. Nesse cenário, diferentemente de 2008, as médias móveis tiveram o melhor resultado, com ganho de

Tabela 7 – Resultado em 2020, ano da pandemia

Robô	Retorno no ano	Queda máx. no ano
Médias móveis reproduzidas	+28,2%	19,1%
ARIMA	+0,7%	23,3%
ARIMA-GARCH Sharpe	+1,0%	20,3%
ARIMA-GARCH Calmar	−12,8%	37,7%
PETR4	−7,2%	64,2%

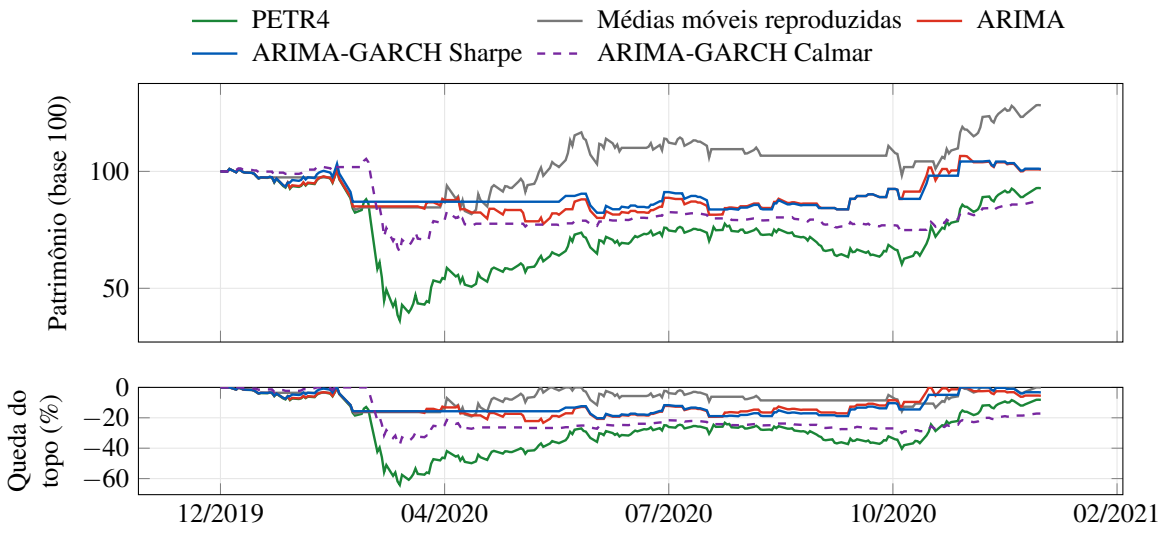
Fonte: Elaboração própria.

Nota: Resultado bruto. A queda máxima é medida dentro do ano, a partir do fechamento de 2019.

28,2% e queda máxima de 19,1%. O ARIMA-GARCH Sharpe protegeu de forma parecida, com queda máxima de 20,3%, mas terminou o ano praticamente sem ganho. O ARIMA-GARCH Calmar, que mantém a posição aberta durante o veto, caiu 37,7%, ainda assim bem menos que a mesma configuração sem o GARCH, que perdeu 49,2% no ano e chegou a cair 59,2%. O resultado mostra que o GARCH protegeu também nessa crise, mas que, nessa queda seguida de recuperação rápida, o cruzamento de médias se saiu tão bem quanto ele na proteção e melhor no retorno.

O Gráfico 13 mostra a evolução do patrimônio ao longo do ano. O ARIMA e o ARIMA-GARCH Sharpe perderam 9,6% em fevereiro, no início da queda, quando ainda tinham quase todo o patrimônio aplicado na ação.

Gráfico 13 – Patrimônio e queda acumulada, 2020



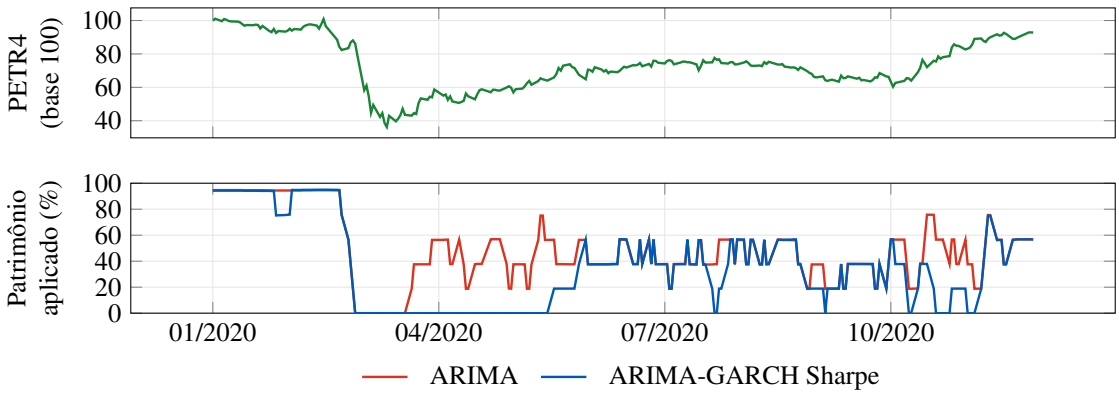
Fonte: Elaboração própria.

Nota: Resultado bruto, com o patrimônio no fim de 2019 igual a 100. A queda é medida dentro do ano.

O Gráfico 14 mostra quanto do patrimônio cada um manteve aplicado. Em março, mês em que a ação caiu 45,9%, os dois ficaram praticamente fora do mercado. O ARIMA sem o GARCH manteve, em média, apenas 8,6% do patrimônio aplicado, o que mostra que, naquele mês, a saída veio da própria previsão do ARIMA. A diferença apareceu depois. Em abril e maio, o GARCH manteve o robô inteiramente fora, enquanto o ARIMA voltou a comprar. A ação

subiu 32,2% em abril, mas o ARIMA perdeu 3,2% naquele mês e 3,0% em maio, e os dois terminaram o ano praticamente empatados.

Gráfico 14 – Preço da PETR4 e patrimônio aplicado na ação, 2020



Fonte: Elaboração própria.

Nota: O patrimônio aplicado é a média dos cinco pregões anteriores, cerca de uma semana. O ARIMA é a mesma configuração do ARIMA-GARCH Sharpe com o GARCH desligado. O preço da PETR4 tem base 100 no fim de 2019.

3.7 Período recente, 2025 a 2026

O último período vai de janeiro de 2025 a setembro de 2026, o mais recente disponível. A Tabela 8 apresenta o resultado.

Tabela 8 – Resultado de 2025 a 2026

Robô	Retorno	Queda máx.	Volatilidade	Negociações
Médias móveis reproduzidas	+18,1%	31,1%	22,7%	12
ARIMA	+14,2%	20,1%	22,0%	46
ARIMA-GARCH Sharpe	−13,7%	17,6%	13,5%	37
ARIMA-GARCH Calmar	−7,4%	12,2%	9,2%	37
PETR4	+33,2%	25,1%	29,1%	–

Fonte: Elaboração própria.

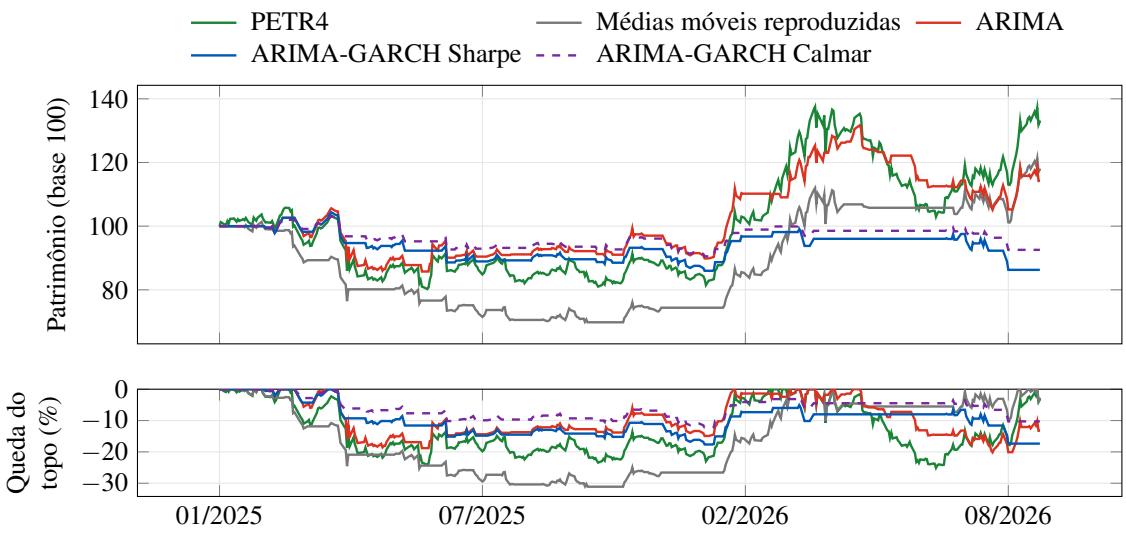
Nota: Resultado bruto. O período termina em 18 de setembro de 2026.

As duas versões com GARCH voltaram a ser as que menos caíram e menos oscilaram, mas foram as únicas com perda. A PETR4 caiu 15,4% em 2025 e subiu 57,5% em 2026. Em 2025, o ARIMA-GARCH Sharpe perdeu 12,5%, mais que o ARIMA, que perdeu 8,5%, e em 2026, mais protegido, ele ficou de fora da alta e terminou o ano com perda de 1,5%, enquanto o ARIMA ganhou 24,8%. Neste período, portanto, a proteção custou mais do que evitou.

O Gráfico 15 mostra a evolução do patrimônio no período.

No saldo registrado pelo MetaTrader 5, no Gráfico 16, o ARIMA-GARCH Sharpe desceu ao longo de 2025, chegou a R$ 86 mil em janeiro de 2026 e não se recuperou. O ARIMA, depois de chegar a R$ 86 mil em junho de 2025, acompanhou a alta e alcançou R$ 128 mil em abril de 2026.

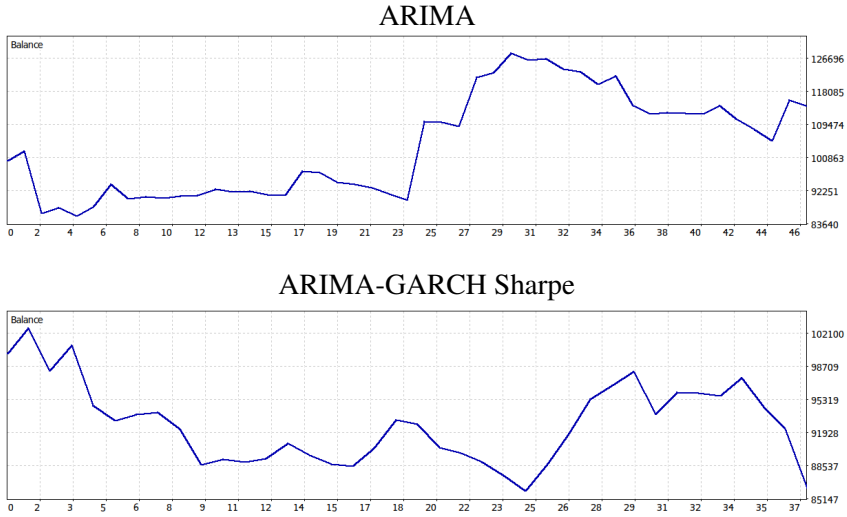
Gráfico 15 – Patrimônio e queda acumulada, 2025 a 2026



Fonte: Elaboração própria.

Nota: Resultado bruto, com o capital inicial igual a 100.

Gráfico 16 – Saldo registrado pelo MetaTrader 5, 2025 a 2026



Fonte: Relatório do testador de estratégias do MetaTrader 5.

Nota: Saldo em reais após cada negociação encerrada (*Balance*, no original). O eixo horizontal é o número da negociação, e as escalas dos dois painéis são diferentes.

3.8 O efeito vem do GARCH?

As seções anteriores comparam robôs diferentes. Para separar o efeito do GARCH do efeito dos demais parâmetros, cada versão foi comparada com a mesma configuração com o GARCH desligado. Contando todas as versões de todos os períodos, são sete comparações, reunidas na Tabela 9.

Em todas as sete comparações, a versão com GARCH teve volatilidade e queda máxima menores que as da mesma configuração sem ele. As comparações não são independentes entre si, já que, em 2013 a 2016 e em 2025 e 2026, o ARIMA-GARCH Sharpe e o ARIMA-GARCH

Tabela 9 – Cada versão com e sem o GARCH

Período	ARIMA-GARCH	Retorno com / sem	Queda máx. com / sem	Volatilidade com / sem
2007 a 2011	Sharpe e Calmar	−8,4% / +7,2%	17,5% / 38,1%	10,5% / 28,2%
2013 a 2016	Sharpe	−13,8% / −9,7%	28,6% / 51,9%	14,1% / 35,1%
2013 a 2016	Calmar	−6,8% / −9,7%	15,5% / 51,9%	7,1% / 35,1%
2017 a 2024	Sharpe	+199,5% / +95,7%	27,2% / 40,5%	24,2% / 32,0%
2017 a 2024	Calmar	+42,9% / −50,1%	40,2% / 79,3%	16,3% / 31,4%
2025 a 2026	Sharpe	−13,7% / +14,2%	17,6% / 20,1%	13,5% / 22,0%
2025 a 2026	Calmar	−7,4% / +14,2%	12,2% / 20,1%	9,2% / 22,0%

Fonte: Elaboração própria.

Nota: Resultado bruto. Em cada linha, *com* é a versão escolhida e *sem* é a mesma configuração com o GARCH desligado.

Calmar têm a mesma configuração sem GARCH, mas o resultado se repetiu em todos os períodos. No retorno, o resultado foi outro. A versão com GARCH rendeu mais em três das sete comparações e menos nas outras quatro, entre elas as duas de 2025 e 2026.

Além dos períodos principais, as configurações escolhidas na otimização de 2010 a 2012, as mesmas do teste de 2013 a 2016, também foram executadas sem nova escolha em 2017 a 2024 e em 2005 a 2026. O segundo período inclui os anos da otimização. A Tabela 10 mostra o resultado desses testes adicionais. Nos quatro casos, a versão com GARCH teve volatilidade e queda máxima menores e retorno maior que os da mesma configuração sem ele.

Tabela 10 – Testes adicionais, com e sem o GARCH

Período	ARIMA-GARCH	Retorno com / sem	Queda máx. com / sem	Volatilidade com / sem
2017 a 2024	Sharpe	+19,9% / +4,6%	31,9% / 57,7%	17,0% / 32,6%
2017 a 2024	Calmar	+7,5% / +4,6%	18,4% / 57,7%	8,6% / 32,6%
2005 a 2026	Sharpe	+74,2% / +16,6%	36,4% / 82,9%	14,3% / 28,8%
2005 a 2026	Calmar	+33,5% / +16,6%	24,4% / 82,9%	7,6% / 28,8%

Fonte: Elaboração própria.

Nota: Resultado bruto, com as configurações escolhidas em 2010 a 2012. As duas usam o mesmo ARIMA, de modo que a configuração sem GARCH é a mesma para as duas.

O efeito sobre o risco se repetiu nas outras três ações, quando se olham todas as configurações da otimização, além das escolhidas. Para isso, elas foram separadas em dois grupos. No primeiro estão as que tiram capital do mercado quando a volatilidade sobe, seja porque ajustam o tamanho da posição, seja porque o veto encerra a posição aberta. Nesse grupo, considerando as quatro ações, o GARCH reduziu a queda máxima em 84% a 92% das comparações em 2007 a 2011 e em 91% a 98% em 2013 a 2016, conforme a ação. Em 2025 e 2026, um período mais calmo, o efeito foi menor, de 51% a 95%. No segundo grupo estão as que apenas impedem novas compras e mantêm a posição aberta. Elas não mostraram o mesmo efeito de forma consistente e reduziram a queda máxima em 31% a 68% das comparações, conforme a ação e o período.

Na comparação com as médias móveis, também no resultado bruto, a vantagem do

GARCH apareceu no risco, mas não no retorno. As versões com GARCH oscilaram menos que as médias móveis em todos os períodos, mas as médias móveis renderam mais que as duas versões em 2013 a 2016 e em 2025 e 2026.

3.9 Como o GARCH protege

O GARCH não informa a direção dos preços, mas poderia melhorar o acerto de forma indireta. Ao vetar as operações nos dias de maior volatilidade prevista, ele deixaria o ARIMA operar apenas quando o seu sinal fosse menos afetado pelo ruído. Os resultados mostram que isso não ocorreu. Em todos os períodos, a taxa de acerto dos robôs com ARIMA, com e sem o GARCH, ficou entre 41% e 59%, perto da metade das negociações, e a diferença entre as versões com e sem o GARCH foi de no máximo quatro pontos percentuais. Esse resultado é coerente com o comportamento das séries de retornos descrito na Seção 1.3, em que o retorno de um dia diz pouco sobre o do dia seguinte.

O formato do ARIMA também não ficou fixo. Como o teste ADF é refeito a cada reavaliação das ordens, o d escolhido pelo robô variou ao longo dos testes. No acompanhamento diário do ARIMA-GARCH Sharpe entre julho de 2019 e junho de 2021, o teste indicou $d = 1$ em 412 dos 496 pregões, ou 83,1%, e $d = 0$ nos demais.

O que o GARCH mudou foi a quantidade de vezes em que o robô agiu. Em 2017 a 2024, o ARIMA-GARCH Sharpe vetou 657 dos 1.983 sinais do ARIMA e aplicou, em média, 43,9% do patrimônio na ação, contra 62,3% da mesma configuração sem o GARCH. A proteção vem, assim, de ficar menos tempo comprado e de ficar fora nos dias de volatilidade prevista mais alta. É o que se espera de um modelo de volatilidade, que serve à gestão de risco (Araya; Aduda; Berhane, 2024). O resultado também vai na mesma direção do observado por Harvey *et al.* (2018), em que ajustar a posição pela volatilidade reduziu a ocorrência de retornos extremos em todos os ativos estudados. Os testes deste trabalho, porém, não separam quanto da redução do risco vem de manter menos capital aplicado e quanto vem de escolher os dias em que se fica fora.

Esse comportamento aparece nas duas crises, nos Gráficos 6 e 14. No período completo de 2007 a 2011, o ARIMA-GARCH aplicou em média 20,1% do patrimônio na ação, contra 45,6% da mesma configuração sem o GARCH.

Um episódio de 2020 mostra o filtro em ação, no ARIMA-GARCH Sharpe de 2017 a 2024, que usa só o filtro. Na abertura de 3 de abril, durante a queda associada à pandemia, o ARIMA previa alta de 2,66% para o dia e, sozinho, teria comprado. A volatilidade prevista pelo GARCH para aquele dia era de 7,58%, cerca de três vezes o limiar vigente, de 2,54%, e a operação foi vetada. Naquele momento, a ação acumulava queda de 48,6% em relação ao fechamento de 2019.

3.10 Efeito dos custos, da Selic e dos proventos

As seções anteriores compararam os modelos pelo resultado bruto. Esta seção mostra como esse resultado muda quando são acrescentados, um de cada vez, os custos das ordens, a remuneração pela Selic do capital que fica fora do mercado e os proventos pagos pela ação, calculados como descrito no Capítulo II. As Tabelas 11 a 14 apresentam o retorno em cada etapa, um período por tabela. Na primeira coluna está o resultado bruto, e cada coluna seguinte acrescenta um fator ao resultado da anterior. Os proventos são uma estimativa, e por isso esta comparação complementa a do resultado bruto, sem substituí-la.

Tabela 11 – Retorno por etapa, 2007 a 2011

Robô	Bruto	+ custos	+ Selic	+ proventos
Médias móveis reproduzidas	−31,6%	−32,8%	−11,6%	−2,9%
ARIMA	−12,7%	−27,1%	−2,1%	+6,5%
ARIMA-GARCH	−8,4%	−15,2%	+28,7%	+33,8%
PETR4	−14,0%	–	–	+2,0%

Fonte: Elaboração própria.

Nota: Cada coluna acrescenta um fator ao resultado da coluna anterior. A Selic rendeu 68,1% no período.

Tabela 12 – Retorno por etapa, 2013 a 2016

Robô	Bruto	+ custos	+ Selic	+ proventos
Médias móveis reproduzidas	+100,8%	+98,1%	+150,9%	+176,0%
ARIMA	−35,8%	−44,8%	−30,5%	−27,9%
ARIMA-GARCH Sharpe	−13,8%	−18,8%	+15,8%	+15,8%
ARIMA-GARCH Calmar	−6,8%	−9,5%	+34,0%	+34,0%
PETR4	−25,6%	–	–	−17,7%

Fonte: Elaboração própria.

Nota: Cada coluna acrescenta um fator ao resultado da coluna anterior. A Selic rendeu 53,9% no período.

Tabela 13 – Retorno por etapa, 2017 a 2024

Robô	Bruto	+ custos	+ Selic	+ proventos
Médias móveis reproduzidas	+47,8%	+43,2%	+90,4%	+396,0%
ARIMA	+95,7%	+70,5%	+114,6%	+356,6%
ARIMA-GARCH Sharpe	+199,5%	+161,8%	+273,8%	+353,0%
ARIMA-GARCH Calmar	+42,9%	+34,4%	+114,1%	+217,0%
PETR4	+143,6%	–	–	+777,9%

Fonte: Elaboração própria.

Nota: Cada coluna acrescenta um fator ao resultado da coluna anterior. A Selic rendeu 86,1% no período.

Os custos pesaram mais para quem negociou mais. Em 2007 a 2011, o ARIMA, com 320 negociações, passou de uma perda de 12,7% para uma de 27,1%, enquanto as médias móveis, com 31, quase não mudaram.

A Selic e os proventos puxam em sentidos opostos. A Selic favorece quem fica mais tempo fora do mercado, porque é o caixa parado que rende. Por isso, ela beneficiou sobretudo o

Tabela 14 – Retorno por etapa, 2025 a 2026

Robô	Bruto	+ custos	+ Selic	+ proventos
Médias móveis reproduzidas	+18,1%	+17,2%	+31,3%	+39,0%
ARIMA	+14,2%	+11,3%	+24,1%	+34,3%
ARIMA-GARCH Sharpe	−13,7%	−15,5%	−0,4%	+5,5%
ARIMA-GARCH Calmar	−7,4%	−8,8%	+9,6%	+13,9%
PETR4	+33,2%	–	–	+57,6%

Fonte: Elaboração própria.

Nota: Cada coluna acrescenta um fator ao resultado da coluna anterior. A Selic rendeu 25,7% no período.

ARIMA-GARCH. Em 2007 a 2011, ele passou de uma perda de 15,2% para um ganho de 28,7%, porque manteve, em média, só 20% do patrimônio aplicado na ação, contra 43% do ARIMA e 48% das médias móveis. Os proventos, ao contrário, favorecem quem fica mais tempo comprado, porque só recebe quem tem a ação. Em 2017 a 2024, como mostra a Tabela 13, eles levaram a PETR4 de 143,6% para 777,9%, acima de todos os robôs, e as médias móveis de 90,4% para 396,0%. Foi o único período em que essas etapas mudaram a ordem entre os robôs e a ação. Nos demais, quem estava à frente no resultado bruto continuou à frente com os custos, a Selic e os proventos. Em 2013 a 2016, por exemplo, as médias móveis ficaram em primeiro lugar em todas as etapas, conforme a Tabela 12.

O retorno, portanto, depende de quais fatores se consideram. O risco, por outro lado, quase não muda. A ordem dos robôs na queda máxima foi a mesma no resultado bruto e depois de cada etapa, em todos os períodos, e a volatilidade praticamente não se alterou. A conclusão sobre o risco não depende dessas escolhas de medida.

3.11 Outras ações

Como complemento, o mesmo procedimento foi repetido em outras três ações, a VALE3, a USIM5 e a POMO4, nos períodos de 2007 a 2011, 2013 a 2016 e 2025 a 2026, com a otimização nos três anos anteriores a cada um. A Tabela 15 mostra o resultado bruto das configurações escolhidas. A comparação com as médias móveis foi feita apenas na PETR4. Como na PETR4, todas as configurações escolhidas usam o filtro, sozinho ou combinado com o ajuste da posição.

O ARIMA-GARCH Calmar teve a menor queda máxima entre os robôs que operaram em sete dos nove casos. Nas duas exceções, em 2025 e 2026, o ARIMA-GARCH Sharpe caiu menos na VALE3, e na USIM5 nenhuma das versões com GARCH chegou a operar. O ARIMA-GARCH Sharpe foi menos regular. Ele caiu mais que o ARIMA em três casos, e nos três a configuração escolhida mantinha a posição aberta durante o veto, como na PETR4. No retorno, o ARIMA sem o GARCH rendeu mais que o ARIMA-GARCH Sharpe em sete dos nove casos, e o maior ganho da tabela foi dele, na POMO4 de 2007 a 2011.

A venda a descoberto, que ficou fora da escolha, foi simulada em todas as configurações dessas ações e também na PETR4. Comparada à mesma configuração que opera só na ponta compradora, ela aumentou a queda máxima em todos os doze casos de ação e período e reduziu

Tabela 15 – Retorno e queda máxima em outras ações

Ação	Período	ARIMA	ARIMA-GARCH Sharpe	ARIMA-GARCH Calmar	Ação
VALE3	2007 a 2011	+60,1% / 43,0%	−38,2% / 40,0%	−10,1% / 25,8%	+23,9% / 67,7%
	2013 a 2016	+34,2% / 34,3%	−27,5% / 32,0%	+2,1% / 9,6%	−40,3% / 79,9%
	2025 a 2026	+4,6% / 10,9%	+23,3% / 5,0%	−0,3% / 12,9%	+34,1% / 21,0%
USIM5	2007 a 2011	−28,7% / 67,0%	+36,4% / 19,1%	+3,3% / 6,1%	−42,7% / 79,1%
	2013 a 2016	−4,3% / 66,0%	−21,5% / 80,4%	−0,9% / 0,9%	−68,8% / 94,3%
	2025 a 2026	+9,3% / 25,6%	sem operações	sem operações	+29,2% / 48,1%
POMO4	2007 a 2011	+275,5% / 24,8%	+4,6% / 10,4%	+4,6% / 10,4%	+138,3% / 71,1%
	2013 a 2016	−21,0% / 40,8%	−67,6% / 70,3%	−21,0% / 26,9%	−58,0% / 77,8%
	2025 a 2026	−4,9% / 27,0%	−10,9% / 34,5%	−1,4% / 5,5%	−36,2% / 54,3%

Fonte: Elaboração própria.

Nota: Resultado bruto, com retorno / queda máxima. Ação: compra e manutenção pela variação do preço.

o retorno em sete deles, considerando a mediana das comparações de cada caso. Uma explicação possível é que, vendendo a descoberto, o robô passa a depender ainda mais da direção prevista pelo ARIMA, cuja taxa de acerto ficou perto da metade, enquanto a proteção vem do GARCH, que só decide se o robô opera e com quanto.

CONSIDERAÇÕES FINAIS

Este trabalho investigou o efeito de acrescentar um modelo de volatilidade condicional a uma estratégia automatizada de negociação cuja previsão de direção permanece a mesma. A questão norteadora foi em que medida a integração dos modelos ARIMA e GARCH em um único algoritmo de negociação altera o desempenho e o risco das operações, em comparação com o uso do ARIMA isolado e com uma estratégia de cruzamento de médias móveis. A introdução perguntava, também, o que efetivamente se ganha ao acrescentar essa camada de modelagem. Em resumo, o ganho está no risco, mais do que no retorno, como se detalha a seguir.

Quanto ao risco, a resposta é clara. Em todos os períodos analisados, a versão do algoritmo com o GARCH oscilou menos e sofreu quedas menores que a mesma configuração com o GARCH desligado, nas sete comparações feitas. O efeito se repetiu nas outras três ações estudadas, nas configurações que tiram capital do mercado quando a volatilidade sobe, sobretudo nos períodos de turbulência. O ano de 2008 resume bem esse comportamento. Enquanto a PETR4 perdia quase metade do seu valor, o ARIMA-GARCH encerrou o ano com uma perda pequena, muito menor que a do ARIMA e a das médias móveis. A proteção veio de o robô ficar menos tempo comprado e de ficar fora nos dias de maior volatilidade prevista, sem melhora na previsão da direção dos preços.

Esse resultado responde à sugestão deixada por Almeida (2016). Ao discutir o prejuízo do seu robô no período que inclui a crise de 2008, o autor apontou a limitação das médias móveis em mercados de alta volatilidade e indicou os modelos de volatilidade como caminho para contorná-la. Nos testes deste trabalho, as médias móveis e o ARIMA sem o GARCH oscilaram de forma muito parecida, o que mostra que trocar o indicador de direção não basta para reduzir o risco. Com o GARCH, o algoritmo oscilou menos que as médias móveis em todos os períodos e sofreu quedas menores em quase todos, com a maior diferença de queda máxima justamente no período de crise em que o robô de Almeida (2016) acumulou prejuízo. Na crise da pandemia, em 2020, a proteção se repetiu, mas as médias móveis protegeram de forma parecida e renderam mais, o que sugere que a vantagem do GARCH sobre elas depende do tipo de crise.

Quanto ao retorno, o efeito foi menos claro. A versão com GARCH terminou com retorno maior que o da mesma configuração sem ele em três das sete comparações e menor nas outras quatro, ao contrário do que ocorreu com o risco. Em relação às médias móveis, o ARIMA-GARCH ganhou em alguns períodos e perdeu em outros, sem vantagem consistente para nenhum dos dois. Esse resultado é coerente com a função do modelo. O GARCH descreve o tamanho das oscilações, sem informar a direção dos preços, e o seu efeito apareceu no risco, mais do que no lucro.

A primeira hipótese afirmava que a integração entre ARIMA e GARCH gera sinais mais eficazes que o ARIMA isolado, tanto em acerto direcional quanto em adequação ao risco. Ela foi parcialmente confirmada. A adequação ao risco foi confirmada pela volatilidade e pela queda máxima menores que as da mesma configuração sem o GARCH, em todas as comparações. O

acerto direcional, por outro lado, não melhorou nem piorou. Com e sem o GARCH, a diferença foi de no máximo quatro pontos percentuais, e a taxa de acerto das negociações permaneceu perto da metade. Esperava-se que, ao deixar de operar nos dias de maior volatilidade prevista, o algoritmo acertasse mais nas operações restantes, o que não ocorreu. O ganho veio de ficar menos tempo exposto nos dias de maior oscilação, sem aumento no acerto.

A segunda hipótese afirmava que a combinação representa um avanço em relação às análises baseadas em médias móveis. Na PETR4, único ativo em que as médias móveis foram comparadas, ela foi confirmada quanto ao risco. Todas as variantes com GARCH oscilaram menos que as médias móveis, e a queda máxima foi menor em quase todas. A exceção foi a variante que mantém a posição aberta durante o veto, no período de 2017 a 2024, a única escolhida na PETR4 em que o veto não encerra a posição. Isso indica que a proteção é mais consistente quando o GARCH, além de impedir novas compras, tira o capital do mercado nos dias de volatilidade alta. Quanto ao retorno, cada algoritmo levou vantagem em períodos diferentes, e não se pode afirmar avanço nesse sentido.

O objetivo geral, desenvolver em MQL5 um *Expert Advisor* que integra os modelos ARIMA e GARCH e avaliar experimentalmente o efeito da inclusão do GARCH, foi alcançado, assim como os objetivos específicos. Os conceitos de tendência e de volatilidade foram reunidos no Capítulo I e serviram de base para a divisão de tarefas entre os modelos. O ARIMA e o GARCH foram implementados em MQL5 e integrados em um mesmo robô, no qual o primeiro indica a direção da operação e o segundo controla o risco, pelo filtro de operações e pelo ajuste do tamanho da posição. A estratégia de Almeida (2016) foi reimplementada, e a reprodução dos seus testes confirmou o comportamento do algoritmo original. Os valores em reais dependem do histórico que o autor usou, que o estudo original não registra, e só se aproximaram dos relatados sob a hipótese de que esse histórico começasse em 2008. Por fim, os três algoritmos foram comparados quanto a risco e retorno em períodos de crise e de calma, sobre dados oficiais e com períodos de otimização e de teste separados.

Os resultados têm limitações. As diferenças de retorno não se repetiram de forma consistente entre os períodos, e a configuração com melhor retorno em um período nem sempre repetiu o desempenho no seguinte, embora a proteção tenha se mantido. Os ganhos se concentraram em poucos anos. A análise se restringe a quatro ativos de um único mercado, e a comparação com as médias móveis, à PETR4. A consistência dos resultados foi avaliada pela repetição entre períodos, configurações e ativos, sem teste estatístico formal, e as comparações não são independentes entre si.

O GARCH foi estimado sobre a mesma janela do ARIMA, de no máximo 240 pregões, tamanho que decorreu da forma como os testes foram montados no testador. Essa janela é relativamente curta para estimar o GARCH, e os seus parâmetros variaram bastante de um dia para o outro. Um teste à parte, com uma janela de 500 pregões apenas para o GARCH, manteve a redução do risco nas configurações que reduzem a posição, mas mudou as configurações escolhidas e os seus retornos, o que reforça a cautela com os números de retorno de cada

configuração.

A escolha das configurações considerou os custos e a remuneração do caixa pela Selic, o que pode favorecer as que passam mais tempo fora do mercado, e os robôs com GARCH escolhidos mantiveram aplicada, em média, uma parte menor do patrimônio que os demais. Parte da redução do risco vem dessa menor exposição, e os testes não separam quanto vem dela e quanto vem da escolha dos dias em que o robô fica fora. A comparação principal usa o resultado bruto, sem custos, sem a remuneração do caixa e sem proventos. Isso favorece os robôs que negociam com mais frequência, como o ARIMA sem o GARCH, desfavorece os que ficam mais tempo fora do mercado, como o ARIMA-GARCH, e desfavorece também a compra e manutenção e as estratégias que permanecem mais tempo compradas, que deixam de receber os proventos. Esses fatores aparecem apenas na análise complementar, e os proventos, como estimativa. Com eles, a compra e manutenção superou todos os robôs em 2017 a 2024. Além disso, os experimentos foram realizados por simulação sobre dados históricos, com execução no preço de abertura e sem considerar a falta de liquidez ou a diferença entre o preço pedido e o executado, condições que só podem ser verificadas em operação real.

Como continuação, o resultado indica que o caminho mais promissor está no controle de risco, que foi onde o modelo mostrou efeito. Uma possibilidade é aplicar o mesmo protocolo a um conjunto maior de ativos e acompanhar o algoritmo em conta de demonstração, em tempo real, o que permitiria também avaliar as ordens de limite de perda e de realização de lucro que Almeida (2016) previu em seu algoritmo, mas não utilizou em seus testes. Outra é testar janelas de estimação maiores, em especial para o GARCH, iniciando a simulação antes do período de teste, e a janela expansiva, que acumula todo o histórico e que, no estudo de Xing *et al.* (2024), deu ao ARIMA melhores resultados que a janela deslizante. Uma terceira é substituir o GARCH simétrico por uma das variações assimétricas citadas no Capítulo I, como o EGARCH e o TGARCH, que fazem a volatilidade reagir mais às quedas que às altas, possibilidade que não foi contemplada neste trabalho.

No eixo da previsão, a próxima etapa é a que estava prevista no projeto original deste trabalho e foi deixada de lado para que a implementação do ARIMA e do GARCH pudesse ser concluída: substituir o ARIMA por uma rede neural recorrente do tipo LSTM, do inglês *Long Short-Term Memory*, capaz de representar relações não lineares, mantendo o GARCH como filtro de risco. A arquitetura do algoritmo, que separa a previsão da direção da avaliação do risco, facilita essa troca. Os resultados deste trabalho, porém, recomendam cautela na expectativa, já que a taxa de acerto das negociações ficou perto da metade e o ganho observado veio do controle de risco.

REFERÊNCIAS

ALMEIDA, F. L. R. *Técnicas de previsão da bolsa com MetaTrader 5*. Monografia (Graduação em Tecnologia em Banco de Dados) – Faculdade de Tecnologia de Indaiatuba, Indaiatuba, 2016. No Repositório Institucional do Centro Paula Souza o trabalho está catalogado sob o título "Análise da ação PETR4 na Bolsa de Valores de São Paulo com MetaTrader 5". Disponível em: https://ric.cps.sp.gov.br/handle/123456789/24287. Acesso em: 20 fev. 2025.

ARAYA, H. T.; ADUDA, J.; BERHANE, T. A hybrid GARCH and deep learning method for volatility prediction. *Journal of Applied Mathematics*, Londres, Reino Unido, v. 2024, 2024. Artigo 6305525. DOI: 10.1155/2024/6305525.

B3. *Empresas listadas: Petróleo Brasileiro S.A. Petrobras, proventos em dinheiro e eventos corporativos*. São Paulo, 2026a. Disponível em: https://sistemaswebb3-listados.b3.com.br/listedCompaniesPage/main/9512/PETR/overview?language=pt-br. Acesso em: 21 out. 2026.

B3. *Séries históricas de cotações (COTAHIST): arquivos anuais de 2003 a 2026*. São Paulo, 2026b. Disponível em: https://www.b3.com.br/pt_br/market-data-e-indices/servicos-de-dados/market-data/historico/mercado-a-vista/series-historicas/. Acesso em: 02 nov. 2026.

B3. *Tarifas de ações e fundos de investimento: à vista*. São Paulo, 2026c. Disponível em: https://www.b3.com.br/pt_br/produtos-e-servicos/tarifas/listados-a-vista-e-derivativos/renda-variavel/tarifas-de-acoes-e-fundos-de-investimento/a-vista/. Acesso em: 21 out. 2026.

BANCO CENTRAL DO BRASIL. *Bolsas de valores*. Brasília, DF, 2025a. Disponível em: https://www.bcb.gov.br/pre/composicao/bv.asp?frame=1. Acesso em: 22 abr. 2025.

BANCO CENTRAL DO BRASIL. *Corretoras e distribuidoras*. Brasília, DF, 2025b. Disponível em: https://www.bcb.gov.br/acessoinformacao/legado?url=https:%2F%2Fwww.bcb.gov.br%2Fpre%2Fcomposicao%2Fcorretoras_distribuidoras.asp. Acesso em: 22 abr. 2025.

BANCO CENTRAL DO BRASIL. *Taxa Selic*. Brasília, DF, 2025c. Disponível em: https://www.bcb.gov.br/estatisticas/detalhamentoGrafico/graficoshome/selic. Acesso em: 22 abr. 2025.

BANCO CENTRAL DO BRASIL. *Sistema Gerenciador de Séries Temporais: série 11, taxa de juros Selic, % ao dia*. Brasília, DF, 2026. Disponível em: https://api.bcb.gov.br/dados/serie/bcdata.sgs.11/dados. Acesso em: 21 set. 2026.

BATISTA, C. B. *Desenvolvimento de robô para automação de investimentos na bolsa de valores utilizando indicadores técnicos*. Monografia (Graduação em Engenharia de Computação) – Pontifícia Universidade Católica de Goiás, Goiânia, 2024. Disponível em: https://repositorio.pucgoias.edu.br/jspui/handle/123456789/8073. Acesso em: 30 mar. 2025.

BOLLERSLEV, T. Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, Elsevier, Amsterdam, v. 31, n. 3, p. 307–327, 1986. DOI: 10.1016/0304-4076(86)90063-1.

CHERNISH, E. *Ferramentas econométricas para previsão de volatilidade: modelo GARCH*. 2024. MQL5. Disponível em: https://www.mql5.com/pt/articles/15223. Acesso em: 02 nov. 2026.

DUBE, F. *Implementando um algoritmo de treinamento ARIMA em MQL5*. 2023. MQL5. Disponível em: https://www.mql5.com/pt/articles/12583. Acesso em: 02 nov. 2026.

DUBE, F. *Implementação do teste aumentado de Dickey-Fuller no MQL5*. 2024. MQL5. Disponível em: https://www.mql5.com/pt/articles/13991. Acesso em: 02 nov. 2026.

ENGLE, R. F. Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica*, The Econometric Society, New Jersey, EUA, v. 50, n. 4, p. 987–1007, 1982. DOI: 10.2307/1912773.

FERNANDES, A. *Fundamentos de análise técnica de ações*. 1. ed. Rio de Janeiro: Alexandre Fernandes, 2014.

GÓMEZ, V.; MARAVALL, A. *Automatic modeling methods for univariate series*. Madrid: Banco de España, 1998. (Documento de Trabajo, n. 9808). Disponível em: https://www.bde.es/f/webbde/SES/Secciones/Publicaciones/PublicacionesSeriadas/DocumentosTrabajo/98/Fic/dt9808e.pdf. Acesso em: 19 out. 2026.

HANNAN, E. J.; RISSANEN, J. Recursive estimation of mixed autoregressive-moving average order. *Biometrika*, v. 69, n. 1, p. 81–94, 1982. DOI: 10.2307/2335856.

HARVEY, C. R. et al. The impact of volatility targeting. *The Journal of Portfolio Management*, v. 45, n. 1, p. 14–33, 2018. DOI: 10.3905/jpm.2018.45.1.014.

MÁXIMO, W. Bolsa cai 30%, e dólar sobe 16% em março com pandemia de coronavírus. *Agência Brasil*, Brasília, 31 mar. 2020. Disponível em: https://agenciabrasil.ebc.com.br/economia/noticia/2020-03/bolsa-cai-30-e-dolar-sobe-16-em-marco-com-pandemia-de-coronavirus. Acesso em: 05 out. 2026.

METAQUOTES LTD. *MetaTrader 5: plataforma de negociação para Forex, ações e futuros*. Limassol: MetaQuotes Ltd., 2024. Disponível em: https://www.metatrader5.com/pt. Acesso em: 01 mar. 2025.

METAQUOTES LTD. *Estratégias de teste*. Limassol, 2026a. Ajuda para o MetaTrader 5. Página da ajuda da plataforma, sem data de publicação. Disponível em: https://www.metatrader5.com/pt/terminal/help/algotrading/testing. Acesso em: 27 out. 2026.

METAQUOTES LTD. *Manual MQL5*. Limassol, 2026b. Referência da linguagem MQL5. Documentação oficial da linguagem, sem data de publicação. Disponível em: https://www.mql5.com/pt/docs. Acesso em: 21 out. 2026.

METAQUOTES LTD. *Relatório de teste*. Limassol, 2026c. Ajuda para o MetaTrader 5. Página da ajuda da plataforma, sem data de publicação. Disponível em: https://www.metatrader5.com/pt/terminal/help/algotrading/testing_report. Acesso em: 05 out. 2026.

METAQUOTES LTD. *Tipos de otimização*. Limassol, 2026d. Ajuda para o MetaTrader 5. Página da ajuda da plataforma, sem data de publicação. Disponível em: https://www.metatrader5.com/pt/terminal/help/algotrading/optimization_types. Acesso em: 27 out. 2026.

PETRÓLEO BRASILEIRO S.A. *Extraordinary General Meeting approves stock split*. Rio de Janeiro, 2005. Comunicado de 22 jul. 2005, enviado à Securities and Exchange Commission (Form 6-K). Disponível em: https://www.sec.gov/Archives/edgar/data/0001119639/000129281405001096/pbra20050722_6k2.htm. Acesso em: 21 out. 2026.

SHARPE, W. F. The Sharpe ratio. *The Journal of Portfolio Management*, Nova Iorque, v. 21, n. 1, p. 49–58, 1994. Disponível em: https://web.stanford.edu/~wfsharpe/art/sr/sr.htm. Acesso em: 05 out. 2026.

THOMAZ, P. S. *Modelos ARMA-GARCH na modelagem da volatilidade de ações financeiras*. Dissertação (Mestrado em Modelagem Computacional) – Universidade Federal do Rio Grande, Rio Grande, 2019. Disponível em: https://repositorio.furg.br/handle/1/10204. Acesso em: 12 out. 2026.

WAZLAWICK, R. S. *Metodologia de pesquisa para ciência da computação*. 3. ed. Rio de Janeiro: LTC, 2021.

XING, Y. et al. *Predicting NVIDIA's next-day stock price: a comparative analysis of LSTM, MLP, ARIMA, and ARIMA-GARCH models*. 2024. arXiv. Preprint. DOI: 10.48550/arXiv.2405.08284. Disponível em: https://arxiv.org/abs/2405.08284. Acesso em: 09 abr. 2025.

APÊNDICES

APÊNDICE A – Código do *Expert Advisor*

Este apêndice apresenta o código do *Expert Advisor* desenvolvido neste trabalho. As rotinas de estimação do ARIMA e do GARCH, de seleção das ordens e das formas de uso do GARCH estão em arquivos próprios, chamados pelas diretivas `#include` do início do código, e são apresentadas no Apêndice B. Para facilitar a leitura, foram retirados os comentários, as mensagens de registro e as opções do robô que não foram usadas nos experimentos, como a aplicação do modelo ao preço sem logaritmo, a definição manual das ordens do ARIMA e as ordens de limite de perda e de realização de lucro. Os valores atribuídos aos parâmetros no código são apenas os padrões do programa. Os valores testados estão no Quadro 3, e os das configurações escolhidas, no Quadro 4.

```
#property copyright "TG - FATEC Indaiatuba (ADS)"
#property version   "3.00"

#include "..\Include\Arima.mqh"
#include "..\Include\Garch.mqh"
#include "..\Include\SelecaoModelo.mqh"
#include "..\Include\Estrategia.mqh"
#include "..\Include\Negociacao.mqh"

input bool                UsarGARCH            = true;

input int                 Max_p                = 4;
input int                 Max_q                = 4;
input int                 Max_d                = 2;
input int                 Janela_Selecao       = 240;
input int                 Barras_Entre_Reselecao = 21;
input bool                IncluirConstante     = false;

input int                 Janela_Estimacao     = 120;

input ENUM_ESTRATEGIA_GARCH Estrategia         = FILTRO_VOLATILIDADE;
input ENUM_MODO_LIMIAR    ModoLimiar           = LIMIAR_PERCENTIL;
input double              Limiar_Absoluto      = 0.020;
input double              Limiar_Percentil     = 50.0;
input double              Vol_Alvo             = 0.015;
input double              Limiar_Sinal         = 0.0;

input double              Fracao_Capital       = 0.95;
input double              Volume_Base          = 100;
input double              Frac_Volume_Min      = 0.25;
input double              Frac_Volume_Max      = 2.00;
input bool                SomenteCompra        = true;
input bool                ZerarNoVeto          = false;

input ulong               MagicNumber          = 20260803;
input int                 Slippage             = 10;

CArima      g_arima;
CGarch      g_garch;
CNegociacao g_negociacao;

int      g_ordem_p = 1, g_ordem_d = 1, g_ordem_q = 1;
bool     g_ordens_definidas = false;
int      g_barras_desde_selecao = 0;
datetime g_ultima_barra = 0;

```

```
47 int OnInit()
48   {
49    g_negociacao.Configurar(_Symbol, MagicNumber, Slippage);
50
51    SelecionarOrdens();
52
53    g_arima.Definir(g_ordem_p, g_ordem_d, g_ordem_q, IncluirConstante);
54    g_garch.Definir(1, 1);
55    g_garch.DefinirOtimizador(300);
56
57    return(INIT_SUCCEEDED);
58   }
59
60 void OnTick()
61   {
62    datetime barra = (datetime)SeriesInfoInteger(_Symbol, _Period, SERIES_LASTBAR_DATE);
63    if(barra == g_ultima_barra)
64       return;
65    g_ultima_barra = barra;
66
67    ReselecionarOrdensSeVencidas();
68    if(!g_ordens_definidas)
69       return;
70
71    double serie[];
72    if(!CarregarSerie(Janela_Estimacao, serie))
73       return;
74
75    if(!g_arima.Ajustar(serie))
76       return;
77
78    double previsao = PrevisaoDoRetorno(serie);
79    if(!MathIsValidNumber(previsao))
80       return;
81
82    double volatilidade = 0.0;
83    double limiar = Limiar_Absoluto;
84    if(UsarGARCH && !EstimarVolatilidade(volatilidade, limiar))
85       return;
86
87    DecisaoOperacao decisao;
88    DecidirOperacao(previsao, volatilidade, UsarGARCH, Estrategia, limiar, Vol_Alvo,
89                    Volume_Base, Frac_Volume_Min, Frac_Volume_Max, Limiar_Sinal, decisao);
90
91    if(decisao.sinal != 0)
92       Executar(decisao);
93    else if(decisao.vetada_pelo_garch && ZerarNoVeto)
94       FecharPosicaoAberta();
95   }
96
97 bool CarregarSerie(const int quantidade, double &serie[])
98   {
99    double fechamento[];
100   int copiados = CopyClose(_Symbol, _Period, 1, quantidade, fechamento);
101   if(copiados < quantidade)
102      return(false);
103
104   ArrayResize(serie, copiados);
105   for(int i = 0; i < copiados; i++)
106     {
107      if(fechamento[i] <= 0.0)
108         return(false);
109      serie[i] = MathLog(fechamento[i]);
```

```
110       }
111     return(true);
112   }
113
114 bool SelecionarOrdens()
115   {
116     double serie[];
117     int janela = MathMax(Janela_Selecao, Janela_Estimacao);
118     if(!CarregarSerie(janela, serie))
119        return(false);
120
121     ResultadoSelecao selecao;
122     bool selecionou = SelecionarModelo(serie, Max_d, Max_p, Max_q, IncluirConstante,
            selecao);
123     if(!selecionou)
124        return(false);
125
126     g_ordem_p = selecao.p;
127     g_ordem_d = selecao.d;
128     g_ordem_q = selecao.q;
129     g_ordens_definidas = true;
130     g_barras_desde_selecao = 0;
131     return(true);
132   }
133
134 void ReselecionarOrdensSeVencidas()
135   {
136     bool vencidas = !g_ordens_definidas ||
137                     (Barras_Entre_Reselecao > 0 && g_barras_desde_selecao >=
                            Barras_Entre_Reselecao);
138     if(vencidas && SelecionarOrdens())
139        g_arima.Definir(g_ordem_p, g_ordem_d, g_ordem_q, IncluirConstante);
140     g_barras_desde_selecao++;
141   }
142
143 double PrevisaoDoRetorno(const double &serie[])
144   {
145     if(g_arima.D() == 1)
146        return(g_arima.PreverDiferenciado());
147     return(g_arima.PreverNivel() - serie[ArraySize(serie) - 1]);
148   }
149
150 bool EstimarVolatilidade(double &volatilidade, double &limiar)
151   {
152     double residuos[];
153     if(!g_arima.ObterResiduos(residuos))
154        return(false);
155     if(!g_garch.Ajustar(residuos))
156        return(false);
157
158     volatilidade = g_garch.VolatilidadeProximoPeriodo();
159
160     if(ModoLimiar == LIMIAR_PERCENTIL)
161       {
162        double historico[];
163        limiar = g_garch.ObterSerieVolatilidade(historico) ? Percentil(historico,
              Limiar_Percentil) : Limiar_Absoluto;
164       }
165     return(true);
166   }
167
168 void Executar(const DecisaoOperacao &decisao)
169   {
```

```
170     MqlTick tick;
171     if(!SymbolInfoTick(_Symbol, tick) || tick.ask <= 0.0 || tick.bid <= 0.0)
172        return;
173
174     ENUM_POSITION_TYPE posicao = g_negociacao.PosicaoAtual();
175     if(decisao.sinal < 0 && SomenteCompra)
176       {
177        if(posicao == POSITION_TYPE_BUY)
178           g_negociacao.Fechar(POSITION_TYPE_BUY);
179        return;
180       }
181
182     bool compra = (decisao.sinal > 0);
183     double preco = compra ? tick.ask : tick.bid;
184     double volume = g_negociacao.AjustarVolume(VolumeDesejado(decisao.volume, preco));
185     if(volume <= 0.0)
186        return;
187
188     ENUM_POSITION_TYPE mesma   = compra ? POSITION_TYPE_BUY : POSITION_TYPE_SELL;
189     ENUM_POSITION_TYPE oposta  = compra ? POSITION_TYPE_SELL : POSITION_TYPE_BUY;
190     if(posicao == oposta)
191       {
192        if(!g_negociacao.Fechar(oposta))
193           return;
194        posicao = g_negociacao.PosicaoAtual();
195       }
196     if(posicao == mesma)
197        return;
198
199     if(compra)
200        g_negociacao.Comprar(volume, preco, 0.0, 0.0, "ARIMA-GARCH BUY");
201     else
202        g_negociacao.Vender(volume, preco, 0.0, 0.0, "ARIMA-GARCH SELL");
203   }
204
205 void FecharPosicaoAberta()
206   {
207    ENUM_POSITION_TYPE posicao = g_negociacao.PosicaoAtual();
208    if(posicao == POSITION_TYPE_BUY || posicao == POSITION_TYPE_SELL)
209       g_negociacao.Fechar(posicao);
210   }
211
212 double VolumeDesejado(const double volume_decisao, const double preco)
213   {
214    double fator = (Volume_Base > 0.0) ? volume_decisao / Volume_Base : 1.0;
215    return(VolumePorCapital(_Symbol, Fracao_Capital * fator, preco));
216   }
```

APÊNDICE B – Código dos modelos

Este apêndice reúne os arquivos chamados pelo *Expert Advisor* do Apêndice A que contêm os modelos. Como no Apêndice A, foram retirados os comentários, as mensagens de registro e as opções experimentais que não fazem parte deste trabalho. As funções auxiliares de mínimos quadrados e dos testes estatísticos, assim como as rotinas de envio de ordens, ficam em arquivos próprios, que não são reproduzidos aqui.

B.1 Classe do ARIMA

A classe `CArima` diferencia a série, estima os coeficientes pelo método de Hannan e Rissanen, calcula os resíduos e o AIC e faz a previsão do período seguinte.

```
#ifndef ALGOTRADINGLAB_ARIMA_MQH
#define ALGOTRADINGLAB_ARIMA_MQH

#include "Numerico.mqh"
#include "Estatistica.mqh"

class CArima
  {
private:
   int      m_p;
   int      m_d;
   int      m_q;
   bool     m_com_constante;

   double   m_constante;
   double   m_phi[];
   double   m_theta[];

   double   m_serie_diferenciada[];
   double   m_residuos[];
   double   m_ultimos_niveis[];
   double   m_sigma2;
   double   m_log_verossimilhanca;
   int      m_inicio_recursao;
   int      m_inicio_verossimilhanca;
   int      m_aquecimento_minimo;
   int      m_observacoes_efetivas;
   bool     m_ajustado;

   void     Diferenciar(const double &origem[], double &destino[], const int ordem);
   bool     EstimarAR(const double &z[], const int ordem, double &coeficientes[], double
       &residuos[]);
   bool     HannanRissanen(void);
   void     CalcularResiduos(void);
   void     CalcularVerossimilhanca(void);

public:
                     CArima(void);

   void     Definir(const int p, const int d, const int q, const bool com_constante =
       true);

   void     DefinirAquecimentoMinimo(const int observacoes) { m_aquecimento_minimo =
       MathMax(0, observacoes); }
```

```
42
43    bool      Ajustar(const double &serie[]);
44
45    double    PreverDiferenciado(void);
46    double    PreverNivel(void);
47
48    bool      ObterResiduos(double &saida[]) const;
49
50    int       NumParametros(void)         const { return(m_p + m_q + (m_com_constante ? 1 :
          0) + 1); }
51    double    AIC(void)                   const;
52
53    int       D(void) const { return(m_d); }
54   };
55
56 CArima::CArima(void) : m_p(1), m_d(1), m_q(1), m_com_constante(true),
57                        m_constante(0.0), m_sigma2(0.0), m_log_verossimilhanca(0.0),
58                        m_inicio_recursao(0), m_inicio_verossimilhanca(0),
                              m_aquecimento_minimo(0),
59                        m_observacoes_efetivas(0), m_ajustado(false)
60   {
61   }
62
63 void CArima::Definir(const int p, const int d, const int q, const bool com_constante)
64   {
65    m_p = MathMax(0, p);
66    m_d = MathMax(0, MathMin(2, d));
67    m_q = MathMax(0, q);
68    m_com_constante = com_constante || (m_d == 0);
69
70    ArrayResize(m_phi, m_p);
71    ArrayResize(m_theta, m_q);
72    ArrayInitialize(m_phi, 0.0);
73    ArrayInitialize(m_theta, 0.0);
74    m_ajustado = false;
75   }
76
77 void CArima::Diferenciar(const double &origem[], double &destino[], const int ordem)
78   {
79    ArrayResize(destino, ArraySize(origem));
80    ArrayCopy(destino, origem);
81
82    for(int vez = 0; vez < ordem; vez++)
83      {
84       int tamanho = ArraySize(destino);
85       if(tamanho < 2)
86          break;
87       for(int i = 0; i < tamanho - 1; i++)
88          destino[i] = destino[i + 1] - destino[i];
89       ArrayResize(destino, tamanho - 1);
90      }
91   }
92
93 bool CArima::EstimarAR(const double &z[], const int ordem, double &coeficientes[], double
       &residuos[])
94   {
95    int tamanho = ArraySize(z);
96    int observacoes = tamanho - ordem;
97    int parametros = ordem + 1;
98
99    if(ordem <= 0 || observacoes <= parametros)
100      return(false);
101
```

```
102     CMatrixDouble X(observacoes, parametros);
103     double dependente[];
104     ArrayResize(dependente, observacoes);
105
106     for(int t = ordem, linha = 0; t < tamanho; t++, linha++)
107       {
108        dependente[linha] = z[t];
109        X.Set(linha, 0, 1.0);
110        for(int i = 1; i <= ordem; i++)
111           X.Set(linha, i, z[t - i]);
112       }
113
114     double erros_padrao[], soma_quadrados = 0.0;
115     if(!MQO(X, dependente, observacoes, parametros, coeficientes, erros_padrao,
            soma_quadrados, false))
116        return(false);
117
118     ArrayResize(residuos, tamanho);
119     ArrayInitialize(residuos, 0.0);
120     for(int t = ordem; t < tamanho; t++)
121       {
122        double ajuste = coeficientes[0];
123        for(int i = 1; i <= ordem; i++)
124           ajuste += coeficientes[i] * z[t - i];
125        residuos[t] = z[t] - ajuste;
126       }
127
128     return(true);
129   }
130
131 bool CArima::HannanRissanen(void)
132   {
133    int tamanho = ArraySize(m_serie_diferenciada);
134
135    if(m_q == 0)
136      {
137       if(m_p == 0)
138         {
139          m_constante = m_com_constante ? Media(m_serie_diferenciada) : 0.0;
140          return(true);
141         }
142
143       int observacoes = tamanho - m_p;
144       int parametros = m_p + (m_com_constante ? 1 : 0);
145       if(observacoes <= parametros)
146          return(false);
147
148       CMatrixDouble X(observacoes, parametros);
149       double dependente[];
150       ArrayResize(dependente, observacoes);
151
152       for(int t = m_p, linha = 0; t < tamanho; t++, linha++)
153         {
154          dependente[linha] = m_serie_diferenciada[t];
155          int coluna = 0;
156          if(m_com_constante)
157             X.Set(linha, coluna++, 1.0);
158          for(int i = 1; i <= m_p; i++)
159             X.Set(linha, coluna++, m_serie_diferenciada[t - i]);
160         }
161
162       double coeficientes[], erros_padrao[], soma_quadrados = 0.0;
```

```
163       if(!MQO(X, dependente, observacoes, parametros, coeficientes, erros_padrao,
              soma_quadrados, false))
164          return(false);
165
166       int coluna = 0;
167       m_constante = m_com_constante ? coeficientes[coluna++] : 0.0;
168       for(int i = 0; i < m_p; i++)
169          m_phi[i] = coeficientes[coluna++];
170       return(true);
171      }
172
173    int ordem_longa = (int)MathFloor(MathPow(MathLog((double)tamanho), 2.0));
174    ordem_longa = MathMax(ordem_longa, m_p + m_q + 1);
175    ordem_longa = MathMin(ordem_longa, tamanho / 5);
176
177    if(ordem_longa < 1 || tamanho - ordem_longa < m_p + m_q + 10)
178       return(false);
179
180    double coeficientes_ar_longo[], erros_estimados[];
181    if(!EstimarAR(m_serie_diferenciada, ordem_longa, coeficientes_ar_longo,
           erros_estimados))
182       return(false);
183
184    int primeiro = MathMax(ordem_longa + m_q, m_p);
185    int observacoes = tamanho - primeiro;
186    int parametros = m_p + m_q + (m_com_constante ? 1 : 0);
187
188    if(observacoes <= parametros + 5)
189       return(false);
190
191    CMatrixDouble X(observacoes, parametros);
192    double dependente[];
193    ArrayResize(dependente, observacoes);
194
195    for(int t = primeiro, linha = 0; t < tamanho; t++, linha++)
196      {
197       dependente[linha] = m_serie_diferenciada[t];
198       int coluna = 0;
199       if(m_com_constante)
200          X.Set(linha, coluna++, 1.0);
201       for(int i = 1; i <= m_p; i++)
202          X.Set(linha, coluna++, m_serie_diferenciada[t - i]);
203       for(int j = 1; j <= m_q; j++)
204          X.Set(linha, coluna++, erros_estimados[t - j]);
205      }
206
207    double coeficientes[], erros_padrao[], soma_quadrados = 0.0;
208    if(!MQO(X, dependente, observacoes, parametros, coeficientes, erros_padrao,
           soma_quadrados, false))
209       return(false);
210
211    int coluna = 0;
212    m_constante = m_com_constante ? coeficientes[coluna++] : 0.0;
213    for(int i = 0; i < m_p; i++)
214       m_phi[i] = coeficientes[coluna++];
215    for(int j = 0; j < m_q; j++)
216       m_theta[j] = coeficientes[coluna++];
217    return(true);
218   }
219
220 void CArima::CalcularResiduos(void)
221   {
222    int tamanho = ArraySize(m_serie_diferenciada);
```

```
223     ArrayResize(m_residuos, tamanho);
224     ArrayInitialize(m_residuos, 0.0);
225
226     m_inicio_recursao = MathMax(m_p, m_q);
227     m_inicio_verossimilhanca = MathMax(m_inicio_recursao, m_aquecimento_minimo);
228
229     for(int t = m_inicio_recursao; t < tamanho; t++)
230       {
231        double ajuste = m_constante;
232        for(int i = 0; i < m_p; i++)
233           ajuste += m_phi[i] * m_serie_diferenciada[t - 1 - i];
234        for(int j = 0; j < m_q; j++)
235           ajuste += m_theta[j] * m_residuos[t - 1 - j];
236        m_residuos[t] = m_serie_diferenciada[t] - ajuste;
237       }
238   }
239
240 void CArima::CalcularVerossimilhanca(void)
241   {
242    int tamanho = ArraySize(m_serie_diferenciada);
243    m_observacoes_efetivas = tamanho - m_inicio_verossimilhanca;
244
245    if(m_observacoes_efetivas <= 0)
246      {
247       m_sigma2 = 0.0;
248       m_log_verossimilhanca = -DBL_MAX;
249       return;
250      }
251
252    double soma_quadrados = 0.0;
253    for(int t = m_inicio_verossimilhanca; t < tamanho; t++)
254       soma_quadrados += m_residuos[t] * m_residuos[t];
255
256    m_sigma2 = soma_quadrados / (double)m_observacoes_efetivas;
257    if(m_sigma2 <= 0.0 || !MathIsValidNumber(m_sigma2))
258      {
259       m_log_verossimilhanca = -DBL_MAX;
260       return;
261      }
262
263    m_log_verossimilhanca = -0.5 * (double)m_observacoes_efetivas *
264                            (MathLog(2.0 * M_PI) + MathLog(m_sigma2) + 1.0);
265   }
266
267 bool CArima::Ajustar(const double &serie[])
268   {
269    m_ajustado = false;
270
271    int n = ArraySize(serie);
272    int minimo = m_p + m_q + m_d + 30;
273    if(n < minimo)
274       return(false);
275
276    ArrayResize(m_ultimos_niveis, MathMin(n, 3));
277    for(int i = 0; i < ArraySize(m_ultimos_niveis); i++)
278       m_ultimos_niveis[i] = serie[n - ArraySize(m_ultimos_niveis) + i];
279
280    Diferenciar(serie, m_serie_diferenciada, m_d);
281    if(ArraySize(m_serie_diferenciada) < m_p + m_q + 20)
282       return(false);
283
284    ArrayResize(m_phi, m_p);
285    ArrayResize(m_theta, m_q);
```

```
286     ArrayInitialize(m_phi, 0.0);
287     ArrayInitialize(m_theta, 0.0);
288     m_constante = 0.0;
289
290     if(!HannanRissanen())
291        return(false);
292
293     for(int i = 0; i < m_p; i++)
294        if(!MathIsValidNumber(m_phi[i]))
295           return(false);
296     for(int j = 0; j < m_q; j++)
297        if(!MathIsValidNumber(m_theta[j]))
298           return(false);
299
300     CalcularResiduos();
301     CalcularVerossimilhanca();
302
303     if(m_log_verossimilhanca == -DBL_MAX)
304        return(false);
305
306     m_ajustado = true;
307     return(true);
308   }
309
310 double CArima::PreverDiferenciado(void)
311   {
312    if(!m_ajustado)
313       return(0.0);
314
315    int tamanho = ArraySize(m_serie_diferenciada);
316    double previsao = m_constante;
317
318    for(int i = 0; i < m_p; i++)
319      {
320       int indice = tamanho - 1 - i;
321       if(indice >= 0)
322          previsao += m_phi[i] * m_serie_diferenciada[indice];
323      }
324    for(int j = 0; j < m_q; j++)
325      {
326       int indice = tamanho - 1 - j;
327       if(indice >= 0)
328          previsao += m_theta[j] * m_residuos[indice];
329      }
330
331    return(previsao);
332   }
333
334 double CArima::PreverNivel(void)
335   {
336    if(!m_ajustado)
337       return(0.0);
338
339    double z = PreverDiferenciado();
340    int n = ArraySize(m_ultimos_niveis);
341
342    if(m_d == 1)
343       return((n >= 1) ? m_ultimos_niveis[n - 1] + z : z);
344    if(m_d == 2 && n >= 2)
345       return(2.0 * m_ultimos_niveis[n - 1] - m_ultimos_niveis[n - 2] + z);
346
347    return(z);
348   }
```

```
349
350 double CArima::AIC(void) const
351   {
352    if(!m_ajustado)
353       return(DBL_MAX);
354    return(CalcularAIC(m_log_verossimilhanca, NumParametros()));
355   }
356
357 bool CArima::ObterResiduos(double &saida[]) const
358   {
359    if(!m_ajustado)
360       return(false);
361
362    int validos = ArraySize(m_residuos) - m_inicio_recursao;
363    if(validos <= 0)
364       return(false);
365
366    ArrayResize(saida, validos);
367    for(int i = 0; i < validos; i++)
368       saida[i] = m_residuos[m_inicio_recursao + i];
369
370    return(true);
371   }
372
373 #endif
```

B.2 Seleção das ordens

A função `DeterminarOrdemD` aplica o teste ADF em sequência para definir a ordem d, e a função `BuscarMelhorPQ` ajusta todas as combinações de p e q de 0 a 4 e fica com a de menor AIC.

```
1  #ifndef ALGOTRADINGLAB_SELECAOMODELO_MQH
2  #define ALGOTRADINGLAB_SELECAOMODELO_MQH
3
4  #include "Arima.mqh"
5  #include "Estatistica.mqh"
6
7  struct ResultadoSelecao
8    {
9     bool     valido;
10    int      p;
11    int      d;
12    int      q;
13   };
14
15 int DeterminarOrdemD(const double &serie[], const int d_maximo)
16   {
17    double atual[];
18    ArrayResize(atual, ArraySize(serie));
19    ArrayCopy(atual, serie);
20
21    for(int d = 0; d <= d_maximo; d++)
22      {
23       ResultadoADF adf;
24       if(!TesteADF(atual, ADF_CONSTANTE, -1, adf))
25          return(-1);
26
27       if(adf.estacionaria)
28          return(d);
```

```
29        if(d == d_maximo)
30           break;
31
32        int tamanho = ArraySize(atual);
33        if(tamanho < 3)
34           break;
35        for(int i = 0; i < tamanho - 1; i++)
36           atual[i] = atual[i + 1] - atual[i];
37        ArrayResize(atual, tamanho - 1);
38       }
39
40     return(d_maximo);
41    }
42
43  bool BuscarMelhorPQ(const double &serie[], const int d, const int p_maximo, const int
       q_maximo,
44                      const bool com_constante, ResultadoSelecao &resultado)
45    {
46     resultado.valido = false;
47     resultado.p = 0;
48     resultado.q = 0;
49     resultado.d = d;
50
51     int aquecimento_comum = MathMax(p_maximo, q_maximo);
52     double melhor_criterio = DBL_MAX;
53
54     for(int p = 0; p <= p_maximo; p++)
55        for(int q = 0; q <= q_maximo; q++)
56          {
57           if(p == 0 && q == 0 && !com_constante && d != 0)
58              continue;
59
60           CArima candidato;
61           candidato.Definir(p, d, q, com_constante);
62           candidato.DefinirAquecimentoMinimo(aquecimento_comum);
63
64           if(!candidato.Ajustar(serie))
65              continue;
66
67           double aic = candidato.AIC();
68           if(!MathIsValidNumber(aic))
69              continue;
70
71           if(aic < melhor_criterio)
72             {
73              melhor_criterio = aic;
74              resultado.p = p;
75              resultado.q = q;
76              resultado.valido = true;
77             }
78          }
79
80     return(resultado.valido);
81    }
82
83  bool SelecionarModelo(const double &serie[], const int d_maximo, const int p_maximo,
       const int q_maximo,
84                        const bool com_constante, ResultadoSelecao &resultado)
85    {
86     int d = DeterminarOrdemD(serie, d_maximo);
87
88     if(d < 0)
89       {
```

```
90        resultado.valido = false;
91        return(false);
92       }
93
94    bool selecionou = BuscarMelhorPQ(serie, d, p_maximo, q_maximo, com_constante,
          resultado);
95    resultado.d = d;
96
97    return(selecionou);
98   }
99
100 #endif
```

B.3 Classe do GARCH

A classe `CGarch` estima os parâmetros do GARCH por máxima verossimilhança, com as restrições passadas ao otimizador da biblioteca ALGLIB, e prevê a volatilidade do período seguinte.

```
1  #ifndef ALGOTRADINGLAB_GARCH_MQH
2  #define ALGOTRADINGLAB_GARCH_MQH
3
4  #include <Math\Alglib\alglib.mqh>
5
6  double LogDensidadeNormal(const double erro_quadrado, const double sigma2)
7    {
8     return(-0.5 * (MathLog(2.0 * M_PI) + MathLog(sigma2) + erro_quadrado / sigma2));
9    }
10
11 class CDadosGarch : public CObject
12   {
13 public:
14    double            erros_quadrado[];
15    int               tamanho;
16    int               p;
17    int               q;
18    int               inicio;
19    double            variancia_amostral;
20
21                      CDadosGarch(void) : tamanho(0), p(1), q(1), inicio(1),
                            variancia_amostral(0.0) {}
22   };
23
24 double MenosLogVerossimilhancaGarch(CDadosGarch &dados, const double &parametros[])
25   {
26    int n = dados.tamanho;
27    int p = dados.p;
28    int q = dados.q;
29
30    if(n <= dados.inicio || ArraySize(parametros) < 1 + p + q)
31       return(1.0e10);
32
33    double omega = parametros[0] * dados.variancia_amostral;
34    if(omega <= 0.0 || !MathIsValidNumber(omega))
35       return(1.0e10);
36
37    double persistencia = 0.0;
38    for(int i = 1; i <= p + q; i++)
39      {
40       if(parametros[i] < 0.0 || !MathIsValidNumber(parametros[i]))
```

```
41          return(1.0e10);
42        persistencia += parametros[i];
43       }
44    if(persistencia >= 1.0)
45       return(1.0e10);
46
47    double sigma2[];
48    ArrayResize(sigma2, n);
49    for(int t = 0; t < dados.inicio; t++)
50       sigma2[t] = dados.variancia_amostral;
51
52    double resultado = 0.0;
53    for(int t = dados.inicio; t < n; t++)
54      {
55       double variancia = omega;
56       for(int i = 1; i <= p; i++)
57          variancia += parametros[i] * dados.erros_quadrado[t - i];
58       for(int j = 1; j <= q; j++)
59          variancia += parametros[p + j] * sigma2[t - j];
60
61       if(variancia <= 0.0 || !MathIsValidNumber(variancia))
62          return(1.0e10);
63       sigma2[t] = variancia;
64
65       resultado -= LogDensidadeNormal(dados.erros_quadrado[t], variancia);
66      }
67
68    return(MathIsValidNumber(resultado) ? resultado : 1.0e10);
69   }
70
71 class CObjetivoGarch : public CNDimensional_Func
72   {
73 public:
74    virtual void      Func(CRowDouble &x, double &func, CObject &obj);
75   };
76
77 void CObjetivoGarch::Func(CRowDouble &x, double &func, CObject &obj)
78   {
79    CDadosGarch *dados = dynamic_cast<CDadosGarch *>(GetPointer(obj));
80    if(dados == NULL)
81      {
82       func = 1.0e10;
83       return;
84      }
85
86    int k = x.Size();
87    double parametros[];
88    ArrayResize(parametros, k);
89    for(int i = 0; i < k; i++)
90       parametros[i] = x[i];
91
92    func = MenosLogVerossimilhancaGarch(dados, parametros);
93   }
94
95 class CGarch
96   {
97 private:
98    int       m_p;
99    int       m_q;
100   double    m_omega;
101   double    m_alfa[];
102   double    m_beta[];
103
```

```
104    double   m_erros_quadrado[];
105    double   m_sigma2[];
106    double   m_variancia_amostral;
107    int      m_inicio;
108    int      m_tamanho;
109    bool     m_ajustado;
110
111    int      m_iteracoes_maximas;
112    double   m_passo_diferencas;
113
114    void     RecalcularVariancias(void);
115
116 public:
117                     CGarch(void);
118
119    void     Definir(const int p, const int q);
120    void     DefinirOtimizador(const int iteracoes_maximas, const double passo_diferencas
           = 1.0e-6);
121
122    bool     Ajustar(const double &residuos[]);
123
124    double   VolatilidadeProximoPeriodo(void) const;
125    bool     ObterSerieVolatilidade(double &saida[]) const;
126   };
127
128 CGarch::CGarch(void) : m_p(1), m_q(1), m_omega(0.0), m_variancia_amostral(0.0),
129                        m_inicio(1), m_tamanho(0), m_ajustado(false),
130                        m_iteracoes_maximas(200), m_passo_diferencas(1.0e-6)
131   {
132    ArrayResize(m_alfa, 1);
133    ArrayResize(m_beta, 1);
134    ArrayInitialize(m_alfa, 0.0);
135    ArrayInitialize(m_beta, 0.0);
136   }
137
138 void CGarch::Definir(const int p, const int q)
139   {
140    m_p = MathMax(1, p);
141    m_q = MathMax(0, q);
142    ArrayResize(m_alfa, m_p);
143    ArrayResize(m_beta, MathMax(1, m_q));
144    ArrayInitialize(m_alfa, 0.0);
145    ArrayInitialize(m_beta, 0.0);
146    m_ajustado = false;
147   }
148
149 void CGarch::DefinirOtimizador(const int iteracoes_maximas, const double passo_diferencas
       )
150   {
151    m_iteracoes_maximas = MathMax(10, iteracoes_maximas);
152    m_passo_diferencas = (passo_diferencas > 0.0) ? passo_diferencas : 1.0e-6;
153   }
154
155 bool CGarch::Ajustar(const double &residuos[])
156   {
157    m_ajustado = false;
158
159    int n = ArraySize(residuos);
160    int minimo = m_p + m_q + 30;
161    if(n < minimo)
162       return(false);
163
164    ArrayResize(m_erros_quadrado, n);
```

```
165     for(int i = 0; i < n; i++)
166        m_erros_quadrado[i] = residuos[i] * residuos[i];
167
168     m_tamanho = n;
169     m_inicio = MathMax(1, MathMax(m_p, m_q));
170
171     double soma = 0.0;
172     for(int i = 0; i < n; i++)
173        soma += m_erros_quadrado[i];
174     m_variancia_amostral = soma / (double)n;
175     if(m_variancia_amostral <= 0.0 || !MathIsValidNumber(m_variancia_amostral))
176        return(false);
177
178     CDadosGarch dados;
179     ArrayResize(dados.erros_quadrado, n);
180     ArrayCopy(dados.erros_quadrado, m_erros_quadrado);
181     dados.tamanho = n;
182     dados.p = m_p;
183     dados.q = m_q;
184     dados.inicio = m_inicio;
185     dados.variancia_amostral = m_variancia_amostral;
186
187     int k = 1 + m_p + m_q;
188
189     double x[];
190     ArrayResize(x, k);
191     x[0] = 0.05;
192     for(int i = 1; i <= m_p; i++)
193        x[i] = 0.10 / (double)m_p;
194     for(int j = 1; j <= m_q; j++)
195        x[m_p + j] = 0.85 / (double)m_q;
196
197     double limite_inferior[], limite_superior[];
198     ArrayResize(limite_inferior, k);
199     ArrayResize(limite_superior, k);
200     limite_inferior[0] = 1.0e-8;
201     limite_superior[0] = 10.0;
202     for(int i = 1; i < k; i++)
203       {
204        limite_inferior[i] = 0.0;
205        limite_superior[i] = 1.0;
206       }
207
208     CMatrixDouble restricao(1, k + 1);
209     for(int i = 0; i <= k; i++)
210        restricao.Set(0, i, 0.0);
211     for(int i = 1; i < k; i++)
212        restricao.Set(0, i, 1.0);
213     restricao.Set(0, k, 0.9999);
214     int tipo_restricao[];
215     ArrayResize(tipo_restricao, 1);
216     tipo_restricao[0] = -1;
217
218     CMinBLEICStateShell estado;
219     CMinBLEICReportShell relatorio;
220     CObjetivoGarch objetivo;
221     CNDimensional_Rep repositor;
222
223     CAlglib::MinBLEICCreateF(k, x, m_passo_diferencas, estado);
224     CAlglib::MinBLEICSetBC(estado, limite_inferior, limite_superior);
225     CAlglib::MinBLEICSetLC(estado, restricao, tipo_restricao, 1);
226     CAlglib::MinBLEICSetInnerCond(estado, 0.0, 0.0, 1.0e-8);
227     CAlglib::MinBLEICSetOuterCond(estado, 1.0e-8, 1.0e-8);
```

```
228     CAlglib::MinBLEICSetMaxIts(estado, m_iteracoes_maximas);
229     CAlglib::MinBLEICOptimize(estado, objetivo, repositor, false, dados);
230
231     double x_final[];
232     CAlglib::MinBLEICResults(estado, x_final, relatorio);
233     int codigo = relatorio.GetTerminationType();
234     if(codigo <= 0 || ArraySize(x_final) < k)
235        return(false);
236
237     m_omega = x_final[0] * m_variancia_amostral;
238     ArrayResize(m_alfa, m_p);
239     for(int i = 0; i < m_p; i++)
240        m_alfa[i] = x_final[1 + i];
241     ArrayResize(m_beta, MathMax(1, m_q));
242     ArrayInitialize(m_beta, 0.0);
243     for(int j = 0; j < m_q; j++)
244        m_beta[j] = x_final[1 + m_p + j];
245
246     if(m_omega <= 0.0 || !MathIsValidNumber(m_omega))
247        return(false);
248
249     RecalcularVariancias();
250     m_ajustado = true;
251     return(true);
252   }
253
254 void CGarch::RecalcularVariancias(void)
255   {
256    ArrayResize(m_sigma2, m_tamanho);
257    for(int t = 0; t < m_inicio; t++)
258       m_sigma2[t] = m_variancia_amostral;
259
260    for(int t = m_inicio; t < m_tamanho; t++)
261      {
262       double variancia = m_omega;
263       for(int i = 1; i <= m_p; i++)
264          variancia += m_alfa[i - 1] * m_erros_quadrado[t - i];
265       for(int j = 1; j <= m_q; j++)
266          variancia += m_beta[j - 1] * m_sigma2[t - j];
267       m_sigma2[t] = MathMax(variancia, 1.0e-300);
268      }
269   }
270
271 double CGarch::VolatilidadeProximoPeriodo(void) const
272   {
273    if(!m_ajustado)
274       return(0.0);
275
276    double variancia = m_omega;
277    for(int i = 1; i <= m_p; i++)
278       variancia += m_alfa[i - 1] * m_erros_quadrado[m_tamanho - i];
279    for(int j = 1; j <= m_q; j++)
280       variancia += m_beta[j - 1] * m_sigma2[m_tamanho - j];
281
282    return(MathSqrt(MathMax(variancia, 1.0e-300)));
283   }
284
285 bool CGarch::ObterSerieVolatilidade(double &saida[]) const
286   {
287    if(!m_ajustado)
288       return(false);
289
290    int n = ArraySize(m_sigma2) - m_inicio;
```

```
291    if(n <= 0)
292       return(false);
293
294    ArrayResize(saida, n);
295    for(int i = 0; i < n; i++)
296       saida[i] = MathSqrt(m_sigma2[m_inicio + i]);
297    return(true);
298   }
299
300 #endif
```

B.4 Decisão de operação

A função `DecidirOperacao` recebe a previsão do ARIMA, que dá a direção, e a volatilidade prevista pelo GARCH, que pode vetar a operação no filtro ou mudar o tamanho da posição no ajuste.

```
1  #ifndef ALGOTRADINGLAB_ESTRATEGIA_MQH
2  #define ALGOTRADINGLAB_ESTRATEGIA_MQH
3
4  enum ENUM_ESTRATEGIA_GARCH
5    {
6     FILTRO_VOLATILIDADE = 0,
7     VOLATILITY_TARGETING = 1,
8     FILTRO_E_TARGETING = 3
9    };
10
11 enum ENUM_MODO_LIMIAR
12   {
13    LIMIAR_ABSOLUTO = 0,
14    LIMIAR_PERCENTIL = 1
15   };
16
17 struct DecisaoOperacao
18   {
19    int      sinal;
20    double   volume;
21    bool     vetada_pelo_garch;
22   };
23
24 void DecidirOperacao(const double previsao,
25                      const double volatilidade,
26                      const bool usar_garch,
27                      const ENUM_ESTRATEGIA_GARCH estrategia,
28                      const double limiar,
29                      const double volatilidade_alvo,
30                      const double volume_base,
31                      const double fracao_minima,
32                      const double fracao_maxima,
33                      const double limiar_sinal,
34                      DecisaoOperacao &decisao)
35   {
36    decisao.sinal = 0;
37    decisao.volume = volume_base;
38    decisao.vetada_pelo_garch = false;
39
40    int direcao = 0;
41    if(previsao > limiar_sinal)
42       direcao = 1;
43    else if(previsao < -limiar_sinal)
```

```
44        direcao = -1;
45
46     if(direcao == 0)
47        return;
48
49     if(!usar_garch)
50       {
51        decisao.sinal = direcao;
52        return;
53       }
54
55     if(volatilidade <= 0.0 || !MathIsValidNumber(volatilidade))
56       {
57        decisao.vetada_pelo_garch = true;
58        return;
59       }
60
61     bool filtra = (estrategia == FILTRO_VOLATILIDADE || estrategia == FILTRO_E_TARGETING);
62     if(filtra && volatilidade > limiar)
63       {
64        decisao.vetada_pelo_garch = true;
65        return;
66       }
67
68     bool dimensiona = (estrategia == VOLATILITY_TARGETING || estrategia ==
           FILTRO_E_TARGETING);
69     if(dimensiona)
70       {
71        double fator = MathMax(fracao_minima, MathMin(fracao_maxima, volatilidade_alvo /
              volatilidade));
72        decisao.volume = volume_base * fator;
73        decisao.sinal = direcao;
74        return;
75       }
76
77     decisao.sinal = direcao;
78    }
79
80  #endif
```

APÊNDICE C – Código do robô de médias móveis

Este apêndice apresenta o código do robô de médias móveis descrito no Capítulo II, que reproduz a estratégia de Almeida (2016). Os parâmetros das médias mantêm os nomes do anexo original, `RapidaMM_Periodo` e `DevagarMM_Periodo`. Como nos Apêndices A e B, foram retirados os comentários, as rotinas de medição dos resultados e as opções que não foram usadas nos experimentos, como os limites de perda e de realização de lucro, que o autor deixou com valor zero nos testes.

```
#property copyright "TG - FATEC Indaiatuba (ADS)"
#property version   "2.00"

#include "..\Include\Negociacao.mqh"

input int                RapidaMM_Periodo   = 10;
input int                DevagarMM_Periodo  = 20;

input double             Volume_Fixo_Input  = 1.0;
input ulong              MagicNumber        = 12349;
input int                Slippage_Points    = 10;
input double             Fracao_Capital     = 0.0;
input bool               SomenteCompra      = false;

CNegociacao g_negociacao;

int      g_media_rapida;
int      g_media_lenta;
double   g_volume_fixo;

int OnInit()
  {
   g_media_rapida = iMA(_Symbol, _Period, RapidaMM_Periodo, 0, MODE_SMA, PRICE_CLOSE);
   g_media_lenta  = iMA(_Symbol, _Period, DevagarMM_Periodo, 0, MODE_SMA, PRICE_CLOSE);
   if(g_media_rapida == INVALID_HANDLE || g_media_lenta == INVALID_HANDLE)
      return(INIT_FAILED);

   g_negociacao.Configurar(_Symbol, MagicNumber, Slippage_Points);
   g_volume_fixo = VolumeFixoDoSimbolo(Volume_Fixo_Input);

   return(INIT_SUCCEEDED);
  }

void OnDeinit(const int reason)
  {
   IndicatorRelease(g_media_rapida);
   IndicatorRelease(g_media_lenta);
  }

double VolumeFixoDoSimbolo(double volume)
  {
   double minimo = SymbolInfoDouble(_Symbol, SYMBOL_VOLUME_MIN);
   double maximo = SymbolInfoDouble(_Symbol, SYMBOL_VOLUME_MAX);
   double passo  = SymbolInfoDouble(_Symbol, SYMBOL_VOLUME_STEP);

   if(volume < minimo) volume = minimo;
   if(volume > maximo) volume = maximo;
   if(passo > 0 && MathAbs(MathMod(volume, passo)) > 1e-10)
     {
      volume = MathRound(volume / passo) * passo;
```

```
51        volume = NormalizeDouble(volume, (passo < 1.0) ? 8 : 0);
52       }
53     if(volume < minimo) volume = minimo;
54     return(volume);
55    }
56
57  double VolumeDaOrdem(const double preco)
58    {
59     if(Fracao_Capital <= 0.0)
60        return(g_volume_fixo);
61     return(VolumePorCapital(_Symbol, Fracao_Capital, preco));
62    }
63
64  void OnTick()
65    {
66     datetime barra = (datetime)SeriesInfoInteger(_Symbol, _Period, SERIES_LASTBAR_DATE);
67     static datetime barra_operada = 0;
68     if(barra == barra_operada)
69        return;
70     barra_operada = barra;
71
72     double valor[1];
73     if(CopyBuffer(g_media_rapida, 0, 1, 1, valor) != 1) return;
74     double rapida_agora = valor[0];
75     if(CopyBuffer(g_media_lenta, 0, 1, 1, valor) != 1) return;
76     double lenta_agora = valor[0];
77     if(CopyBuffer(g_media_rapida, 0, 2, 1, valor) != 1) return;
78     double rapida_antes = valor[0];
79     if(CopyBuffer(g_media_lenta, 0, 2, 1, valor) != 1) return;
80     double lenta_antes = valor[0];
81
82     MqlTick tick;
83     if(!SymbolInfoTick(_Symbol, tick) || tick.ask == 0 || tick.bid == 0)
84        return;
85
86     ENUM_POSITION_TYPE posicao = g_negociacao.PosicaoAtual();
87
88     if(rapida_agora > lenta_agora && rapida_antes <= lenta_antes)
89       {
90        if(posicao == POSITION_TYPE_SELL)
91          {
92           if(!g_negociacao.Fechar(POSITION_TYPE_SELL))
93              return;
94           Sleep(1);
95           posicao = g_negociacao.PosicaoAtual();
96          }
97        if(posicao != POSITION_TYPE_BUY)
98          {
99           double preco = tick.ask;
100          g_negociacao.Comprar(VolumeDaOrdem(preco), preco, 0, 0, "Compra MM Rev");
101         }
102       return;
103      }
104
105    if(rapida_agora < lenta_agora && rapida_antes >= lenta_antes)
106      {
107       if(posicao == POSITION_TYPE_BUY)
108         {
109          if(!g_negociacao.Fechar(POSITION_TYPE_BUY))
110             return;
111          Sleep(1);
112          posicao = g_negociacao.PosicaoAtual();
113         }
```

```
114       if(SomenteCompra)
115          return;
116       if(posicao != POSITION_TYPE_SELL)
117         {
118          double preco = tick.bid;
119          g_negociacao.Vender(VolumeDaOrdem(preco), preco, 0, 0, "Venda MM Rev");
120         }
121      }
122   }
```